

Midnight-Native Secure Onboarding Design

LFDT-Nightstream

March 2026

Contents

Midnight-Native Secure Onboarding Design	3
1. Executive Summary	3
2. Design Principles	4
2.1 Keys Never Leave TEEs; Authorization via ZK Witnesses	4
2.2 Single Seed, Three Wallets	5
2.3 Chain Abstraction via Intent Model	5
2.4 One Key Per Device	6
2.5 Privacy-by-Design Identity	6
2.6 Seedless User Experience	7
3. System Architecture	8
3.1 Six-Layer Architecture Overview	8
3.2 Layer Interactions	10
3.3 Data Flow During Onboarding	10
4. Human-Readable Naming System	11
4.1 Architecture: Registry and Resolver in Compact	11
4.2 Namehash Using persistentHash	12
4.3 Name Format	12
4.4 Commit-Reveal Registration Flow	12
4.5 Multi-Address Resolution	14
4.6 Privacy-Aware Resolution	15
4.7 Offchain Subdomains: Wildcard and CCIP-Read	16
4.8 Profile Records	16
4.9 Name Wrapper with Fuses	17
4.10 Integration with Onboarding Flow	18
4.11 Name Resolution Flow	18
4.12 Comparison: ENS vs NEAR Named Accounts vs Midnight Naming	19
4.13 Name Registration Anti-Abuse	20
5. Midnight Platform Integration	24
5.1 Curves: BLS12-381 and Jubjub	24
5.2 Three Wallet Address Types	24
5.3 Bech32m mn_ Addresses	25
5.4 Witness-Based ZK Authorization	25
5.5 Compact Circuit Token Operations	26
5.6 DUST Fee Model	27

5.7 Private State Encryption	27
5.8 Proof Pipeline	28
5.9 dApp-Wallet Connection Standard (Midnight Connection Protocol) . .	30
5.10 SDK Onboarding Wrapper	37
5.11 Alternative Onboarding Entry Points	42
6. QR Code Onboarding Flow	44
6.1 Protocol Overview	44
6.2 Detailed Sequence Diagram	45
6.3 Phase Details	48
6.4 Second-Device Linking via QR	51
6.5 Onboarding Resilience (Offline / Degraded Network)	51
7. Account Model	53
7.1 Named Accounts	53
7.2 Key Types	54
7.3 Device Addition and Removal	55
7.4 Comparison with NEAR	56
8. Identity and Credential Layer	56
8.1 Midnight's Native Attestation Trees	56
8.2 Multi-Attribute Attestation Trees	57
8.3 zkMe as Off-Chain Issuer	57
8.4 Nullifier-Based Reuse Prevention	58
8.5 Privacy Guarantees	58
8.6 Attestation Tree Verification Flow	59
8.7 Credential Lifecycle Management	60
9. Key Management	63
9.1 Derivation Architecture	63
9.2 CIP-1852 Derivation Tree	64
9.3 TEE Wrapping Architecture	65
9.4 Signing Operations	66
9.5 Chain-Level Key Caching	66
9.6 Multi-Account and Identity Switching	67
10. Recovery Design	68
10.1 DeRec Seed Recovery	68
10.2 Recovery Flow	69
10.3 Why Credentials Survive Recovery	71
10.4 Multi-Key Account Recovery (Device-Level)	71
10.5 Institutional Recovery	72
10.6 Emergency Procedures	74
11. Security Model	79
11.1 Threat Model	79
11.2 Findings Summary	79
11.3 Trust Boundary Diagram	81
11.4 Critical Attack Vectors and Mitigations	82
12. Cross-Platform Considerations	83
12.1 Platform TEE Comparison	83
12.2 Unified Wrapping Strategy	84
12.3 Platform-Specific Notes	84
12.4 Cross-Platform Recovery	84

12.5 Privacy-Preserving Analytics	85
13. Appendix A — Security Review Summary	86
13.1 Cryptographic Findings	87
13.2 Protocol Findings	88
14. Appendix B — Research Sources	92
14.1 Research Stream Index	92
14.2 Primary References	93
14.3 Standards References	94
15. Appendix C – Customer Journey Map: Newcomer Nadia	95
15.1 Journey Stages	95
15.2 Emotion Curve	101
15.3 Critical Insights	102
15.4 Design Recommendations Summary	102

Midnight-Native Secure Onboarding Design

Status: Draft v0.1 **Date:** 2026-03-29 **Scope:** Complete Document (Sections 1-15)

1. Executive Summary

This document specifies a Midnight-native onboarding system that takes a new user from zero state to full platform access in one continuous flow. The user scans a QR code. Behind the scenes, a server-authenticated ECDH channel is established, a 256-bit BIP39 seed is generated inside the device’s Trusted Execution Environment, and three wallet layers – Shielded (ZswapCoinPublicKey), Night (UserAddress), and Dust – are derived via CIP-1852 hierarchical deterministic derivation. A human-readable name (username.midnight) is registered through a commit-reveal Compact circuit, an initial NIGHT airdrop subsidized by the onboarding provider starts DUST fee generation, and identity credentials are issued as attestation tree leaves anchored to on-chain Merkle roots. At the end, the user has a named account with full-access keys, privacy-preserving identity credentials, and can transact across all three token types on Midnight – without ever seeing a mnemonic phrase, understanding a cryptographic curve, or managing a raw address.

The design draws on eight investigation streams:

1. **CAKE (Chain Abstraction Key Encapsulation)** – Intent-based cross-chain transactions and MPC chain signatures for multi-chain access from a single account.
2. **OpenWallet Foundation** – Verifiable credential issuance, holder-controlled selective disclosure, and wallet interoperability standards.
3. **DeRec (Decentralized Recovery)** – Shamir secret sharing with Merkle commitments, helper-based seed recovery with (3,5) threshold, and ML-KEM post-quantum transport.

4. **NEAR Protocol** – Named accounts decoupled from keys, multi-key account model with full-access and function-call key tiers, and device-centric key management.
5. **zkMe** – zkKYC verification with Groth16 zk-SNARK proofs, FHE biometric processing, Soulbound Token credential anchoring, and zero-knowledge identity assertions.
6. **ENS (Ethereum Name Service)** – Namehash algorithms, registry/resolver architecture, CCIP-Read offchain resolution, Name Wrapper with fuse-based permission control, and wildcard subdomain delegation.
7. **Trusted Execution Environments** – iOS Secure Enclave (P-256 only), Android StrongBox, ARM CCA, AES-256-GCM key wrapping for non-native curve material, and hardware-backed attestation.
8. **Midnight Platform Documentation** – BLS12-381/Jubjub curves, Compact circuit language, ZKIR constraint systems, witness-based ZK authorization, three-layer wallet architecture, Poseidon-based persistentHash, attestation trees, and the SDK transaction pipeline.

Every component maps directly to Midnight’s actual architecture. Authorization uses ZK witnesses natively – the same mechanism Compact circuits already use for private inputs. Identity verification uses Midnight’s attestation tree pattern – Merkle roots on-chain, leaves off-chain, Poseidon domain-separated hashing. Fee subsidization uses the DUST regeneration model – non-transferable fee tokens that regenerate from NIGHT holdings. The naming system is a Compact contract, not bolted on as an external service. This is not a generic blockchain wallet design adapted for Midnight; it is Midnight-native, exploiting the platform’s unique properties at every layer.

The target experience: scan a QR code, wait approximately 40-60 seconds (dominated by ZK proof generation for name registration and initial credential issuance), and land on a fully functional identity with a human-readable name, privacy-preserving credentials, and the ability to send and receive both shielded and unshielded tokens. No seed phrases. No address copying. No gas token purchases. The complexity is real – BLS12-381 key derivation, Poseidon hash trees, PLONK proofs – but it is hidden entirely behind that single QR scan.

2. Design Principles

2.1 Keys Never Leave TEEs; Authorization via ZK Witnesses

The core security rule: cryptographic key material never exists in exportable form outside a Trusted Execution Environment. The BIP39 seed is generated inside the TEE. It is encrypted at rest using an AES-256-GCM wrapping key that is itself bound to the TEE hardware – on iOS, this is a Secure Enclave P-256 key used solely for symmetric wrapping; on Android StrongBox, the same AES wrapping approach is used for cross-platform consistency. When a signing operation is needed, the seed is decrypted into application memory for the minimum duration required, used to derive the specific key needed, and then zeroized.

This fits naturally with Midnight’s witness-based authorization model. In Compact cir-

cuits, private inputs are supplied through witness declarations – functions that execute off-chain and feed secret values into the ZK proof. A typical authorization pattern: the circuit takes a witness-provided secret key, hashes it with persistentHash using a domain separator, and compares the result to a public ledger value. The ZK proof demonstrates knowledge of the key without revealing it. The TEE model maps directly: the key is derived in the TEE, passed as a witness value to the local proof generation, and never touches the network. The proof server runs locally – witnesses never leave the device. The blockchain node verifies the proof but never sees the witness. This is not a compromise; it is the native way Midnight contracts authorize operations.

In practice, compromising the user’s account requires either physical access to the TEE hardware or a break in the ZK proof system (BLS12-381 discrete log or Poseidon preimage resistance). Network-level attacks, server compromises, and phishing cannot extract key material because it is never transmitted. The only data that crosses a trust boundary is a zero-knowledge proof, which by definition reveals nothing about its witness inputs.

2.2 Single Seed, Three Wallets

A single BIP39 mnemonic (256-bit entropy, 24 words – though the words are never shown to the user) is the root of all key material. Through BIP32 hierarchical deterministic derivation following Midnight’s CIP-1852 path (`m/44'/2400'/{account}'/{role}/{index}`), this single seed deterministically produces keys for all three wallet layers: Shielded tokens via the Zswap role (index 3), Night tokens via NightExternal (index 0) and NightInternal (index 1), and Dust fee tokens via the Dust role (index 2). A fifth role, Metadata (index 4), handles encryption keys for private contract state.

This single-seed design simplifies recovery considerably. If the seed is recovered (via DeRec helper reconstruction or backup), all wallet layers are automatically restored. There is no need to separately back up shielded keys, Night keys, and Dust keys – they are all deterministic functions of the same root. Identity credentials also survive recovery: the user’s secret key that forms the leaf of attestation trees is derived from the same seed, so the Merkle proofs remain valid against the unchanged on-chain roots. The only state that must be separately restored is encrypted private contract state (AES-256-GCM encrypted local storage per contract), which can be re-imported from an encrypted backup or re-synced from collaborative contract partners.

Coin type 2400 is Midnight’s registered BIP44 identifier, so there is no collision with other blockchain HD derivation paths. Each account index produces a completely independent set of addresses, allowing multiple personas from a single seed – though the default onboarding flow creates a single account (index 0) to keep things simple.

2.3 Chain Abstraction via Intent Model

Users should not need to know which chain they are interacting with. The CAKE-inspired intent model puts a declarative interface over cross-chain operations: the user says what they want (“send 100 USDC to alice.midnight”), and the system figures out how to execute it. If the operation requires bridging assets from Ethereum

or touching a Cardano-native token, the chain abstraction layer handles routing, fee estimation, and execution without exposing chain-specific details.

On Midnight specifically, chain abstraction addresses the dual-address complexity. A single `username.midnight` name resolves to multiple address types: a `ZswapCoin-PublicKey` for shielded operations, a `UserAddress` for Night transfers, a Dust address for fee registration, and optionally cross-chain addresses (BTC, ETH, SOL) via SLIP-44 coin type resolution. The naming system's multi-address resolver determines which address type to use based on the operation context. Sending shielded tokens? The resolver returns the `ZswapCoinPublicKey`. Sending Night? It returns the `UserAddress`. The user sees neither address – they see `alice.midnight`.

The cross-chain component uses MPC (Multi-Party Computation) chain signatures, inspired by NEAR's chain signature protocol. Instead of managing separate private keys for each external chain, the user's Midnight account can authorize cross-chain transactions through a threshold MPC signing ceremony. The user's TEE-held key contributes a signature share, combined with shares from MPC network nodes, to produce a valid signature on the target chain. No bridge contracts or wrapped tokens needed for simple cross-chain transfers.

2.4 One Key Per Device

Following NEAR's multi-key account model, each device holds its own independent key pair. The account identity (the `username.midnight` name) is decoupled from any individual key. Adding a new device means generating a fresh seed in that device's TEE and registering the new device's public key as an additional full-access key on the account's on-chain record. Revoking a compromised device: any remaining full-access key can remove the compromised key via a circuit call.

This eliminates the catastrophic failure mode of single-key accounts. Losing one device does not mean losing the account – any other device with a registered full-access key can continue operating. It also removes the need to transfer key material between devices (a dangerous operation that would require decrypting the seed from one TEE, transmitting it, and re-encrypting in another TEE). Each device generates its own key material and registers it with the shared account.

Function-call keys extend this for dApp authorization. When a user grants a dApp permission to perform specific operations, a scoped function-call key is derived and registered. This key can only invoke specific Compact circuits and has a DUST spending allowance. If the dApp is compromised, damage is limited to the scope of its function-call key – the user's full-access keys and other dApp authorizations are unaffected. Think of it as OAuth scopes, but enforced at the blockchain protocol level through Compact circuit authorization checks.

2.5 Privacy-by-Design Identity

Identity verification has to work without creating a surveillance infrastructure. The zkMe-inspired zkKYC model flips the traditional KYC pattern: instead of every service collecting and storing PII, a single trusted issuer (zkMe) verifies identity once and issues

cryptographic attestations. These attestations are encoded as Midnight attestation tree leaves – `persistentHash([domain_separator, user_secret_key])` – with the Merkle root published to a Compact contract. The user holds their secret key and Merkle proof locally.

When a service needs identity verification (age ≥ 18 , citizenship, accredited investor status), the user provides a ZK proof of Merkle tree membership. The service's Compact circuit verifies the proof and records a nullifier (a domain-separated hash of the user's secret key) to prevent credential reuse. The service learns exactly one bit of information: eligible or not. It does not learn the user's name, age, address, or even which leaf in the tree the user occupies. Domain separation between the attestation leaf hash and the nullifier hash ensures that even the nullifier cannot be linked back to the attestation.

This is not theoretical – it is the actual pattern from Midnight's developer guide (Chapter 11: Selective Disclosure). The Compact compiler's disclosure analysis enforces this at compile time: if a circuit accidentally leaks witness data through the public transcript, the compiler rejects it. Privacy is a compiler guarantee, not a developer discipline. The design extends this with multi-attribute attestation trees (separate roots for age, citizenship, investor status, etc.) and Soulbound Token anchoring (non-transferable on-chain tokens that signal credential status without revealing details).

2.6 Seedless User Experience

The user never sees a mnemonic phrase. The 24-word BIP39 mnemonic exists internally as the deterministic derivation root, but it is generated inside the TEE, encrypted immediately with the AES-256-GCM wrapping key, and never displayed. Recovery goes through the DeRec protocol: the raw seed is split via (3,5) Shamir secret sharing, with encrypted shares distributed to designated helpers. The user recovers their account by contacting any 3 of 5 helpers, not by typing 24 words from a paper backup stored in a filing cabinet.

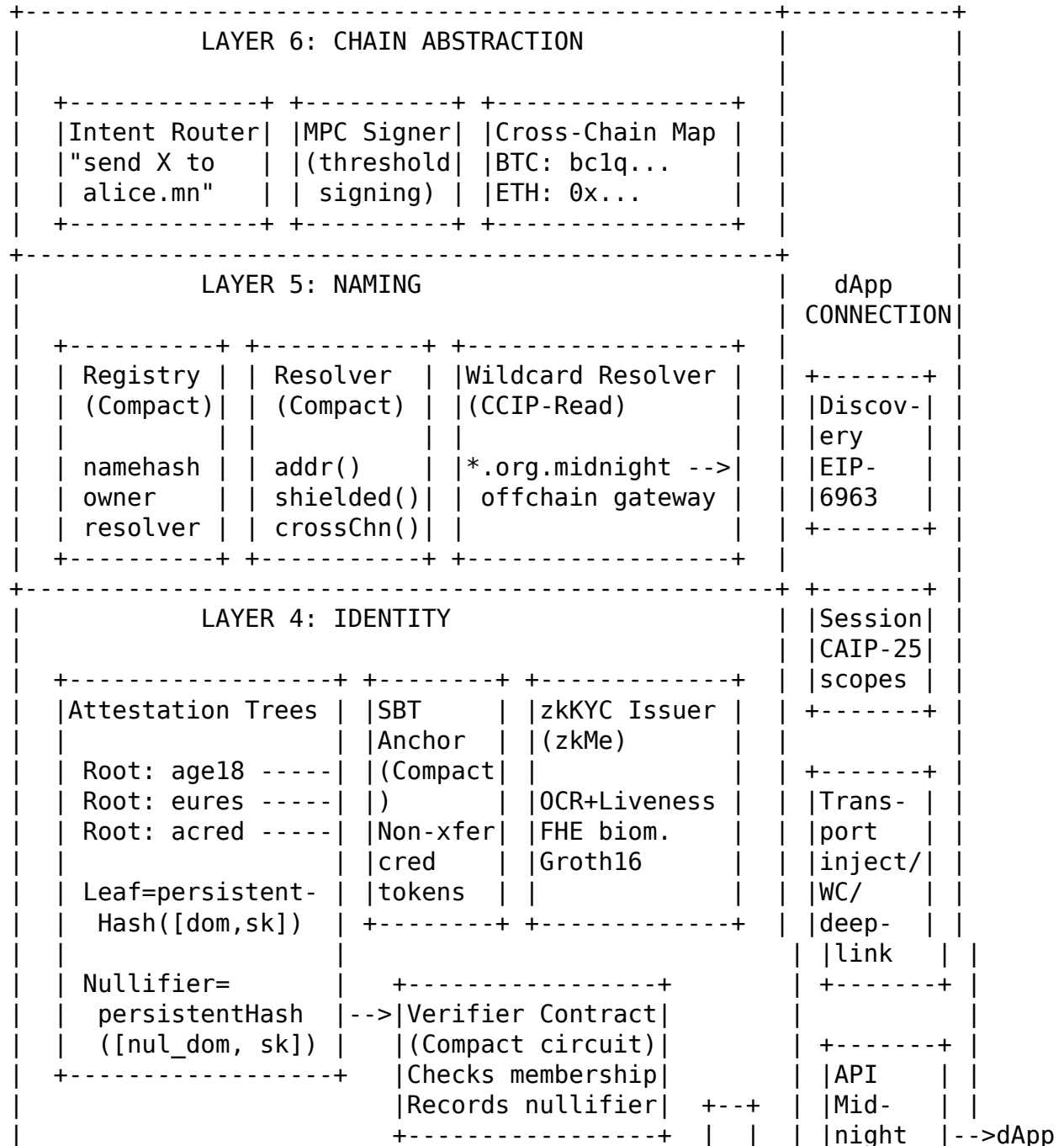
This eliminates the single largest source of user error and security failure in cryptocurrency: mnemonic management. Users write mnemonics on paper that gets lost. They store them in cloud notes that get compromised. They enter them into phishing sites. They fail to back them up at all. The seedless approach removes all of these failure modes by taking the mnemonic out of the user's awareness entirely. The seed exists, the derivation is standard, and an expert user could theoretically extract and manage it – but the default path never exposes it.

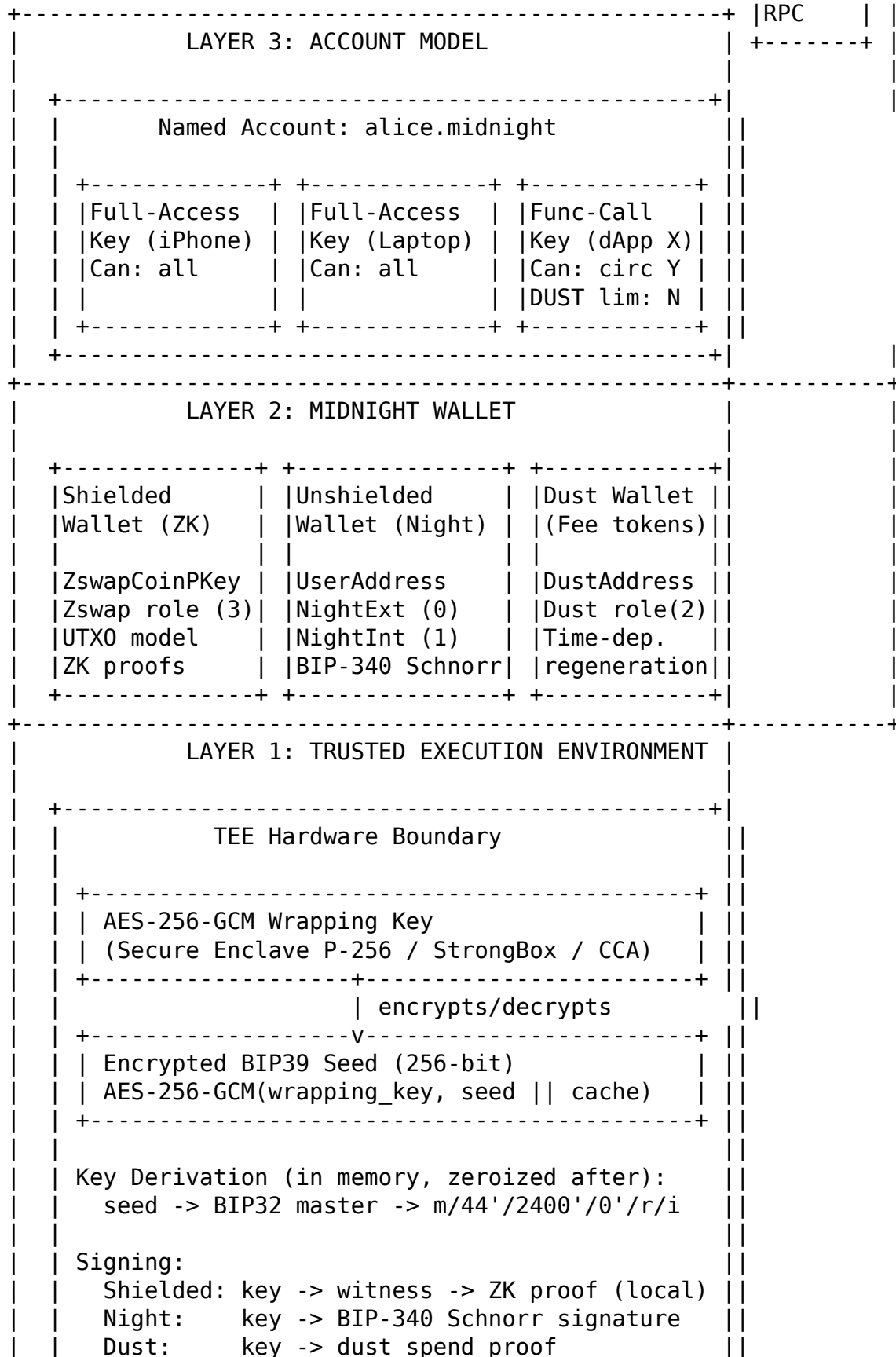
The tradeoff is a dependency on the DeRec helper network for recovery. If all five helpers are simultaneously unavailable and the user has lost all devices, the account is unrecoverable. This is mitigated by helper selection guidance (choose geographically and socially diverse helpers), daily challenge-response verification (detect helper unavailability early), and the option to increase the total helper count while maintaining the threshold. The system also supports BIP-85 compartmentalization as an escape hatch: an expert user can derive a child mnemonic from the hidden root for external backup, accepting the security tradeoffs of mnemonic exposure in exchange for an additional recovery path.

3. System Architecture

3.1 Six-Layer Architecture Overview

The onboarding system has six layers, each building on the layer below. These are not just conceptual – they map to distinct trust domains, cryptographic primitives, and Compact contracts.





```
|  +-----+
+-----+
```

The cross-cutting **dApp Connection** surface on the right spans Layers 2-5, representing the four-stage pipeline through which external dApps interact with the wallet: **Discovery** (EIP-6963 multi-injected provider events), **Session** (CAIP-25 permission scopes), **Transport** (provider injection, WalletConnect v2 relay, or platform deeplinks), and **API** (Midnight-specific JSON-RPC methods). Layer 1 (TEE) is excluded because dApps never touch key material. Layer 6 (Chain Abstraction) is excluded because cross-chain intents route internally through the wallet's own intent router, not through the dApp connection surface.

3.2 Layer Interactions

The layers interact through well-defined interfaces. Layer 1 (TEE) provides key material to Layer 2 (Wallet) through a decrypt-derive-sign-zeroize cycle. Layer 2 provides address types to Layer 3 (Account Model), which maps device-specific keys to a single named account. Layer 4 (Identity) consumes secret keys from Layer 1 to construct attestation tree leaves and nullifiers. Layer 5 (Naming) binds the Layer 3 account to a human-readable identifier and resolves Layer 2 addresses. Layer 6 (Chain Abstraction) wraps Layers 2-5 in an intent-based interface that hides address type selection and cross-chain routing.

The main trust boundary is between Layer 1 and everything else. Key material decrypted from the TEE enters application memory briefly during derivation and signing, then is zeroized. The application memory window is the primary attack surface - mitigated by mlock (preventing page-out to swap), immediate zeroization, and minimal derivation scope (derive only the specific role key needed, not the full tree).

A secondary trust boundary exists **between the local device and the proof server. In Midnight's architecture, the proof server runs locally - a Rust process on the same machine, not a remote service. Witness values (including derived secret keys) are passed to the proof server via local IPC and never cross a network boundary.** The proof server generates the ZK proof (~18-21 seconds for a typical circuit) and returns it to the SDK for transaction assembly and submission. This local proof architecture is essential: if proof generation were remote, witness confidentiality would depend on transport security rather than mathematical guarantees.

3.3 Data Flow During Onboarding

The complete onboarding flow touches all six layers:

1. **QR scan** establishes an authenticated channel (Layer 6 intent: "onboard new user")
2. **Seed generation** in TEE (Layer 1)
3. **Three wallet addresses derived** (Layer 2: Shielded + Night + Dust from CIP-1852 path)
4. **Account creation** with first device key (Layer 3: register full-access key)
5. **Name registration** via commit-reveal (Layer 5: username.midnight bound to account)

6. **Developer-subsidized NIGHT airdrop** (Layer 2: starts DUST generation for fee payment)
7. **Identity credential issuance** (Layer 4: zkKYC attestation tree enrollment)
8. **Social account linking** for recovery factors (Layer 4 + DeRec)
9. **DeRec helper designation** (Layer 1: seed share distribution to 5 helpers)

Steps 2-3 are instantaneous (key derivation is sub-millisecond). Steps 4-5 each require a Compact circuit execution with ZK proof generation (approximately 18-21 seconds each). Step 6 is a standard Night token transfer. Step 7 is off-chain (zkMe verification) followed by an on-chain Merkle root update. Steps 8-9 are off-chain. The total onboarding time is dominated by proof generation: approximately 40-60 seconds for the two on-chain circuit calls (account creation and name registration), plus variable time for identity verification depending on the zkKYC provider.

4. Human-Readable Naming System

4.1 Architecture: Registry and Resolver in Compact

The naming system follows the ENS two-contract architecture, adapted for Compact and Midnight's privacy model. Two contracts manage all name operations:

The Registry Contract is the canonical source of ownership. It maps a namehash (a Poseidon-based recursive hash of the name) to an owner address, a resolver contract address, and a TTL (time-to-live) value. The registry does not interpret names – it operates entirely on namehashes. Owning a namehash lets you set its resolver, create subdomains, and transfer ownership.

Resolver Contracts map namehashes to records – addresses, profile metadata, content hashes, and application-specific fields. Separating resolution from registration lets name owners pick different resolver implementations: a basic resolver that maps to a single address, a multi-address resolver that handles all three Midnight address types plus cross-chain addresses, or a privacy-aware resolver that requires ZK proof of authorization before revealing shielded addresses.

```
// Registry contract (simplified)
import CompactStandardLibrary;

export ledger owners: Map<Bytes<32>, Bytes<32>>;           // namehash → owner
export ledger resolvers: Map<Bytes<32>, Bytes<32>>;        // namehash → resolver addr
export ledger ttls: Map<Bytes<32>, Unsigned<64>>;          // namehash → expiry

witness owner_key(): Bytes<32>;

export circuit set_resolver(namehash: Bytes<32>, resolver: Bytes<32>): [] {
  const sk = owner_key();
  const owner_hash = persistentHash<Vector<2, Bytes<32>>>([pad(32, "owner:"), sk]);
  assert(owners[namehash] == owner_hash, "not owner");
  resolvers[namehash] = resolver;
}

export circuit set_subnode(
  parent_hash: Bytes<32>,
  label_hash: Bytes<32>,
  new_owner: Bytes<32>
): [] {
```

```

const sk = owner_key();
const owner_hash = persistentHash<Vector<2, Bytes<32>>>([pad(32, "owner:"), sk]);
assert(owners[parent_hash] == owner_hash, "not parent owner");
const subnode_hash = persistentHash<Vector<2, Bytes<32>>>([parent_hash, label_hash]);
owners[subnode_hash] = new_owner;
}

```

4.2 Namehash Using persistentHash

ENS uses a recursive keccak256-based namehash algorithm (EIP-137). The Midnight version replaces keccak256 with persistentHash, Midnight's native Poseidon-based hash function. Poseidon is ZK-friendly – it produces far fewer constraints in a circuit than keccak256, making name resolution much cheaper to prove.

The namehash algorithm:

```

namehash("") = 0x0000...0000 (32 zero bytes)
namehash("midnight") = persistentHash([namehash(""), label_hash("midnight")])
namehash("alice.midnight") = persistentHash([namehash("midnight"), label_hash("alice")])

```

Where `label_hash(label) = persistentHash([pad(32, label)])`.

This produces a deterministic 32-byte identifier for any name in the hierarchy. The recursive structure means `alice.midnight` and `bob.midnight` share no computable relationship (Poseidon preimage resistance), preventing enumeration of registered names from their hashes alone.

4.3 Name Format

Names follow the hierarchical dot-separated format familiar from DNS and ENS:

- **Top-level:** `midnight` (the root namespace, DAO-controlled)
- **Second-level:** `alice.midnight` (user registrations)
- **Subdomains:** `payment.alice.midnight`, `work.alice.midnight`
- **Organizational:** `member.dao.midnight`, `employee.corp.midnight`
- **Short form:** `.mn` as an alias for `.midnight` (optional, for display only)

Name validation rules: - Minimum 3 characters for second-level names (prevent squatting of short names) - ASCII alphanumeric plus hyphens (no leading/trailing hyphens) - Case-insensitive (normalized to lowercase before hashing) - Maximum 255 bytes total (matching DNS limits) - Reserved names managed by DAO governance (e.g., `wallet.midnight`, `bridge.midnight`)

4.4 Commit-Reveal Registration Flow

Name registration uses a commit-reveal scheme to prevent frontrunning. Without it, a mempool observer could see a pending `register("alice")` transaction and submit their own with a higher fee. The commit-reveal scheme hides the desired name during the commitment phase.

Step 1: Commit

The user computes a commitment hash locally:

```
commitment = persistentHash([label_hash("alice"), owner_address, secret])
```

The secret is a random 32-byte value generated in the TEE. The commitment is submitted to the registrar contract's commit circuit. This circuit stores only the commitment hash on-chain – the name, owner, and secret are all hidden.

Step 2: Wait

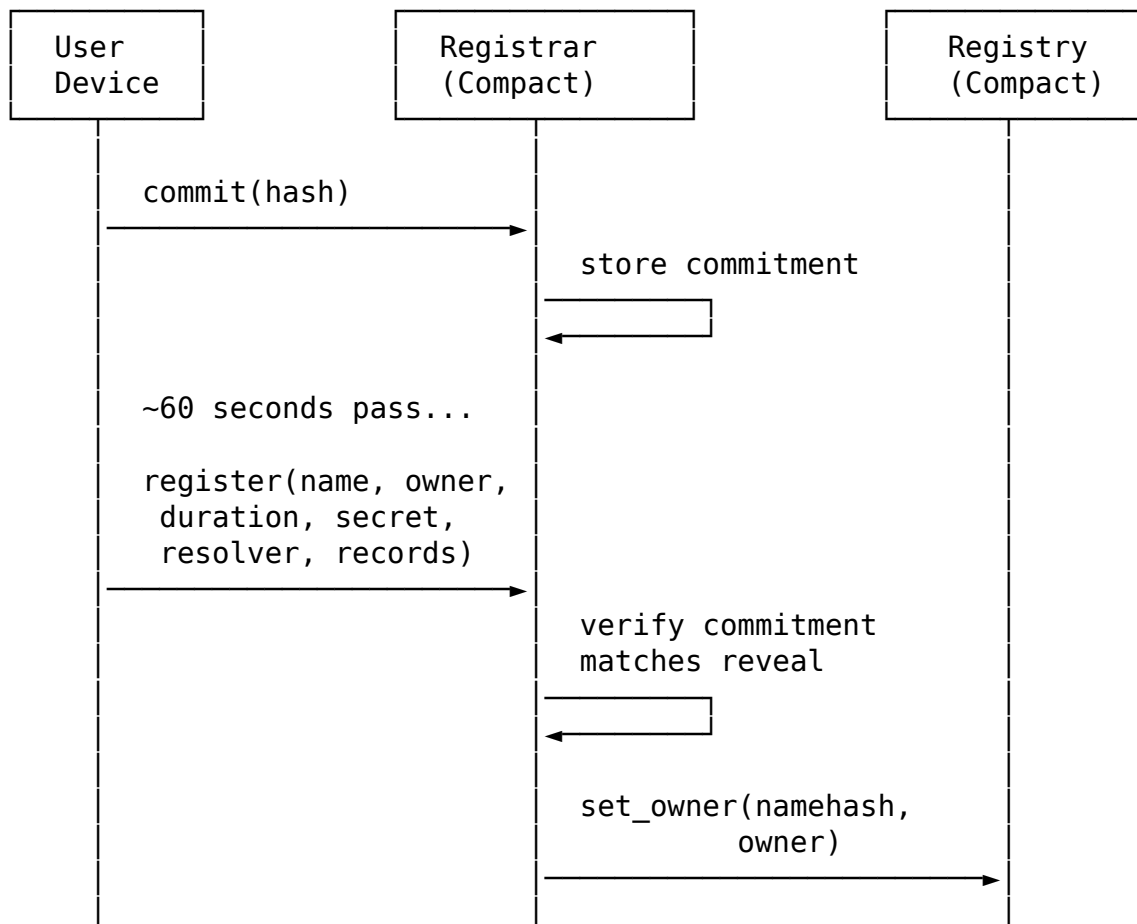
A minimum waiting period of approximately 60 seconds (or a configurable number of blocks) must elapse. This ensures the commitment is included in a finalized block before the reveal. An attacker who sees the commitment cannot determine what name it corresponds to (Poseidon preimage resistance).

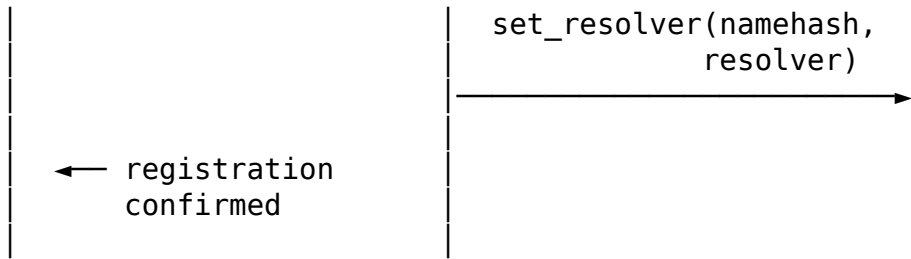
Step 3: Reveal and Register

The user submits the actual registration:

```
register(name, owner, duration, secret, resolver, initial_records)
```

The registrar circuit recomputes the commitment from the revealed values and verifies it matches the stored commitment. If it matches and the name is available, the registration proceeds: the namehash is computed, the owner is recorded in the registry, the resolver is set, and the initial records (addresses, profile data) are written.





The entire commit-reveal flow requires two on-chain transactions, each with a ZK proof. Total time: approximately 40-50 seconds (two proof generations of 18-21 seconds each, plus block confirmation).

4.5 Multi-Address Resolution

This is the key Midnight-specific adaptation. ENS was originally single-address (Ethereum), later extended to multi-chain via ENSIP-9. Midnight's naming system has to be multi-address from day one because Midnight itself has three distinct address types.

The resolver contract supports the following resolution functions:

Function	Returns	Use Case
<code>addr(namehash)</code>	UserAddress (32 bytes)	Receiving Night (unshielded) token transfers
<code>shieldedAddr(namehash)</code>	ZswapCoinPublicKey + EncryptionPublicKey	Receiving shielded token transfers
<code>dustAddr(namehash)</code>	DustAddress (BLS scalar)	Dust generation registration
<code>crossChainAddr(namehash, coinType)</code>	Bytes<N> (variable)	Cross-chain addresses (BTC=0, ETH=60, SOL=501 per SLIP-44)

A single name resolves to all of these simultaneously. When a user sets up their name during onboarding, all three Midnight address types are written to the resolver as initial records. Cross-chain addresses are added later as the user connects external chain wallets through the chain abstraction layer.

```

// Multi-address resolver (simplified)
export ledger night_addrs: Map<Bytes<32>, Bytes<32>>;
export ledger shielded_addrs: Map<Bytes<32>, Bytes<64>>; // cpk + epk
export ledger dust_addrs: Map<Bytes<32>, Bytes<32>>;
export ledger cross_chain: Map<Bytes<64>, Bytes<64>>; // namehash+coinType → addr

export circuit set_night_addr(namehash: Bytes<32>, addr: Bytes<32>): [] {
  // verify ownership via registry callback
  night_addrs[namehash] = addr;
}

export circuit set_shielded_addr(namehash: Bytes<32>, addr: Bytes<64>): [] {
  shielded_addrs[namehash] = addr;
}

```

4.6 Privacy-Aware Resolution

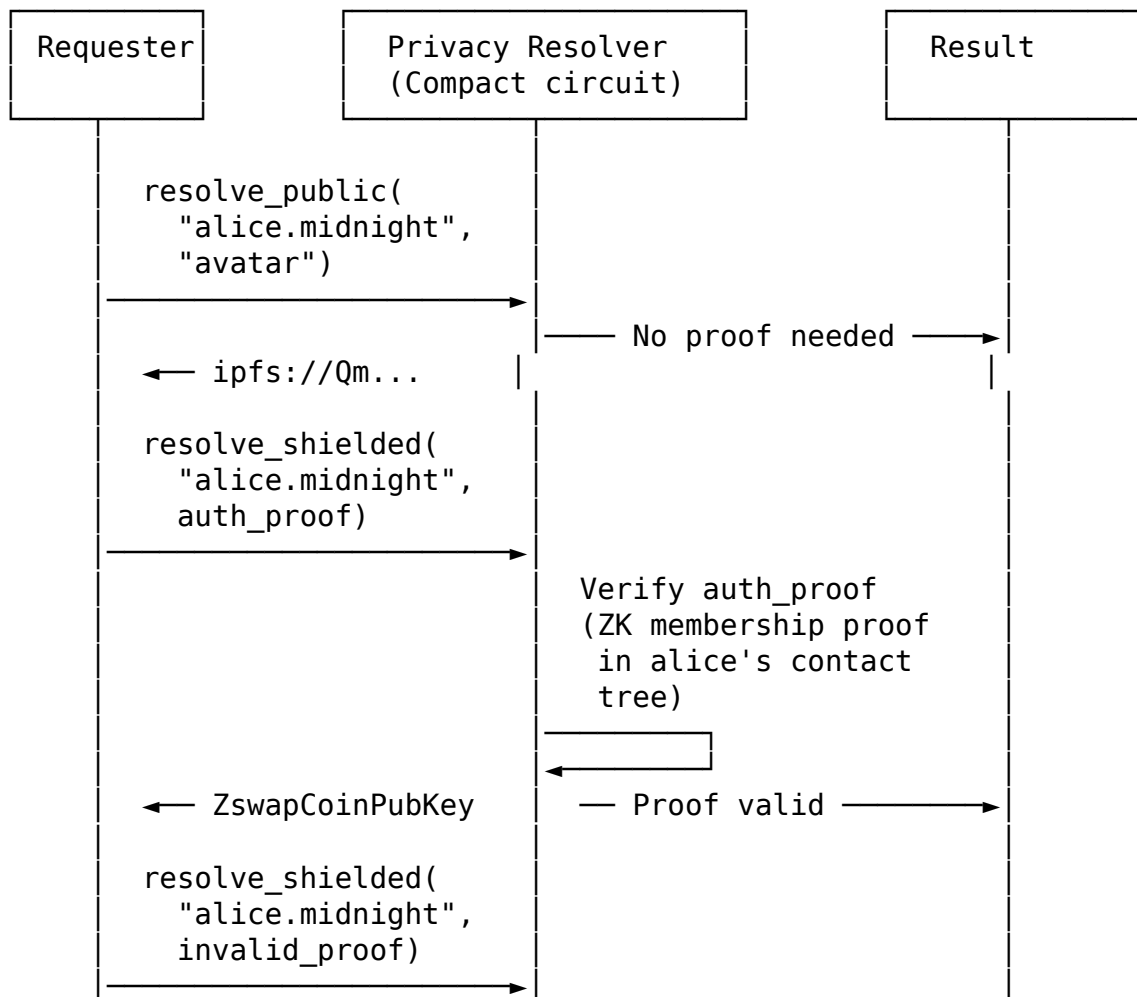
This is where Midnight's naming system differs most from ENS. On Ethereum, all name resolution is public – anyone can look up `vitalik.eth` and see all associated addresses. On Midnight, the resolver can enforce privacy policies per record type.

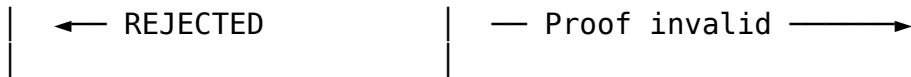
Public records (avatar, display name, bio) are freely readable by anyone. They are stored as disclosed ledger state.

Semi-private records (Night address) may be disclosed at the owner's choice. The `UserAddress` is typically public (Night transfers are transparent anyway), but the owner can choose to require a relationship proof before revealing it.

Private records (shielded address) require ZK proof of authorization before the resolver returns the address. The requesting party must prove they have a valid relationship with the name owner – perhaps membership in a shared attestation tree, or possession of a token issued by the name owner. This prevents shielded address harvesting: you cannot enumerate all shielded addresses in the system by querying all registered names.

The privacy-aware resolution flow:





Resolution queries for public records can use CCIP-Read (Cross-Chain Interoperability Protocol Read) to resolve off-chain, leaving no on-chain trace of who looked up whom. This matters for profile records: viewing someone's avatar should not create a permanent on-chain record.

4.7 Offchain Subdomains: Wildcard and CCIP-Read

Organizations need to issue subdomains to members at scale without per-name gas costs. The wildcard resolver pattern (from ENS ENSIP-10) resolves an entire subdomain tree from a single on-chain contract.

When `member42.dao.midnight` is queried:

1. The resolver for `dao.midnight` is a wildcard resolver
2. It does not find `member42.dao.midnight` registered on-chain
3. It issues a CCIP-Read callback to the DAO's offchain gateway
4. The gateway returns the address records along with a signed proof
5. The resolver verifies the signature and returns the result

The DAO manages its member directory off-chain (a database, an LDAP server, a smart contract on another chain – anything), and the wildcard resolver bridges that directory into the Midnight naming system transparently. New members get `user-name.dao.midnight` instantly at zero on-chain cost.

This matters for onboarding: a sponsoring organization can pre-register `newuser.org.midnight` as part of their QR onboarding flow, giving the user a name immediately without waiting for on-chain commit-reveal to complete.

4.8 Profile Records

The resolver supports extensible profile records, adapted from ENS text records (ENSIP-5):

Record Key	Type	Example	Privacy
avatar	URI	ipfs://Qm..., eip155:1/erc721:0x.../42	Public
display	String	Alice Johnson	Public
description	String	ZK developer and midnight enthusiast	Public
url	URI	https://alice.dev	Public
com.twitter	String	@alice_midnight	Public
com.github	String	alice-mn	Public
contenthash	Bytes	IPFS/Swarm content hash	Public

Record Key	Type	Example	Privacy
zkme.status	Enum	verified / unverified	Public
zkme.age18	Boolean	true	Semi-private
shielded.primary	Bytes<64>	ZswapCoinPublicKey + EncryptionPublicKey	Private

The `zkme.status` record signals that the user has completed zkKYC verification without revealing what was verified. A dApp can check this record to see whether the user is verified, then use the attestation tree circuit to confirm specific properties (age, citizenship) with a ZK proof. The name record gives discoverability; the attestation proof gives privacy-preserving verification.

4.9 Name Wrapper with Fuses

The Name Wrapper is an optional contract between the registry and the name owner. It controls permissions through irreversible “fuses” – once a fuse is burned, the corresponding permission is permanently revoked.

Fuse	Effect	Use Case
CANNOT_TRANSFER	Name is permanently bound to current owner	Soulbound names tied to identity
CANNOT_CREATE_SUBDOMAIN	New subdomains can be created under this name	Freeze organizational hierarchy
CANNOT_SET_RESOLVER	Resolver is permanently locked	Prevent resolver hijacking
CANNOT_UNWRAP	Name cannot be removed from the wrapper	Ensure fuses remain enforced
PARENT_CANNOT_CONTROL	Parent domain owner loses control over this subdomain	Emancipated subdomains
CUSTOM_FUSE_1..8	Application-specific restrictions	dApp-defined permissions

Soulbound names (with CANNOT_TRANSFER burned) are a natural fit for identity-linked naming. When a user registers `alice.midnight` during onboarding, burning the CANNOT_TRANSFER fuse makes the name permanently tied to that account – it cannot be sold, traded, or accidentally transferred. This prevents name-squatting marketplaces from developing around identity names while still allowing transferable names for other use cases (brand names, project names).

4.10 Integration with Onboarding Flow

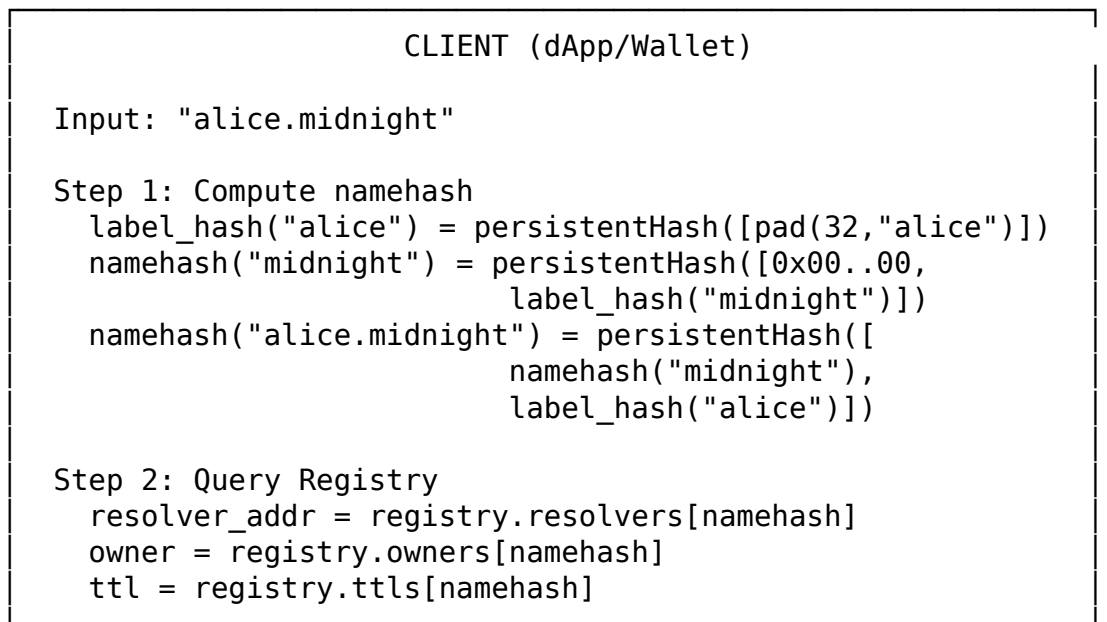
Name registration is not a separate post-onboarding step – it is woven into the QR scan flow as an atomic operation. The sequence:

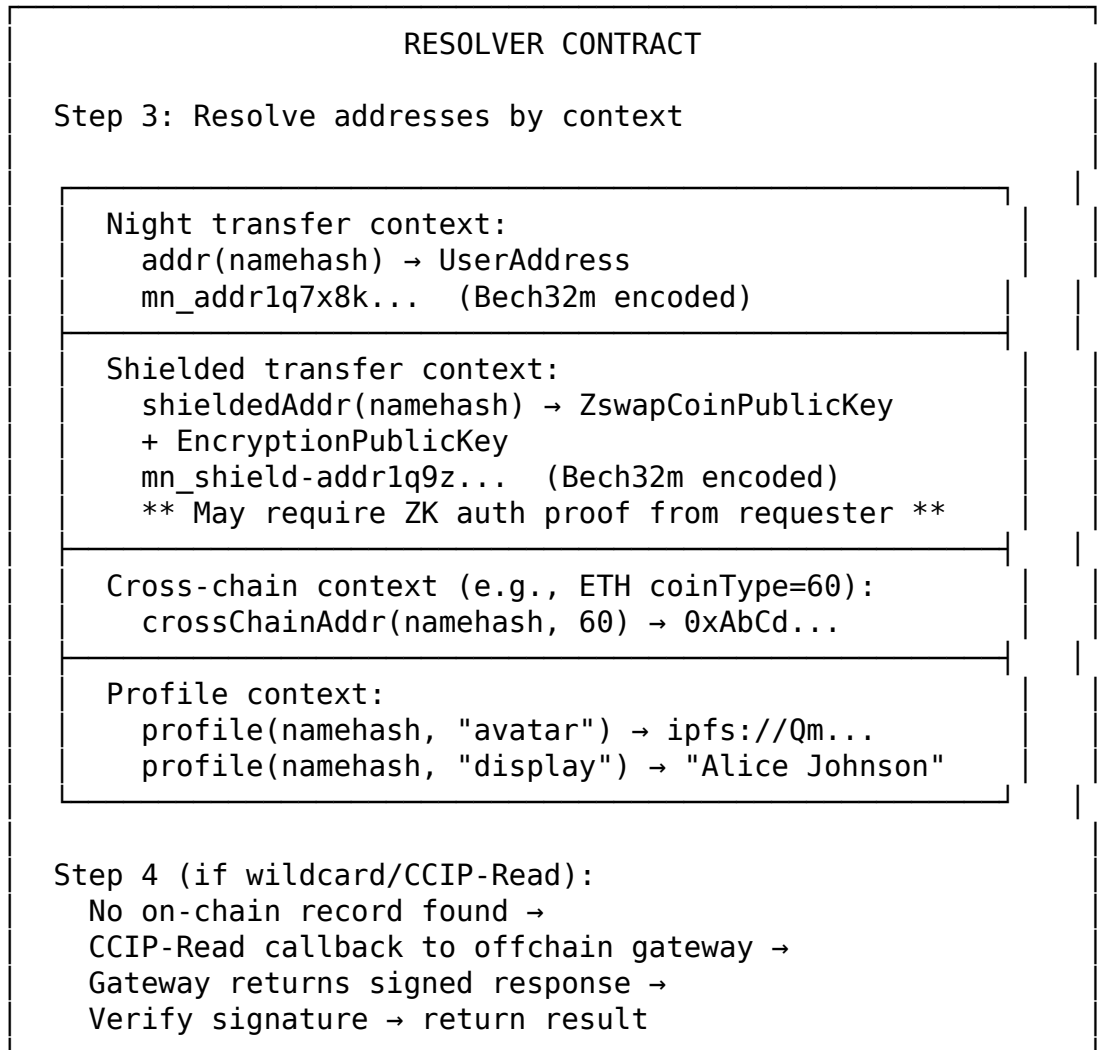
1. User scans QR code
2. Device generates seed in TEE, derives wallet addresses
3. Device submits `commit(persistentHash([label, owner, secret]))` to the registrar
4. While waiting for the commit to finalize (~20 seconds for proof + block confirmation):
 - NIGHT airdrop is sent to the user's Night address
 - DUST generation begins
 - Identity credential issuance starts (off-chain zkMe flow)
5. After commit finalizes, device submits `register(name, owner, duration, secret, resolver, records)`
6. Name is live: `alice.midnight` resolves to all three wallet addresses

The user chooses their name as part of the QR onboarding UI – a simple text field with availability checking. Name availability is checked off-chain (querying the indexer for the namehash's owner record) before the commit is submitted, providing instant feedback. The actual registration happens in the background while other onboarding steps proceed in parallel.

For organization-sponsored onboarding, the name can be pre-assigned: the QR code payload includes a desired name (`newuser.acme.midnight`), and the organization's wildcard resolver makes it available immediately. The user can later register a personal second-level name (`newuser.midnight`) and keep both.

4.11 Name Resolution Flow





4.12 Comparison: ENS vs NEAR Named Accounts vs Midnight Naming

Dimension	ENS (Ethereum)	NEAR Named Accounts	Midnight Naming (Proposed)
Implementation	Smart contract layer (EIP-137)	Protocol-level (built into consensus)	Compact circuit layer (application-level)
Hash Function	keccak256 (recursive namehash)	N/A (names are account IDs)	persistentHash / Poseidon (ZK-friendly)
Name Format	alice.eth	alice.near	alice.midnight (or .mn)
Hierarchy	Full subdomain support via registry	Sub-accounts (sub.alice.near)	Full hierarchy via registry + resolver

Dimension	ENS (Ethereum)	NEAR Named Accounts	Midnight Naming (Proposed)
Multi-Chain Addresses	Via ENSIP-9/EIP-2304 (SLIP-44 coin types)	NEAR-only (Chain Signatures for cross-chain)	Native dual-address (Shielded + Unshielded) + cross-chain via SLIP-44
Privacy	No privacy — all records public on-chain	No privacy — account data public	Privacy-native: shielded address resolution requires ZK proof; tiered record visibility (public/semi-private/private)
Registration	Commit-reveal (keccak256, ~60s wait)	Account creation tx (~1 NEAR)	Commit-reveal via Compact circuit (persistentHash, ~60s wait + proof generation)
Offchain Subdomains	CCIP-Read + wildcard (EIP-3668/ENSIP-10)	Not supported	CCIP-Read + wildcard (adapted for Midnight)
Governance	ENS DAO (\$ENS token)	Protocol governance	DAO-controlled registry parameters
Cost	Gas + \$5-640/year depending on length	~1 NEAR (~\$5) one-time	DUST fees (subsidizable by developers)
Token Standard	ERC-1155 (Name Wrapper)	N/A (protocol-native)	Soulbound naming token (non-transferable option via fuses)

Key advantage: Privacy-aware resolution. ENS and NEAR expose all address records publicly. Midnight's design allows selective disclosure – public profiles coexist with ZK-gated shielded addresses, and resolution queries via CCIP-Read leave no on-chain trace. Using Poseidon (persistentHash) instead of keccak256 also makes the name-hash natively compatible with Midnight's ZK circuits, so on-chain name verification works within Compact programs without external precompiles.

4.13 Name Registration Anti-Abuse

The commit-reveal scheme (Section 4.4) prevents frontrunning. An attacker watching the mempool sees only a Poseidon commitment hash – the desired name is hidden until the reveal. That handles one class of abuse. This section addresses the rest: rate

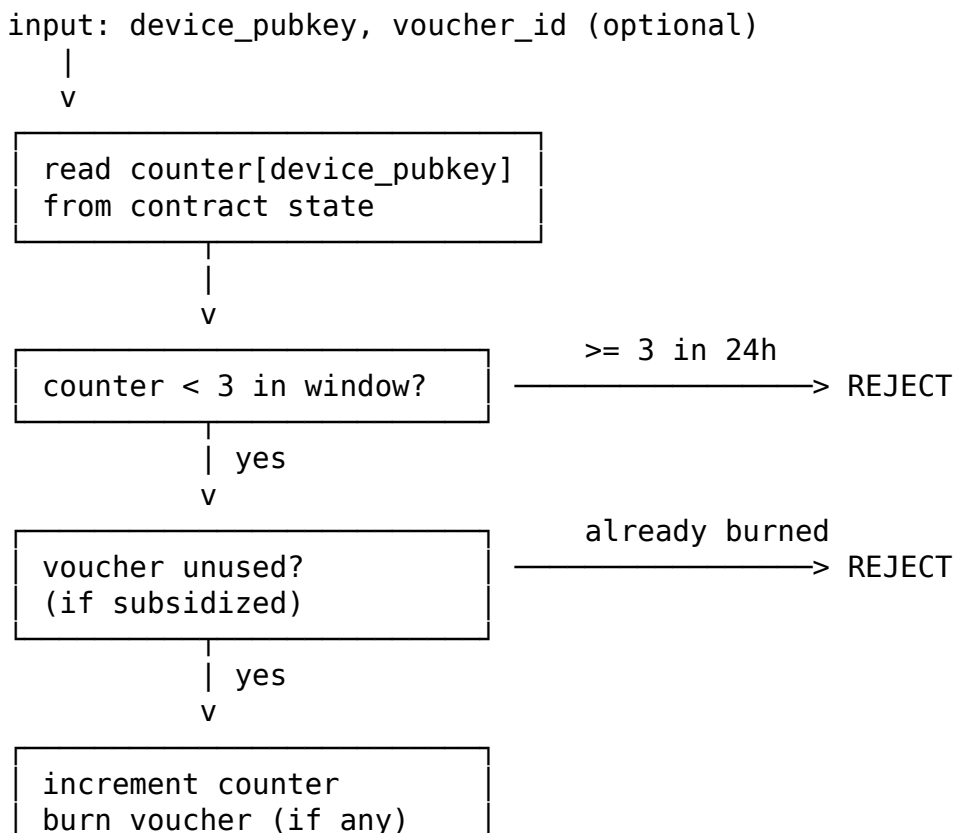
abuse, squatting, homoglyph attacks, and bot registrations.

Rate Limiting Uncapped registrations let a single actor claim thousands of names in an afternoon. Two hard caps prevent this:

Scope	Limit	Enforcement
Per device key	3 registrations per 24-hour rolling window	Registrar Compact circuit checks a counter keyed to the device's public key. The counter resets after 24 hours (measured in block height, not wall clock).
Per subsidized onboarding	1 registration total	The DUST subsidy voucher is a single-use token. The registrar circuit burns it on first use. A second commit from the same voucher fails at the circuit level.

Users who need more than 3 names per day (domain resellers, enterprise bulk provisioning) must submit a DAO governance proposal to raise their cap. This is intentionally inconvenient – the naming system is for identity, not speculation.

Rate Limit Check (inside registrar circuit):



```
proceed to commit-reveal
```

Name Squatting Deterrence A flat registration fee invites squatting – register `pay.midnight` for \$5, sit on it, sell it for \$5,000. Annual renewal fees with length-based pricing make hoarding expensive:

Name Length	Annual Renewal Multiplier	Example Annual Cost
3 characters	10x base fee	High (deterrent-priced)
4 characters	3x base fee	Moderate
5+ characters	1x base fee	Standard

Base fee is denominated in DUST and set by DAO governance. The multiplier applies to renewal only – initial registration costs the same regardless of length (to avoid punishing legitimate short-name holders at signup).

Unused name reclamation. If a name has zero associated transactions (no sends, no receives, no contract interactions) for 12 consecutive months, it is flagged as dormant. The owner receives a 90-day warning notification. If the name remains unused after the warning period, it enters a reclaim auction where any user can bid. Proceeds go to the DAO treasury. The original owner can cancel the reclaim at any time during the warning period by performing any on-chain transaction with that name.

This is not punitive. It is a garbage collection mechanism for a shared namespace.

Homoglyph Prevention Unicode allows dozens of characters that look identical to Latin letters. Without filtering, an attacker registers `alice.midnight` (Cyrillic a) and phishes users who think they are sending to `alice.midnight` (Latin a). The registrar circuit rejects confusable names at registration time.

Normalization. All name inputs pass through ENSIP-15 normalization before hashing. This is the same normalization ENS adopted after years of homoglyph abuse. It applies:

1. Unicode NFC normalization.
2. Rejection of mixed-script labels (Latin + Cyrillic in the same label is invalid).
3. Rejection of characters from the Unicode confusables list (UTS #39).
4. Case folding (uppercase mapped to lowercase).

The normalization happens client-side before the commitment hash is computed. The registrar circuit verifies that the revealed name passes the same normalization check – if the raw name differs from its normalized form, the registration fails.

Visual warnings. Even after normalization, some confusable pairs survive (e.g., `rn` vs `m` at small font sizes). The wallet UI displays a warning when resolving a name that is visually similar to an existing registered name:

```
! Warning: This name looks similar to
  "alice.midnight" (registered 2025-11-03)
```

You are about to send to:
"alice.midnight"

[Continue Anyway] [Cancel]

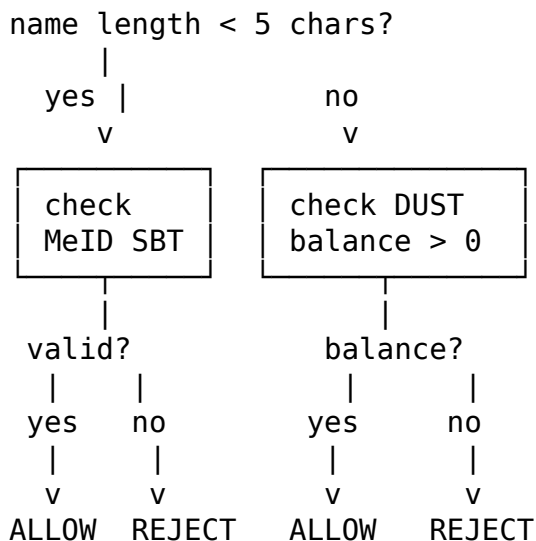
This is a UI-layer defense, not a protocol-layer one. The protocol rejects the obvious confusables. The UI catches the rest.

Bot Prevention Automated scripts can generate valid device keys and grind through the commit-reveal flow. Two barriers stop them:

Premium names (fewer than 5 characters). Registration requires a valid MeID soulbound token (SBT) from zkMe (see Section 7). The registrar circuit checks that the caller holds a non-expired MeID SBT. Since zkMe issuance requires a live biometric verification session, bots cannot obtain one. One person, one MeID, one shot at premium names.

Standard names (5+ characters). Registration requires a non-zero DUST balance. DUST comes from converting NIGHT tokens, NIGHT comes from the airdrop or staking rewards, and the airdrop requires completing a real onboarding session (QR scan, TEE key generation, proof generation). Each step in this chain is bottlenecked by a real device with a real TEE. A bot farm would need physical devices with genuine secure enclaves – at which point the economics of name squatting collapse.

Bot Prevention Decision Tree:



Neither barrier is perfect in isolation. Combined with rate limiting, renewal fees, and homoglyph filtering, they make large-scale name abuse economically irrational.

5. Midnight Platform Integration

This section maps every onboarding component to Midnight's actual cryptographic and architectural primitives. Where choices are made, they are justified by platform constraints and capabilities.

5.1 Curves: BLS12-381 and Jubjub

Midnight's cryptography is built on the BLS12-381 pairing-friendly elliptic curve with the Jubjub twisted Edwards curve embedded within it. This is a different curve ecosystem from Ethereum (secp256k1) or most Ed25519-based chains. All on-chain operations - ZK proofs, token commitments, Schnorr signatures for unshielded UTXOs - operate natively in BLS12-381/Jubjub.

This directly affects TEE integration. The iOS Secure Enclave supports only P-256 (secp256r1) for asymmetric operations. It cannot do BLS12-381 scalar multiplication or Jubjub point operations. The design works around this by using the Secure Enclave exclusively for AES-256-GCM symmetric encryption: a P-256 key pair is generated in the Secure Enclave and used to derive a symmetric wrapping key that encrypts the BLS12-381 seed material. The actual BLS12-381 key derivation happens in application memory after the seed is decrypted from the TEE.

The BLS12-381 scalar field has modulus `0x73eda753299d7d483339d80809a1d80553bda402fffe5bfe` (approximately 2^{255}). All wallet addresses are ultimately BLS12-381 field elements: the `BLSScalar` type in the wallet SDK encodes a bigint that must be less than this modulus. The `Bech32mCodec` handles the encoding of these field elements into human-readable `mn_` prefixed addresses.

Android StrongBox has broader curve support but is slow (around 9 seconds for key generation on some hardware). For cross-platform consistency, the Android implementation uses the same AES-256-GCM wrapping approach rather than attempting native BLS12-381 operations in StrongBox.

5.2 Three Wallet Address Types

Midnight's three-token architecture produces three distinct address types, each derived from the same BIP39 seed via different CIP-1852 role indices:

ZswapCoinPublicKey (Shielded Address) - Derivation: `m/44'/2400'/0'/3/0` (Zswap role, index 3) - Format: 32-byte coin public key + 32-byte encryption public key - Bech32m type: `shield-addr` - Purpose: Receiving shielded tokens via zero-knowledge proof transfers - The encryption public key lets the sender encrypt the token note so only the recipient can decrypt and spend it - Operations: `mintShieldedToken`, `sendShielded`, `receiveShielded` - all ZK-private

UserAddress (Night/Unshielded Address) - Derivation: `m/44'/2400'/0'/0/0` (NightExternal role, index 0) - Format: 32-byte public key - Bech32m type: `addr` - Purpose: Receiving Night (native) token transfers and contract interactions - Change addresses use NightInternal role: `m/44'/2400'/0'/1/index` - Signing: BIP-340

Schnorr signatures (not ECDSA) - compatible with Midnight's UTXO verification - Operations: `mintUnshieldedToken`, `sendUnshielded` - transparent on-chain

DustAddress (Fee Address) - Derivation: `m/44'/2400'/0'/2/0` (Dust role, index 2) - Format: BLS12-381 scalar (`bigint < field modulus`) - Bech32m type: `dust` - Purpose: DUST fee token generation and spending - DUST is non-transferable - it is generated time-dependently from NIGHT holdings and consumed by transaction fees - DUST regenerates automatically: a user holding NIGHT continuously generates DUST, which is spent as transaction fees

All three address types are encoded using the Bech32m standard with the `mn` human-readable prefix. A network identifier (MainNet, TestNet, DevNet) is embedded in the encoding, preventing accidental cross-network sends. The `MidnightBech32m` class handles encoding (`encode(networkId, item)`) and decoding (`parse(str)` followed by `decode(Type, networkId)`).

5.3 Bech32m mn_ Addresses

The address format is Bech32m (BIP-350), an error-detecting encoding that catches up to 4 character errors via a BCH checksum. Midnight addresses start with `mn` followed by a separator, the payload, and the checksum.

Example address formats:

<code>mn_addr1q7x8k2j...</code>	(<code>UserAddress</code> , Night tokens)
<code>mn_shield-addr1q9z4m...</code>	(<code>ShieldedAddress</code> , shielded tokens)
<code>mn_dust1qabc...</code>	(<code>DustAddress</code> , fee tokens)
<code>mn_shield-cpk1qdef...</code>	(<code>ShieldedCoinPublicKey</code> , proof of ownership)
<code>mn_shield-epk1qghi...</code>	(<code>ShieldedEncryptionPublicKey</code> , message encryption)

The type identifier after `mn_` determines the payload structure and decoding logic. The `Bech32mCodec` class provides a generic codec for any Bech32m-encoded type, while specific address classes (`ShieldedAddress`, `UnshieldedAddress`, `DustAddress`) provide typed constructors with validation.

For the naming system, a single `alice.midnight` name maps to at least three distinct Bech32m addresses. The resolver contract stores raw 32-byte payloads; the wallet SDK handles Bech32m encoding/decoding on the client side. Users never see or type these addresses - they are internal identifiers resolved from the human-readable name.

5.4 Witness-Based ZK Authorization

Midnight's authorization model differs from both account-based (Ethereum) and UTXO-script (Cardano/Bitcoin) models. Authorization works through ZK witnesses: a secret value is provided off-chain, used inside a Compact circuit to produce a proof, and the proof is verified on-chain without revealing the secret.

The canonical authorization pattern in Compact:

```

export ledger owner: Bytes<32>;

witness get_key(): Bytes<32>;

circuit verify_owner(): [] {
  const sk = get_key();
  const computed_owner = persistentHash<Vector<2, Bytes<32>>>([
    pad(32, "owner:"),
    sk
  ]);
  assert(computed_owner == owner, "unauthorized");
}

```

The ledger stores owner – a Poseidon hash of the secret key with a domain separator. The circuit takes the secret key as a witness (never disclosed), recomputes the hash, and asserts equality. The ZK proof demonstrates that the prover knows a preimage of the stored hash without revealing that preimage.

This maps directly to the TEE-held key model:

1. The BIP39 seed is decrypted from the TEE's AES-256-GCM envelope
2. The specific role key is derived: `m/44'/2400'/0'/role/index`
3. The derived key is passed to the Compact runtime as a witness value
4. The proof server generates a ZK proof (locally, approximately 18-21 seconds)
5. The key is zeroized from application memory
6. The proof is submitted to the blockchain node for verification

At no point does the key leave the device. The proof server is a local process. The blockchain node sees only the proof and the public transcript (which, by Compact's disclosure analysis, cannot contain the witness value). This is not defense in depth; it is a mathematical guarantee.

For the account model, each device key is registered as:

```
device_owner_hash = persistentHash(["owner:", device_secret_key])
```

The account contract stores a set of authorized device_owner_hash values. Any registered device can authorize operations by providing its secret key as a witness. Adding a new device means any existing full-access key proves its authorization and adds the new device's hash to the set.

5.5 Compact Circuit Token Operations

All token transfers on Midnight go through Compact circuits that generate ZK proofs. For onboarding UX, this means every state-changing operation incurs proof generation latency.

Shielded token operations (ZK-private):

- `mintShieldedToken(amount, recipient_cpk)` – creates a new shielded UTXO committed to the recipient's Zswap-CoinPublicKey. The amount, sender, and recipient are all hidden; the proof demonstrates the minting policy is satisfied.
- `sendShielded(amount, recipient_cpk, recipient_epk)` – spends input UTXOs and creates output UTXOs. The proof demonstrates conservation of value (inputs = outputs + fee) without revealing amounts. The note is encrypted to the recipient's encryption public key.
- `re-`

ceiveShielded(encrypted_note) - decrypts a note using the recipient's encryption secret key, producing a spendable UTXO.

Unshielded token operations (transparent): - mintUnshieldedToken(amount, recipient_addr) - creates Night tokens at the recipient's UserAddress. Amount and addresses are public. - sendUnshielded(amount, recipient_addr) - transfers Night tokens. Signed with BIP-340 Schnorr signature.

Proof timing (measured on devnet): - Deploy: approximately 21 seconds - Circuit call (typical): approximately 18 seconds - State query: near-instantaneous (no proof needed)

During onboarding, proof generation hits at two points: account creation (~18 seconds) and name registration (~18 seconds for the commit, ~18 seconds for the reveal). The NIGHT airdrop from the sponsoring developer is an unshielded transfer (fast, no ZK proof needed by the recipient). Proof generation overlaps with off-chain operations (zkKYC verification, DeRec helper setup) to minimize perceived wait time.

5.6 DUST Fee Model

DUST is Midnight's non-transferable fee token. Unlike Ethereum's gas (paid in ETH) or Cardano's fees (paid in ADA), DUST cannot be bought, sold, or transferred. It regenerates over time based on the user's NIGHT holdings. Two implications for onboarding:

New users need NIGHT, not DUST. A user with zero NIGHT generates zero DUST and cannot pay fees. The onboarding flow must include a NIGHT airdrop from the sponsoring developer. Once the user holds NIGHT, DUST regenerates automatically. The airdrop amount determines how quickly the user accumulates enough DUST for their first self-funded transaction.

Developers can subsidize users. Because DUST is non-transferable, developers cannot just airdrop DUST. They airdrop NIGHT, which starts DUST generation. The economics are intentional: NIGHT is the scarce resource, DUST is the regenerating fee resource. The cost of onboarding a new user is denominated in NIGHT, not in a volatile gas market.

The DustWallet in the SDK tracks DUST generation status, including the time-dependent regeneration rate, current balance, and spending history. The Dust-GenerationDetails type provides the current DUST generation rate, and Dust-GenerationDtimeUpdate tracks changes in the generation rate as NIGHT holdings change.

5.7 Private State Encryption

Compact contracts can maintain private state - data stored locally on the user's device, encrypted with AES-256-GCM, never published to the blockchain. This is distinct from the ZK privacy of shielded tokens (where data exists on-chain in committed form). Private state is truly off-chain: it exists only on the devices that created it.

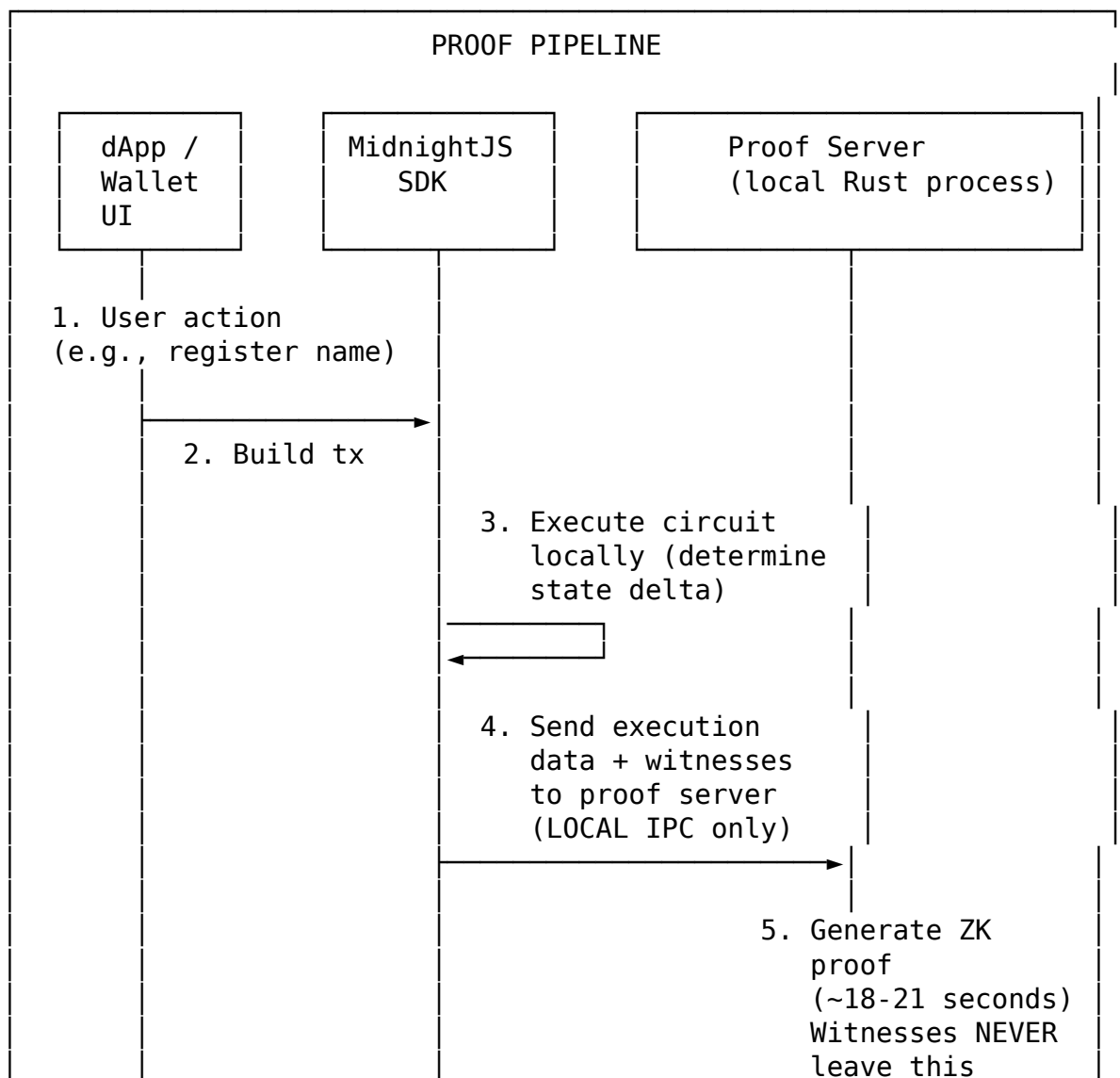
For the onboarding system, private state is used for: - **Attestation tree Merkle proofs:** The sibling hashes and path directions that prove membership in an attes-

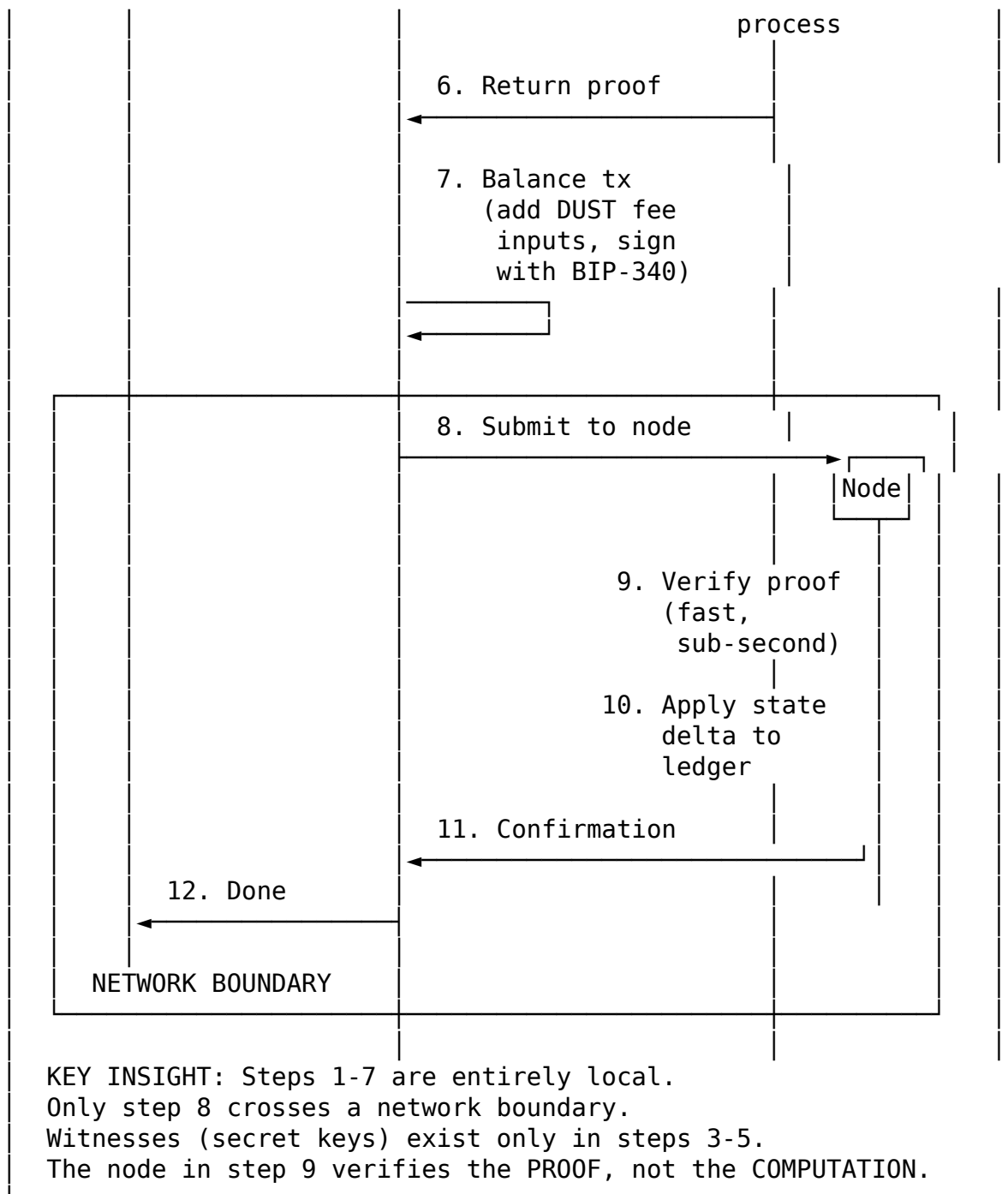
tation tree are stored as private state. They are never submitted on-chain; they are used locally to construct ZK proofs. - **Name registration secrets**: The random secret used in the commit-reveal scheme is stored as private state until the reveal phase completes. - **Function-call key allowances**: The DUST spending limits for per-dApp function-call keys are tracked in local private state.

Private state is encrypted using PBKDF2-derived keys and AES-256-GCM, with the encryption key material ultimately derived from the same BIP39 seed (via the Metadata role at derivation index 4). This means private state recovery depends on seed recovery: if the user loses all devices and recovers via DeRec, they must also restore private state from an encrypted backup or re-obtain it from the relevant contract counterparties.

5.8 Proof Pipeline

The pipeline from user action to on-chain state change:





The diagram shows the key security property: cryptographic key material never crosses a network boundary. The proof server is local. The node verifies a proof, not a witness. The dApp, the SDK, and the proof server all run on the user's device. The only data submitted to the network is the proof (which reveals nothing about its inputs) and the public state delta (which, by Compact's disclosure analysis, contains only intentionally disclosed values).

For the onboarding flow specifically, this pipeline executes three times: 1. **Account creation**: circuit proves the first device key hash, registers it on the account contract

2. **Name commit:** circuit stores the commitment hash (hides the desired name) 3. **Name reveal/register:** circuit verifies the commitment, records ownership in the registry

Each execution takes approximately 18-21 seconds, but steps 2 and 3 are separated by the mandatory waiting period (approximately 60 seconds). The total wall-clock time for the proof-intensive portion of onboarding is therefore approximately 18 (account) + 18 (commit) + 60 (wait) + 18 (reveal) = approximately 114 seconds. In practice, the account creation and commit proofs can be parallelized, and off-chain operations (NIGHT airdrop, zkKYC initiation, DeRec setup) fill the waiting periods, bringing the perceived onboarding time closer to 60-90 seconds.

5.9 dApp-Wallet Connection Standard (Midnight Connection Protocol)

Every blockchain ecosystem eventually needs a formal interface between dApps and wallets. Cardano has CIP-30. Ethereum has EIP-1193 plus EIP-6963. Solana has the Wallet Standard plus Mobile Wallet Adapter. None of these were designed for privacy-preserving chains, and that gap is not cosmetic – it is structural. Midnight’s wallet exposes three distinct address types (shielded, unshielded, DUST), coordinates long-running ZK proof generation, and manages selective credential disclosure. Bolting these onto CIP-30 or EIP-1193 would produce an adapter layer that fights the underlying standard at every turn. Midnight needs its own connection protocol: the Midnight Connection Protocol (MCP).

Why a New Standard The core problem: existing dApp-wallet standards assume a single address type, fast signing operations, and no privacy distinctions between transaction types. Midnight breaks all three assumptions.

1. **Three address types.** A Midnight wallet holds a `ZswapCoinPublicKey` (shielded), a `UserAddress` (Night/unshielded), and a DUST fee address. A dApp connecting to a wallet needs to know which address type it is requesting, and the wallet needs to gate access to each independently. CIP-30’s `getUsedAddresses()` returns a flat list. EIP-1193’s `eth_accounts` returns EOA addresses. Neither has a concept of address-type scoping.
2. **Proof coordination.** Signing a transaction on Ethereum takes milliseconds. On Midnight, generating a ZK proof takes approximately 18 seconds. A dApp cannot call `signTx()` and block the UI thread for 18 seconds. The connection protocol must support asynchronous proof generation with progress callbacks, cancellation, and background execution.
3. **Credential disclosure.** Midnight dApps routinely ask users to prove attributes (age $\geq$ 18, citizenship, accredited investor) via ZK proofs against attestation trees. No existing connection standard has a concept of “request a credential proof” as a first-class operation. Midnight needs `midnight_proveCredential` as a peer to `eth_sendTransaction`.
4. **Privacy-differentiated operations.** Sending shielded tokens is fundamentally different from sending unshielded Night tokens – different privacy guarantees,

different proof requirements, different user consent implications. The connection protocol must distinguish these and enforce appropriate consent gates.

Layered Architecture MCP follows the layered pattern that Solana and modern Ethereum have converged on. Four layers, each independently evolvable:

Layer 4: API
Midnight-specific RPC methods
midnight_getShieldedAddress, midnight_signTx,
midnight_proveCredential, midnight_proofStatus
Layer 3: Session
CAIP-25 authorization with Midnight scopes
Required/optional namespace negotiation
Layer 2: Transport
Browser injection WalletConnect v2 Deep links
Layer 1: Discovery
EIP-6963-style event announcements
window.midnight.{walletName} registration

Layer 1: Discovery. Wallets announce themselves by dispatching `midnight:announceProvider` events on window, carrying a provider info object with `rdns` (reverse domain name, e.g., `network.midnight.lace`), `uuid`, `name`, `icon`, and the provider instance. DApps listen for these events and present discovered wallets. This follows EIP-6963 exactly – no reason to reinvent wallet discovery. The `window.midnight` namespace is already established in Midnight’s existing dApp connector API.

Layer 2: Transport. Three transports, selected based on context:

- **Browser injection** (desktop): The wallet extension injects an MCP provider directly. Lowest latency, no relay. This is the primary path for desktop dApps.
- **WalletConnect v2 relay** (mobile cross-device): For mobile wallets connecting to desktop dApps. Uses WalletConnect’s encrypted relay with the midnight namespace. Session proposals carry Midnight-specific required methods.
- **Deep links** (mobile same-device): `midnight://connect?session=<token>` for mobile dApps connecting to a mobile wallet on the same device. Follows Solana MWA’s pattern of local ephemeral sessions.

Layer 3: Session. CAIP-25 session authorization. The dApp proposes required and optional namespaces. The wallet evaluates and responds. The key difference from other chains: Midnight sessions are scoped to specific address types and operation categories.

Layer 4: API. The actual RPC methods. These are Midnight-specific and privacy-aware. See the method table below.

CAIP Identifiers for Midnight MCP uses CAIP-2 and an extended CAIP-10 to identify chains and accounts:

Chain identification (CAIP-2): - Mainnet: midnight:1 - Testnet: midnight:testnet

Account identification (CAIP-10 extended):

Standard CAIP-10 encodes an account as chain_id:address. Midnight extends this with an address-type qualifier to distinguish the three wallet layers:

- midnight:1:shielded:mn_slq... - ZswapCoinPublicKey (shielded tokens)
- midnight:1:unshielded:mn_ulq... - UserAddress (Night tokens)
- midnight:1:dust:mn_dlq... - DUST fee address

The address-type qualifier (shielded, unshielded, dust) is not part of the base CAIP-10 spec. MCP extends CAIP-10 with a type segment because Midnight's addresses are not interchangeable – sending shielded tokens to an unshielded address is a protocol error, not just a UX mistake. The qualifier prevents it at the connection layer.

Session Authorization Scopes MCP defines five session scopes, requested via CAIP-25 namespace negotiation:

Scope	Grants access to	Default
midnight:shielded	Shielded address, shielded balance, shielded send/receive	Required
midnight:unshielded	Night address, Night balance, Night transfers	Optional
midnight:dust	DUST balance, fee estimation, DUST regeneration status	Required
midnight:proof	Proof generation requests, proof status polling, cancellation	Required
midnight:credentials	Credential disclosure proofs, attestation tree membership	Optional

A minimal session proposal for a privacy-focused dApp:

```
const sessionProposal = {
  requiredNamespaces: {
    midnight: {
      chains: ["midnight:1"],
      methods: [
        "midnight_getShieldedAddress",
        "midnight_getBalance",
        "midnight_signTx",
        "midnight_proofStatus"
      ],
      events: ["accountsChanged", "proofProgress"],
      scopes: ["midnight:shielded", "midnight:dust", "midnight:proof"]
    }
  },
  optionalNamespaces: {
    midnight: {
      scopes: ["midnight:unshielded",
```

```

    }
  }
};
"midnight:credential"]

```

The wallet displays this to the user as: “This dApp wants to: view your shielded balance, send shielded transactions, generate proofs. It optionally requests: view your Night balance, verify your credentials.” The user can approve all, approve only required, or reject.

Progressive Authorization MCP does not front-load permissions. The initial connection grants read-only access within the approved scopes. Signing and proof generation require explicit per-action consent:

1. **Connect** – dApp discovers wallet, user approves session scopes. DApp can now read balances and addresses within approved scopes.
2. **Read** – dApp queries balances, transaction history, credential status. No user interaction required (already consented via scope).
3. **Sign** – dApp requests a transaction signature. Wallet prompts the user with transaction details, privacy impact (shielded vs. unshielded), and estimated proof time. User approves or rejects.
4. **Prove** – dApp requests credential proof. Wallet prompts with exactly which attributes will be disclosed (e.g., “Prove: age $\geq$ 18. No other information will be revealed.”). User approves or rejects.

This is stricter than CIP-30, where `enable()` grants full API access, and closer to Solana’s feature-gated model. The rationale: on a privacy chain, the cost of accidental disclosure is higher than on a transparent chain. Progressive authorization ensures the user consents at each escalation.

Sign-In with Midnight (SIWM) Adapted from EIP-4361 (Sign-In with Ethereum) and Solana’s SIWS. SIWM lets a dApp authenticate a user by proving ownership of a `user-name.midnight` name:

1. DApp constructs a SIWM message: domain, address (the shielded address), statement (“Sign in to ExampleDApp”), URI, nonce, issued-at timestamp, and the Midnight name (`alice.midnight`).
2. Wallet displays the message for user review.
3. User approves. Wallet signs the message with the key associated with the name’s ownership record (the key registered in the naming contract).
4. DApp verifies the signature against the on-chain name registry. If `alice.midnight` resolves to a public key that validates the signature, authentication succeeds.

SIWM differs from SIWE in one key respect: the signed message includes the Midnight name, and verification checks the name registry contract, not just raw key ownership. This means SIWM proves “I am `alice.midnight`” rather than “I control address `0xabc...`”. The name adds a human-readable identity layer to the authentication.

Privacy-First Defaults MCP enforces shielded-by-default behavior:

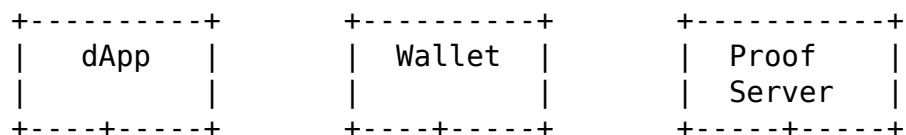
- **Default address type is shielded.** When a dApp calls `midnight_getAddress()` without specifying a type, the wallet returns the shielded address. Requesting the unshielded address requires the `midnight:unshielded` scope and an explicit type parameter.
- **Deshielding warnings.** If a dApp requests an operation that moves tokens from shielded to unshielded state, the wallet displays a warning: “This operation will make your balance publicly visible on-chain. Proceed?” This follows the pattern established by Zcash wallets (Zashi, YWallet), which warn users before z-to-t transfers.
- **No silent scope escalation.** A dApp with only `midnight:shielded` scope cannot request unshielded operations. The wallet rejects the call with `SCOPE_NOT_AUTHORIZED`. The dApp must request a session upgrade (re-prompting the user) to gain additional scopes.
- **Credential minimization.** When a dApp requests credential proofs, the wallet displays exactly which attributes are being proven and confirms that no additional information leaks. The Compact compiler’s disclosure analysis provides this guarantee – the wallet can inspect the circuit and report its public outputs.

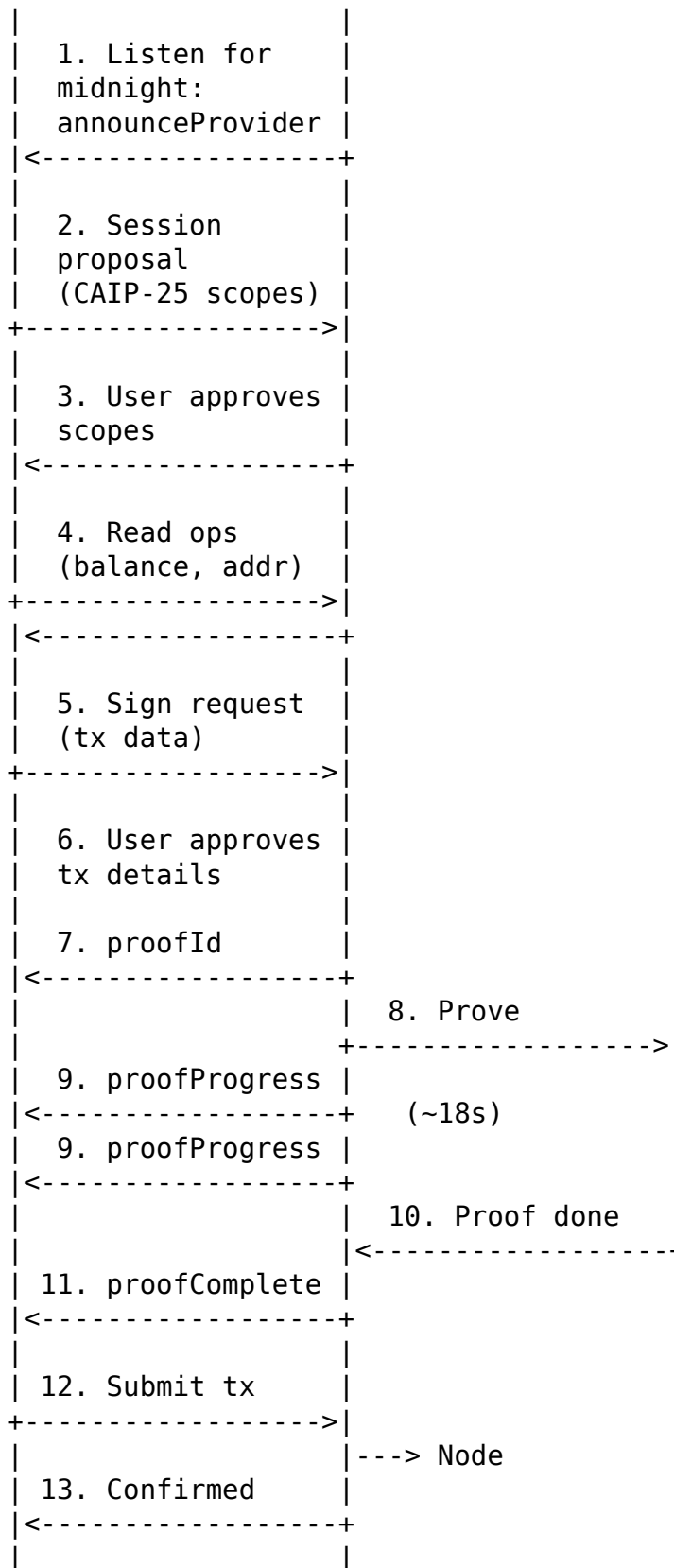
Proof Generation Coordination ZK proof generation is the hardest UX problem in the connection protocol. An 18-second blocking operation is unacceptable in a web context. MCP handles this with an asynchronous proof lifecycle:

1. **Request.** DApp calls `midnight_signTx(txData)`. Wallet validates, prompts user for consent.
2. **Initiate.** User approves. Wallet returns a `proofId` immediately (not the completed proof).
3. **Progress.** Wallet emits `proofProgress` events on the session channel: { `proofId`, `stage`: "circuit_exec" | "witness_gen" | "proving" | "balancing", `progress`: 0.0-1.0, `estimatedRemaining`: seconds }.
4. **Complete.** Wallet emits `proofComplete` event with the balanced, signed transaction. DApp calls `midnight_submitTx(proofId)` to broadcast.
5. **Cancel.** DApp or user can call `midnight_cancelProof(proofId)` at any point before completion. The proof server terminates the proving job.
6. **Error.** If proof generation fails (circuit error, insufficient DUST, proof server crash), wallet emits `proofFailed` event with error details.

DApps should display a progress indicator during proof generation. The `estimatedRemaining` field (derived from proof server telemetry) enables accurate countdown timers. The recommended UX: a non-modal progress bar with “Generating zero-knowledge proof... ~12s remaining” and a cancel button.

Connection Flow





Midnight RPC Methods

Method	Scope Required	Description
midnight_getShieldedAddress	midnight:shielded	Return bech32m shielded address
midnight_getUnshieldedAddress	midnight:unshielded	Return bech32m Night address
midnight_getDustAddress	midnight:dust	Return DUST address
midnight_getBalance	per address type	Balance for specified address type
midnight_signTx	midnight:proof	Build tx, generate proof (async)
midnight_submitTx	midnight:proof	Submit proven tx to node
midnight_proofStatus	midnight:proof	Poll proof generation status
midnight_cancelProof	midnight:proof	Cancel in-progress proof
midnight_proveCredential	midnight:credential	Generate credential ZK proof
midnight_resolveName	name (public)	Resolve name.midnight to addresses
midnight_signMessage	midnight:shielded	Sign arbitrary message (SIWM)

Comparison with Existing Standards

Feature	CIP-30	EIP-1193/6963	Solana WS	WC v2	MCP
Discovery	window.cardano	EIP-6963 events	Registry	QR/deep link	EIP-6963-style events
Multi-wallet	Namespaced	Event-based	Registry	N/A (P2P)	Event-based
Transport	Browser only	Browser only	Browser only	Relay + QR	Browser + WC v2 + deep link
Address types	1 (+ reward)	1	1	Chain-dep.	3 (shielded/unshielded)
Privacy scoping	None	None	None	None	5 scopes w/ consent gates

Feature	CIP-30	EIP-1193/6963	Solana WS	WC v2	MCP
Auth model	enable() all	eth_requestAccounts	connect()	Session NS	Progressive (read->sign->prove) Async w/ progress call- backs
Proof coord.	N/A	N/A	N/A	N/A	First- class prove- Cre- den- tial
Credential proofs	N/A	N/A	N/A	N/A	SIWM (name- based)
Sign-in	N/A	EIP-4361 (SIWE)	SIWS	N/A	Yes (proofId + events)
Async signing	No	No	No	Yes (relay)	MCP spec (pro- posed)
Spec formality	CIP process	EIP process	GitHub	WC specs	

The fundamental difference: every other standard treats the wallet as a synchronous signer. MCP treats the wallet as an asynchronous proof coordinator with privacy-scoped authorization. This is not added complexity for its own sake – it reflects the reality that ZK proof generation is slow, privacy requires explicit consent gates, and three address types need independent access control.

5.10 SDK Onboarding Wrapper

The onboarding protocol described in Sections 6-8 involves seven coordinated sub-systems: QR channel establishment, seed generation, wallet derivation, name registration, DUST subsidization, DeRec setup, and optional zkKYC. A dApp developer should not need to understand any of this. The SDK onboarding wrapper – `midnight.onboard()` – collapses the entire flow into a single async call.

```
import { midnight } from '@midnight-ntwrk/midnight-js-sdk';

interface OnboardConfig {
  // Required
```

```

provider: 'qr' | 'deeplink' | 'redirect';

// Name registration
name?: string; // e.g., 'alice' -> alice.midnight
nameAutoSuffix?: boolean; // append random suffix if taken

// Fee subsidization
dustSponsor?: {
  apiKey: string; // developer's sponsorship key
  maxSubsidy?: bigint; // cap in DUST units
};

// Recovery
recovery?: {
  method: 'derec';
  helpers?: number; // default 5
  threshold?: number; // default 3
  preselected?: string[]; // pre-designated helper IDs
};

// Identity (optional)
identity?: {
  provider: 'zkme' | 'custom';
  credentials: string[]; // e.g., ['age18', 'citizenship']
  skipIfExists?: boolean; // skip if creds already issued
};

// UX callbacks
onProgress?: (event: OnboardProgress) => void;
onProofStart?: (proofId: string, circuit: string) => void;
onProofProgress?: (proofId: string, pct: number) => void;
onProofComplete?: (proofId: string) => void;
}

interface OnboardResult {
  session: MidnightSession; // active MCP session
  wallets: {
    shielded: string; // mn_slq... bech32m
    unshielded: string; // mn_ulq... bech32m
    dust: string; // mn_dlq... bech32m
  };
  name: string | null; // registered .midnight name
  dustBalance: bigint; // initial DUST from sponsor
  recovery: {
    status: 'configured' | 'skipped';
    helpers: number;
    threshold: number;
  };
  credentials: string[]; // issued credential types
}

type OnboardProgress = {
  stage: 'channel' | 'seed' | 'derive' | 'name_commit'
    | 'name_wait' | 'name_reveal' | 'dust' | 'derec'
    | 'identity' | 'complete';
  message: string;
  progress: number; // 0.0 - 1.0 overall
};

async function onboard(
  config: OnboardConfig
): Promise<OnboardResult>;

```

API Signature

What It Abstracts A single `midnight.onboard()` call executes the following sequence internally:

```
+-----+
|               midnight.onboard()               |
| 1. Establish channel                             |
| +---> QR display / deep link / redirect          |
| +---> ECDH key exchange (P-256)                  |
| +---> Server-authenticated TLS-like channel      |
| 2. Generate seed                                 |
| +---> 256-bit entropy in TEE                     |
| +---> BIP39 mnemonic (internal only)             |
| +---> AES-256-GCM wrap with TEE key              |
| 3. Derive wallets                                |
| +---> CIP-1852 m/44'/2400'/0'/role/0            |
| |   role 0: NightExternal (unshielded)          |
| |   role 1: NightInternal (change)              |
| |   role 2: DUST                                |
| |   role 3: Zswap (shielded)                    |
| |   role 4: Metadata (encryption)               |
| +---> Register device key on account contract    |
|         (ZK proof #1, ~18s)                     |
| 4. Register name (if config.name set)            |
| +---> Commit: hash(name + salt) on-chain         |
|         (ZK proof #2, ~18s)                     |
| +---> Wait ~60s (front-running protection)       |
| +---> Reveal: register name in contract          |
|         (ZK proof #3, ~18s)                     |
| 5. DUST subsidy (if config.dustSponsor set)      |
| +---> Sponsor sends NIGHT to user address        |
| +---> DUST regeneration begins automatically     |
| 6. DeRec setup (if config.recovery set)          |
| +---> Split seed via Shamir (3,5)                |
| +---> Encrypt shares with ML-KEM                 |
| +---> Distribute to helpers                      |
| +---> Verify helper acknowledgments              |
| 7. zkKYC (if config.identity set)               |
| +---> Redirect to zkMe (or custom provider)      |
| +---> Receive attestation leaf                   |
| +---> Anchor Merkle root on-chain                |
| +---> Mint SBT credential token                  |
|
```

```

|
| 8. Return OnboardResult
|   +---> Active session, addresses, name,
|           balances, recovery status, credentials
|
+-----+

```

Steps 1-3 are sequential and mandatory. Steps 4-7 are conditional (based on config) and partially parallelizable – DUST subsidization and DeRec setup can run concurrently with the name registration waiting period. The onProgress callback fires at each stage transition, giving the dApp enough information to render a progress UI.

Three Integration Models Model 1: Component (2-3 lines). For dApps that want a drop-in onboarding UI with zero configuration beyond a sponsor key.

```

import { MidnightOnboard } from '@midnight-ntwrk/midnight-ui';

// In your React component:
<MidnightOnboard
  dustSponsor={{ apiKey: 'sp_live_...' }}
  onComplete={(result) => setUser(result)}
/>

```

The <MidnightOnboard> component renders a QR code (desktop) or a “Connect Wallet” button (mobile), handles the entire onboarding flow internally, and calls onComplete with the OnboardResult. It manages its own state, displays proof-generation progress, and handles errors with retry prompts. The developer’s only responsibility is providing a sponsor key and handling the result.

Model 2: Hooks (5-10 lines). For dApps that need control over the UI but not the protocol. React hooks expose each stage as observable state.

```

import { useOnboard } from '@midnight-ntwrk/midnight-react';

function SignupPage() {
  const {
    start, cancel, result, error,
    stage, progress, proofStatus
  } = useOnboard({
    provider: 'qr',
    name: username,
    dustSponsor: { apiKey: SPONSOR_KEY },
    recovery: { method: 'derec' },
  });

  return (
    <div>
      <button onClick={start}>Create Account</button>
      {stage && <ProgressBar value={progress}
        label={stage} />}
      {proofStatus && <p>Proof: {proofStatus.pct}%</p>}
      {error && <p>Error: {error.message}</p>}
      {result && <p>Welcome, {result.name}!</p>}
    </div>
  );
}

```

The hook manages the full onboarding state machine. stage and progress update reactively as the protocol advances. proofStatus provides real-time proof generation progress (percentage, estimated remaining time) so the dApp can render its own proof

UI. cancel aborts the flow at any point, cleaning up partial state.

Model 3: Server-side (enterprise). For regulated environments where onboarding is initiated from a backend service – e.g., an exchange onboarding users in bulk, or an enterprise provisioning employee wallets.

```
import { MidnightAdmin }
  from '@midnight-ntwrk/midnight-server';

const admin = new MidnightAdmin({
  apiKey: 'admin_live_...',
  proofServer: 'https://proof.internal:6300',
});

// Pre-provision a wallet and name
const invite = await admin.createInvite({
  name: 'employee42',
  dustSponsor: { budget: 10_000n },
  recovery: {
    method: 'derec',
    preselected: [
      'helper://corp-recovery.example.com'
    ],
  },
  identity: {
    provider: 'custom',
    credentials: ['employee', 'clearance_l2'],
  },
});

// Send invite link to user -- they complete
// on-device seed generation and key derivation
// on their own TEE
console.log(invite.url);
// https://join.midnight.network/i/abc123...
```

The server-side model splits the flow: the enterprise pre-provisions the name, DUST subsidy, and recovery configuration, then sends the user an invite link. The user's device handles the security-critical parts (seed generation, key derivation, TEE enrollment) – key material never touches the server. This satisfies compliance requirements (the enterprise controls the recovery helper and credential issuance) without compromising the security model (keys stay in the user's TEE).

Session Management and Reconnection After onboarding completes, Onboard-Result.session is an active MCP session (Section 5.9). The SDK persists session tokens in encrypted local storage (IndexedDB with AES-256-GCM, keyed to the TEE wrapping key). On subsequent visits:

```
import { midnight } from '@midnight-ntwrk/midnight-js-sdk';

// Attempt to restore existing session
const session = await midnight.reconnect();

if (session) {
  // Session restored -- user is authenticated
  const balance = await session.getBalance('shielded');
} else {
  // No valid session -- trigger onboard or connect
  const result = await midnight.onboard({ ... });
}
```

Session tokens expire after 24 hours of inactivity. The wallet extension can refresh to-

kens silently if the user has the extension open. If the token has expired, `reconnect()` returns `null` and the dApp must re-initiate connection (not full onboarding – just SIWM re-authentication if the wallet already exists).

Progress Callbacks for Proof Generation Proof generation is the longest single operation in onboarding. The `onProofStart`, `onProofProgress`, and `onProofComplete` callbacks give dApps fine-grained visibility:

```
const result = await midnight.onboard({
  provider: 'qr',
  name: 'alice',
  dustSponsor: { apiKey: SPONSOR_KEY },
  onProofStart: (id, circuit) => {
    console.log(`Proof started: ${circuit}`);
    // circuit = "account_register" | "name_commit"
    //           | "name_reveal"
  },
  onProofProgress: (id, pct) => {
    updateProgressBar(pct);
    // pct goes from 0.0 to 1.0
    // Fires every ~500ms during proving
  },
  onProofComplete: (id) => {
    console.log(`Proof ${id} complete`);
  },
  onProgress: (event) => {
    // Overall onboarding progress
    showStatus(event.message);
    // "Generating account proof..."
    // "Waiting for name commit confirmation..."
    // "Setting up recovery helpers..."
  },
});
```

Three proofs are generated during a full onboarding (account registration, name commit, name reveal). Each takes approximately 18 seconds. The `onProgress` callback provides stage-level updates (“Generating account proof...”), while `onProofProgress` provides sub-second granularity within each proof. DApps should always show a progress indicator during proof generation – an unexplained 18-second wait is a guaranteed abandonment point.

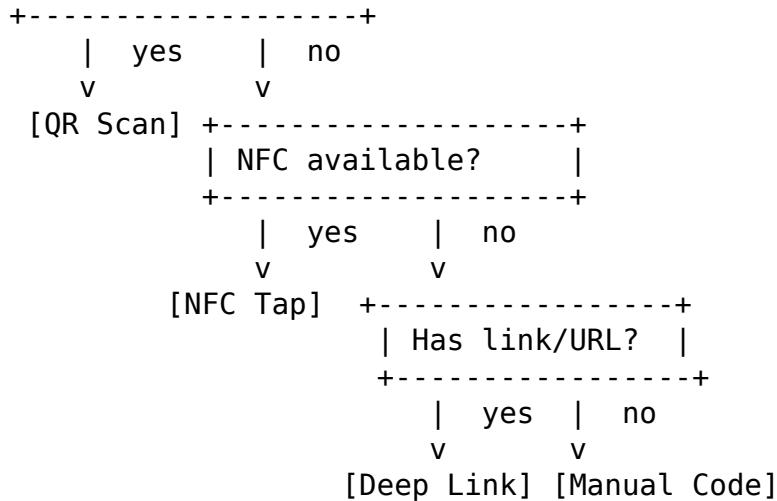
5.11 Alternative Onboarding Entry Points

QR scanning is the default entry point, but it is not the only one. Users who cannot use a camera – or who are onboarding from a context where camera access is unavailable (kiosk, CLI, assistive device) – need alternatives. All entry points funnel into the same server-authenticated ECDH session described in Section 6.2. The only difference is how the initial session payload reaches the user’s device.

Entry Point Decision Tree

```

User starts onboarding
      |
      v
+-----+
| Camera available? |
```



Deep Link Format: `midnight://onboard?session=<session_id>&server=<relay_host>`

The deep link encodes the same payload as the QR code: a session identifier and a relay server address. When the user taps the link, the Midnight wallet app opens, extracts the session ID, and initiates the ECDH handshake over the relay WebSocket – identical to the QR path from Phase 1 onward. Deep links work in any context where a URL is clickable: email invites, chat messages, web pages. The relay server authenticates via its TLS certificate, same as Section 6.2.

Use cases: enterprise invite emails (`admin.createInvite()` from Section 5.10 generates these), friend-to-friend sharing via messaging apps, web-based onboarding flows where camera access is impractical.

NFC Tap The onboarding kiosk (or a friend’s phone running the Midnight app) broadcasts an NFC Data Exchange Format (NDEF) record containing the session payload. The new user taps their phone to the kiosk or friend’s device. The phone reads the NDEF record, extracts the session ID and relay address, and proceeds with ECDH. NFC range is under 4 cm, which provides physical proximity assurance equivalent to QR visual line-of-sight.

The NDEF payload is identical in structure to the QR payload: `{ session_id, relay, server_pk }`. No additional protocol machinery is needed. The NFC tag or peer device acts purely as a transport for the initial bootstrap payload.

Manual Pairing Code For users who cannot use camera, NFC, or clickable links – including users relying on screen readers or other assistive technology – the fallback is an 8-character alphanumeric pairing code displayed on the initiating device (kiosk, dApp, or friend’s phone). The code is derived from the session ID: `base32(truncate(SHA-256(session_id), 40 bits))`, giving 8 characters from the set `[A-Z2-7]`. The user types this code into the Midnight app’s manual pairing screen.

The app sends the code to the relay server, which looks up the matching session and returns the full session payload. From there, ECDH proceeds as normal. The code expires

after 5 minutes or one successful use, whichever comes first. Brute-force resistance: $32^8 = \sim 1.1$ trillion combinations, rate-limited to 5 attempts per session.

Accessibility Each entry point supports assistive technology:

- **QR scan:** Camera viewfinder announces “Point camera at QR code” via VoiceOver/TalkBack. On successful scan, haptic feedback + audio confirmation.
- **Deep link:** Standard OS link handling. No additional accessibility work needed – the link opens the app directly.
- **NFC tap:** Audio prompt “Hold phone near device to connect.” Haptic feedback on successful read.
- **Manual code:** Input field is a standard text input, fully accessible to screen readers. Large-format code display (minimum 24pt) on the initiating device for low-vision users.
- **Proof wait:** During the 18-second proof generation windows, the app announces progress at 25% intervals (“Generating proof... 25% complete”). No silent waits longer than 5 seconds without an announcement.

Security Properties All four entry points produce the same cryptographic outcome: a P-256 ECDH shared secret negotiated over a server-authenticated relay, with the relay’s identity pinned to its TLS certificate. The transport for the initial payload (photons hitting a camera sensor, NFC radio, HTTPS redirect, or manual code entry) does not affect the security of the resulting channel. The session ID is a 256-bit random value generated by the relay server; the manual code is a truncated hash of that ID used only for lookup, not as key material.

6. QR Code Onboarding Flow

The QR code onboarding flow is the primary entry point for new users. A single scan produces a fully functional Midnight identity – wallet, credentials, name, and recovery – in under 60 seconds of user-facing interaction (excluding background proof generation at ~ 18 -21 seconds per circuit call). The protocol is atomic: if any phase fails, the entire session rolls back and a new QR is required.

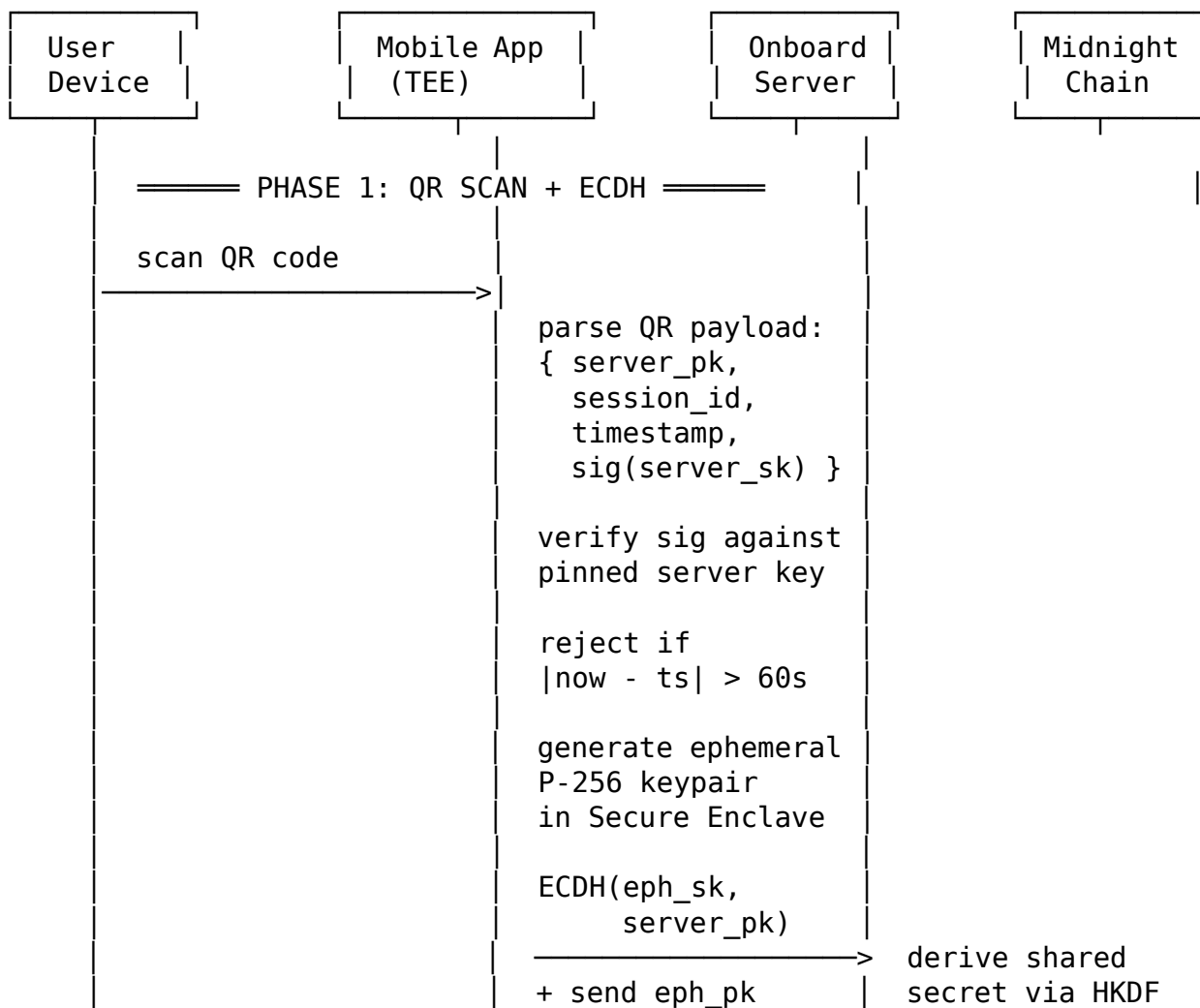
6.1 Protocol Overview

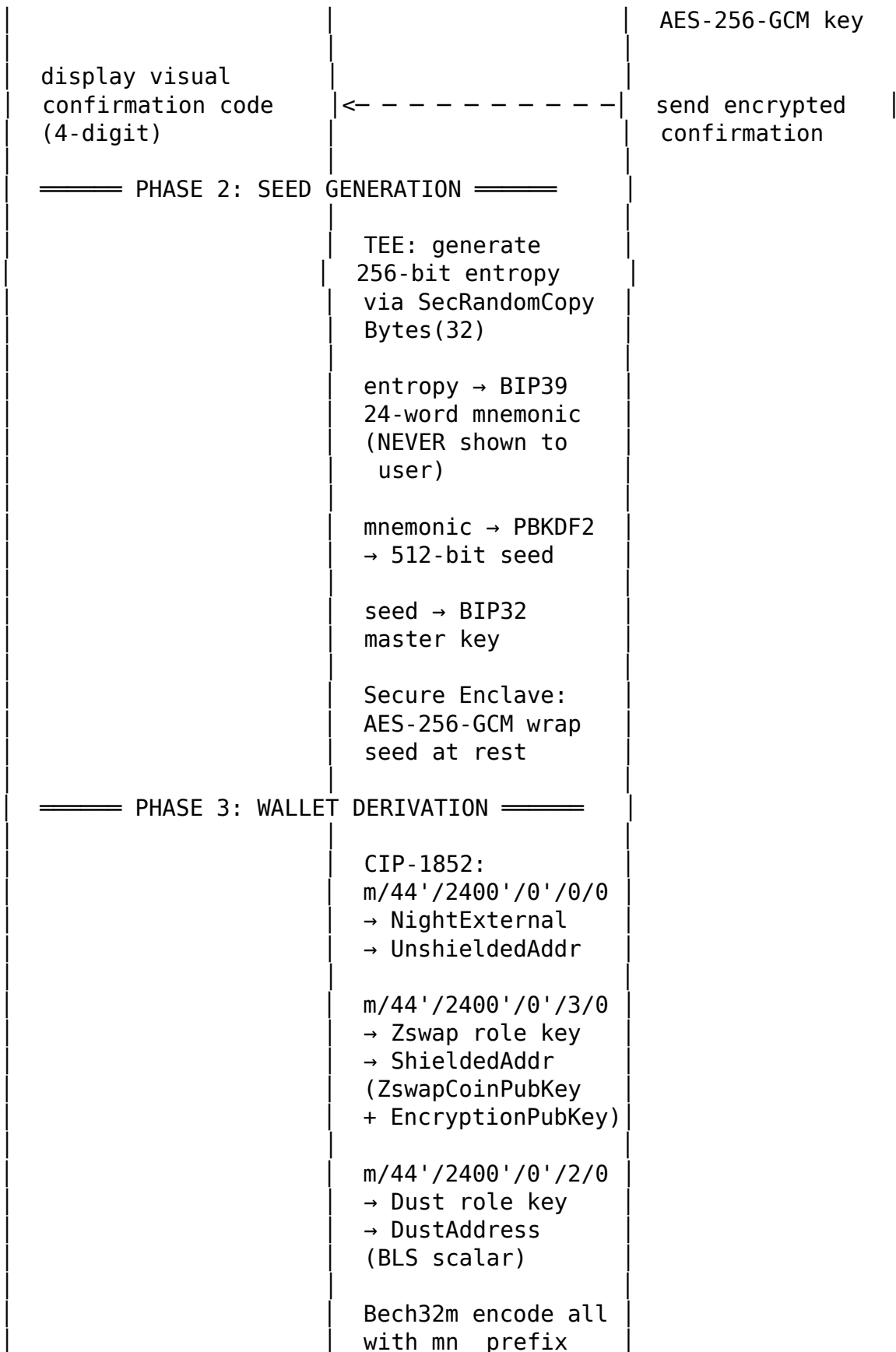
The flow has seven phases plus an optional second-device linking step. Each phase builds on artifacts from the previous one.

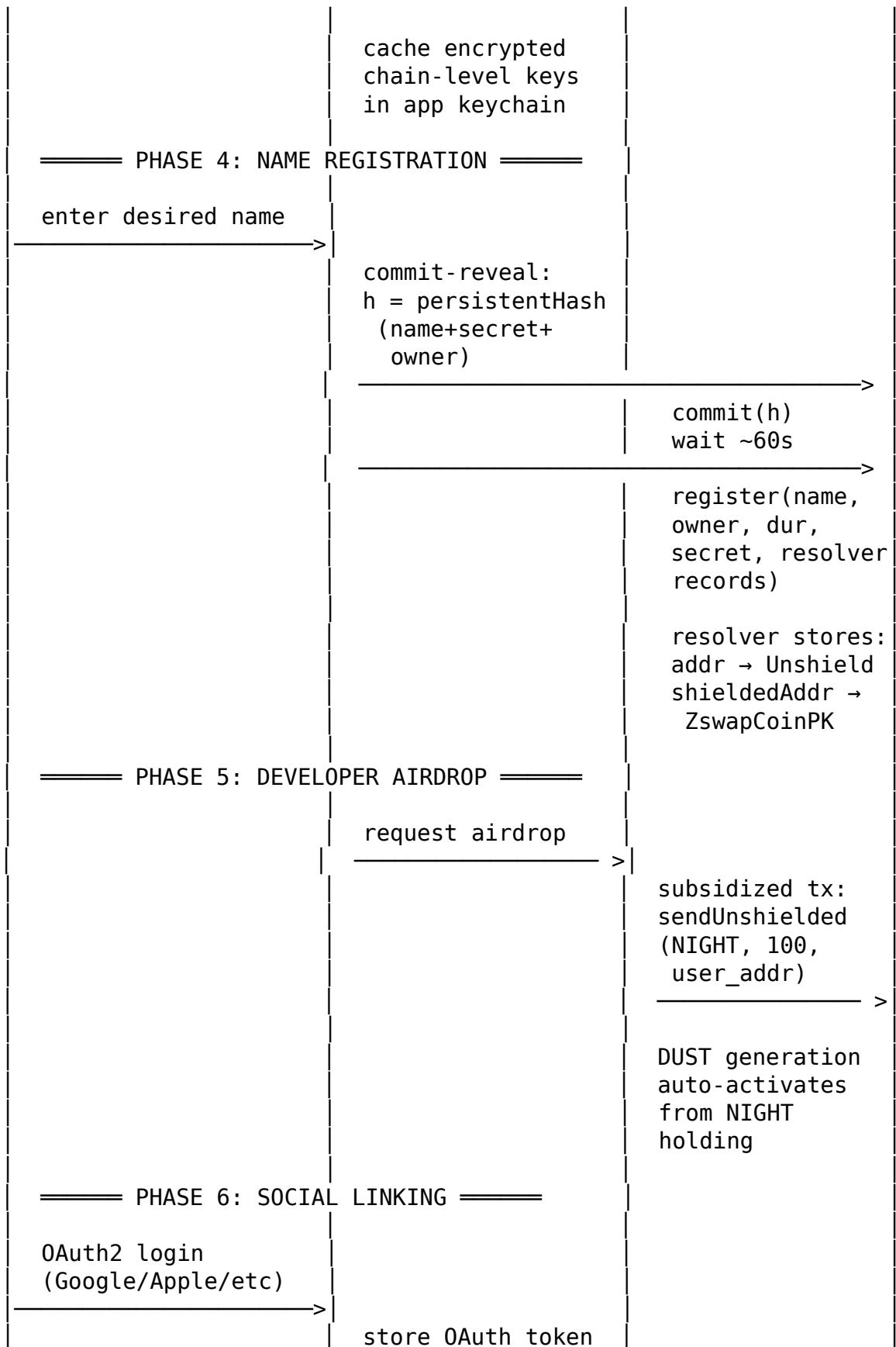
Phase	Action	Duration	Artifact Produced
1	QR scan + ECDH	<2s	Encrypted session channel
2	Seed generation in TEE	<1s	BIP39 mnemonic (hidden), master key

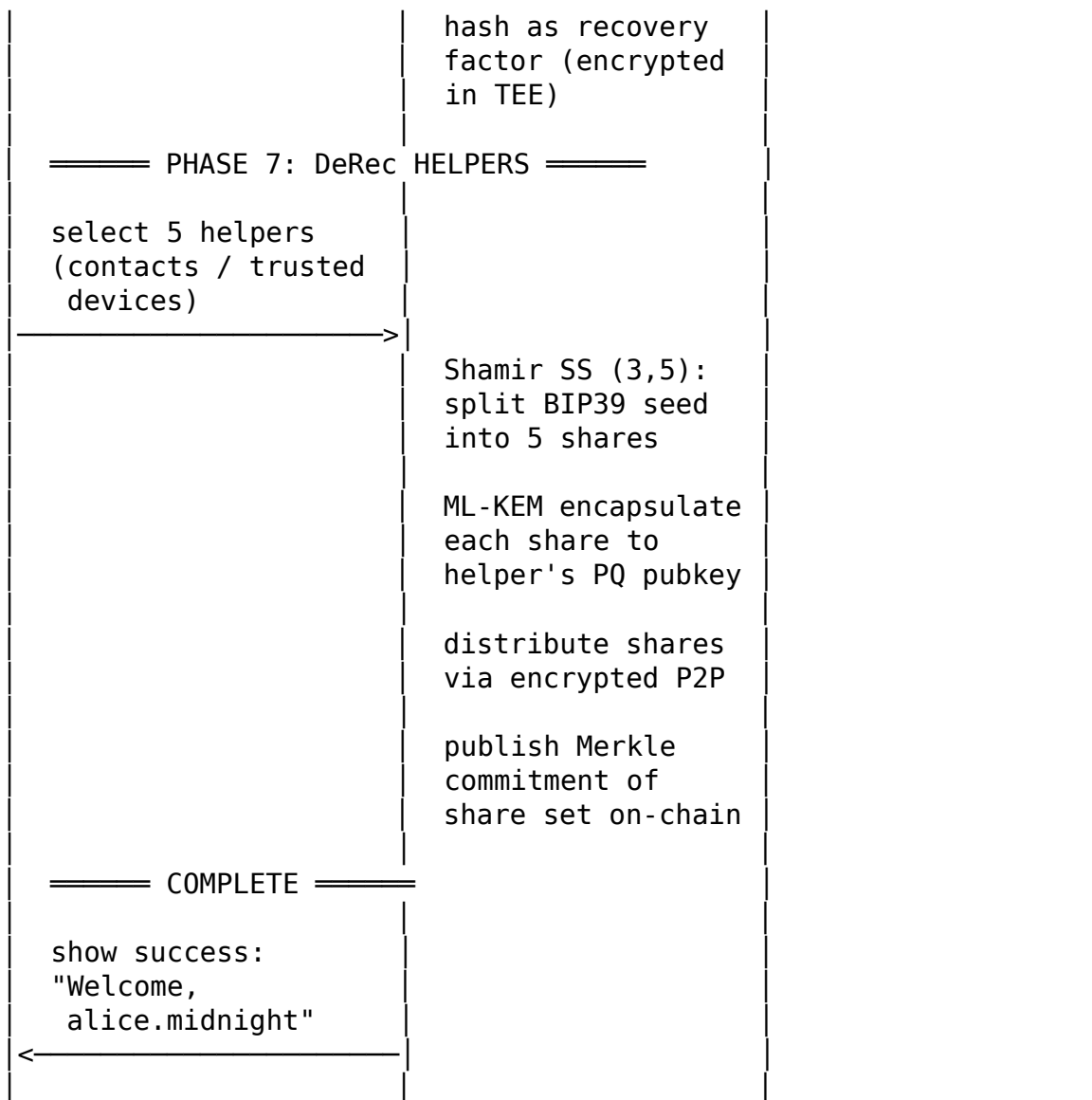
Phase	Action	Duration	Artifact Produced
3	Three-layer wallet derivation	<1s	Shielded, Night, Dust addresses
4	Name registration	~20s	username.midnight resolved identity
5	Developer-subsidized airdrop	~18s	NIGHT balance, DUST generation active
6	Social account linking	5-15s	OAuth2 recovery factor
7	DeRec helper designation	3-10s	(3,5) Shamir shares distributed

6.2 Detailed Sequence Diagram









6.3 Phase Details

Phase 1: Server-Authenticated ECDH The QR code contains a signed payload to prevent interception and replay:

Field	Size	Description
server_pk	33 bytes	Compressed P-256 public key
session_id	16 bytes	Cryptographically random nonce
timestamp	8 bytes	Unix milliseconds (UTC)
signature	64 bytes	ECDSA P-256 over SHA-256(session_id timestamp)
version	1 byte	Protocol version for forward compatibility

The mobile app has the server's public key pinned at build time. Any QR code signed by a different key is rejected. The 60-second timestamp window limits relay attacks – an attacker photographing a QR code has at most one minute to use it, but the single-use `session_id` is invalidated server-side after first contact.

The ECDH shared secret is derived via HKDF-SHA-256:

```
shared_secret = ECDH(ephemeral_sk, server_pk)
session_key    = HKDF-SHA-256(shared_secret, session_id, "midnight-onboard-v1", 32)
```

All subsequent communication in the session is encrypted with AES-256-GCM using this session key. The visual confirmation code (4 digits derived from `HMAC-SHA-256(session_key, "confirm")[:2]`) provides out-of-band verification that no MitM is present.

Phase 2: Seed Generation in TEE Entropy comes from the platform's hardware random number generator: - **iOS**: `SecRandomCopyBytes(kSecRandomDefault, 32, &buffer)` - **Android**: `SecureRandom` backed by `StrongBox RNG` - **Laptop**: `/dev/urandom` (Linux) or `CryptGenRandom` (Windows)

The 256-bit entropy is converted to a 24-word BIP39 mnemonic via the standard algorithm (SHA-256 checksum appended, mapped to the 2048-word English wordlist). The mnemonic is fed through PBKDF2-HMAC-SHA-512 with 2048 iterations and an empty passphrase (seedless UX – no mnemonic is shown to the user) to produce a 512-bit seed.

The seed is immediately wrapped with the TEE's AES-256-GCM symmetric key and the plaintext is zeroized from memory. The wrapped blob goes into the platform keychain (iOS Keychain / Android Keystore / OS keyring).

Phase 3: Three-Layer Wallet Derivation Using Midnight's registered BIP44 coin type 2400 and CIP-1852 derivation paths:

```
m/44'/2400'/0'/0/0 → NightExternal → UnshieldedAddress (32 bytes, Bech32m: mn_addr...)
m/44'/2400'/0'/1/0 → NightInternal → Change address (internal use)
m/44'/2400'/0'/2/0 → Dust → DustAddress (BLS12-381 scalar, Bech32m: mn_dust...)
m/44'/2400'/0'/3/0 → Zswap → ShieldedAddress (CoinPubKey + EncPubKey, Bech32m: addr...)
m/44'/2400'/0'/4/0 → Metadata → Metadata encryption key
```

Each derived key is independently encrypted with the TEE wrapping key and cached in the app keychain. This caching strategy means the raw seed only needs to be decrypted during initial setup or recovery – day-to-day signing decrypts only the specific role key needed.

Phase 4: Name Registration The naming system uses a commit-reveal scheme adapted for Compact circuits, preventing frontrunning:

1. **Commit:** `commit(persistentHash([name, secret, owner_address]))` - hides the desired name in a hash commitment published on-chain.
2. **Wait:** At least 60 seconds (~10 blocks at 6s block time) to ensure the commit is buried deep enough to prevent reordering attacks.
3. **Reveal:** `register(name, owner, duration, secret, resolver, records)` - reveals the name and binds it to the user's addresses.

The resolver contract stores multi-address resolution: - `addr(namehash)` resolves to the `UnshieldedAddress` for Night transfers - `shieldedAddr(namehash)` resolves to the `ZswapCoinPublicKey` for shielded operations - Cross-chain addresses (BTC, ETH, SOL) can be added later via SLIP-44 coin types

During onboarding, the server subsidizes the registration transaction by providing DUST for the proof and submission fees.

Phase 5: Developer-Subsidized Airdrop New users receive an initial NIGHT balance via a developer-funded faucet transaction. This:

1. **Starts DUST generation:** DUST regenerates time-dependently from NIGHT holdings. Without NIGHT, the user cannot pay transaction fees.
2. **Funds initial operations:** The NIGHT balance lets the user perform their first shielded or unshielded transfers.
3. **Creates self-sufficiency:** Once the user has NIGHT, DUST continuously regenerates for ongoing fee payment.

The airdrop uses `sendUnshielded` from the developer's wallet to the user's `UnshieldedAddress`. The transaction takes ~18 seconds for proof generation.

Phase 6: Social Account Linking OAuth2 tokens from Google, Apple, or other identity providers are hashed and stored as encrypted recovery factors. Two uses:

1. **Secondary authentication:** If the user loses their device but retains access to their social account, the OAuth token can unlock a recovery flow.
2. **Credential bootstrapping:** The OAuth identity can serve as an input to the zkMe verification flow (Section 8), pre-populating identity claims.

The social link is stored locally (encrypted in TEE) and optionally as a hashed reference in the user's on-chain resolver record.

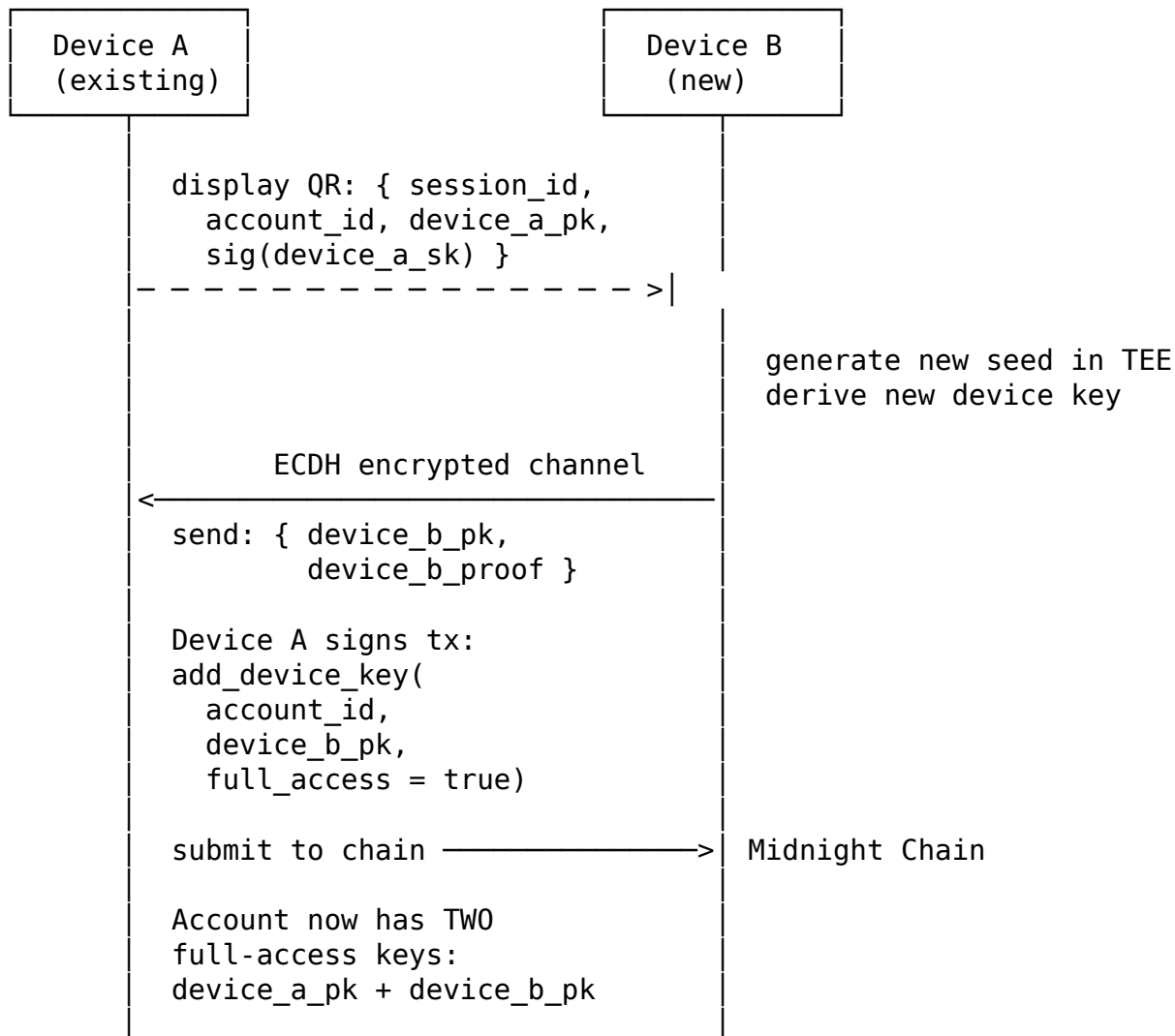
Phase 7: DeRec Helper Designation The user selects five trusted contacts or devices as DeRec helpers. The system then:

1. Splits the raw BIP39 seed into five shares using Shamir's Secret Sharing with a (3,5) threshold.
2. Encapsulates each share using the helper's ML-KEM-768 public key for post-quantum secure transport.
3. Distributes shares via encrypted peer-to-peer channels.
4. Publishes a Merkle commitment of the share set on-chain, binding the shares to the user's account.

See Section 10 for the full recovery protocol.

6.4 Second-Device Linking via QR

After initial onboarding, the user can link additional devices. Each device generates its own independent seed in its own TEE – no seed transfer between devices.



The critical property: **the account ID does not change when keys are added or removed**. This is the NEAR-inspired account model described in Section 7.

6.5 Onboarding Resilience (Offline / Degraded Network)

Onboarding spans multiple network round-trips, at least three ZK proof generations, and interactions with four independent services (relay server, proof server, name registry, and optionally zkMe and DeRec helpers). Any of these can fail mid-flow. The protocol handles this through checkpoint-based resumption: the SDK persists onboarding state to local encrypted storage after each completed phase, so the user never repeats work that already succeeded.

Checkpoint-Resume Flow

```
Phase 1: Channel      --[checkpoint]--> state.channel
Phase 2: Seed gen     --[checkpoint]--> state.seed
Phase 3: Wallets      --[checkpoint]--> state.wallets
Phase 4: Name commit  --[checkpoint]--> state.name_commit
Phase 5: Name reveal  --[checkpoint]--> state.name_reveal
Phase 6: DUST         --[checkpoint]--> state.dust
Phase 7: DeRec        --[checkpoint]--> state.derec
Phase 8: zkKYC        --[checkpoint]--> state.identity
```

|
v

On failure at any phase:

1. Save checkpoint to encrypted local storage
2. Show user: "Connection lost. Tap to resume."
3. On reconnect: load checkpoint, skip phases done
4. Resume from the last incomplete phase

Each checkpoint is encrypted with AES-256-GCM using the TEE wrapping key and stored in IndexedDB. Checkpoints contain only the artifacts needed to resume (session tokens, derived keys, transaction hashes) – never the raw seed or mnemonic, which remain in TEE memory only. Checkpoints expire after 24 hours; after that, the user must start a fresh onboarding session.

Failure Scenarios and Recovery **Proof server failure.** If the proof server crashes or becomes unreachable during proof generation, the SDK retries with exponential backoff: 1 second, 2 seconds, 4 seconds. After 3 consecutive failures, the SDK queues the proof request locally and notifies the user: “Proof generation temporarily unavailable. Your onboarding will resume automatically.” The queued request retries every 60 seconds in the background. If the proof was partially computed (e.g., the server crashed 14 seconds into an 18-second proof), the entire proof restarts from scratch – partial proofs are not resumable. No user-visible state is lost because the checkpoint captured everything before the proof attempt began.

zkMe unavailability. Identity verification is optional and non-blocking. If zkMe is unreachable during Phase 8, the SDK marks identity verification as “deferred” in the onboarding result:

```
result.credentials = []; // empty, not failed
result.deferredActions = [{
  type: 'identity_verification',
  reason: 'provider_unavailable',
  retryAfter: '2025-01-15T10:00:00Z'
}];
```

The user completes onboarding without credentials. The wallet app checks for zkMe availability periodically (every 6 hours) and surfaces a notification when the service returns: “Identity verification is now available – tap to complete.” Credential issuance then proceeds as a standalone post-onboarding step, reusing the existing session and keys.

DeRec helper unreachable. If one or more designated helpers cannot be contacted

during Phase 7, the SDK attempts the remaining helpers first. If the threshold is met (e.g., 3 of 5 helpers respond), recovery setup succeeds with the available helpers; the missing helpers are retried in the background. If the threshold is not met, recovery setup is deferred entirely. The wallet app shows a persistent (but non-blocking) reminder: “Recovery not fully configured – 2 of 5 helpers paired.” Daily reminders continue until the user has paired enough helpers to meet the threshold. The user can manually trigger helper pairing from the wallet settings at any time.

Name registry unavailable. If the name registry contract is unreachable or the chain is congested during Phase 4 or 5, the user receives a temporary address-based identity. The wallet displays the user’s shielded address as their identifier, with a note: “Name registration pending.” The SDK queues the commit-reveal sequence and executes it when the registry becomes available. Once the name is registered, the wallet updates the display and notifies the user: “Your name `alice.midnight` is now active.” Other users who received the temporary address can still send funds to it; once the name resolves, both the address and the name route to the same account.

Full network loss during onboarding. If the device loses network connectivity entirely, the SDK checkpoints whatever phase is in progress and waits. No timers run, no retries fire. When connectivity returns (detected via the browser’s online event or the OS network reachability API), the SDK reloads the checkpoint and resumes. If the ECDH session with the relay has expired (sessions live for 10 minutes), the SDK establishes a new relay session and re-authenticates. The seed, derived keys, and any completed on-chain transactions are unaffected – they persist locally or on-chain independent of the relay session.

State Consistency Guarantees Checkpoint-resume introduces a risk: what if an on-chain transaction was submitted but the confirmation was not received before the network dropped? The SDK handles this by checking transaction status on resume. Before re-submitting any transaction, it queries the chain for the transaction hash stored in the checkpoint. If the transaction is already confirmed, the SDK advances to the next phase. If the transaction is pending, the SDK waits for confirmation. If the transaction was dropped (not in mempool, not confirmed), the SDK re-submits. This prevents duplicate on-chain operations – particularly duplicate name commits, which would waste DUST.

7. Account Model

The account model adapts NEAR Protocol’s multi-key architecture for Midnight’s privacy-preserving context. The key idea: **an account is a named identity, not a public key**. Keys are authorization mechanisms bound to accounts – they can be added, removed, and rotated without changing the account’s identity or on-chain state.

7.1 Named Accounts

Every onboarded user has a named account: `alice.midnight`. This name is:

- **Stable:** Persists across device changes, key rotations, and recovery events.
- **Resolvable:** The naming system (Section 4) maps the name to current addresses.
- **Decoupled from keys:** The account ID is a hash of the registered name, not derived from any cryptographic key.

7.2 Key Types

The account model has two categories of keys, mirroring NEAR's full-access and function-call key distinction:

Key Type	Scope	Held By	Capabilities
Full-Access Key	Entire account	Device TEE	All operations: transfers, key management, credential issuance, name management
Function-Call Key	Single contract / circuit	dApp session	Scoped to specific Compact circuits; limited DUST allowance; cannot modify keys or transfer NIGHT

Full-Access Keys Each device holds one full-access key, derived from its own independent seed in its own TEE. A full-access key can:

- Sign any transaction on behalf of the account
- Add or remove other keys (including other full-access keys)
- Update name resolver records
- Authorize credential issuance
- Designate or change DeRec helpers

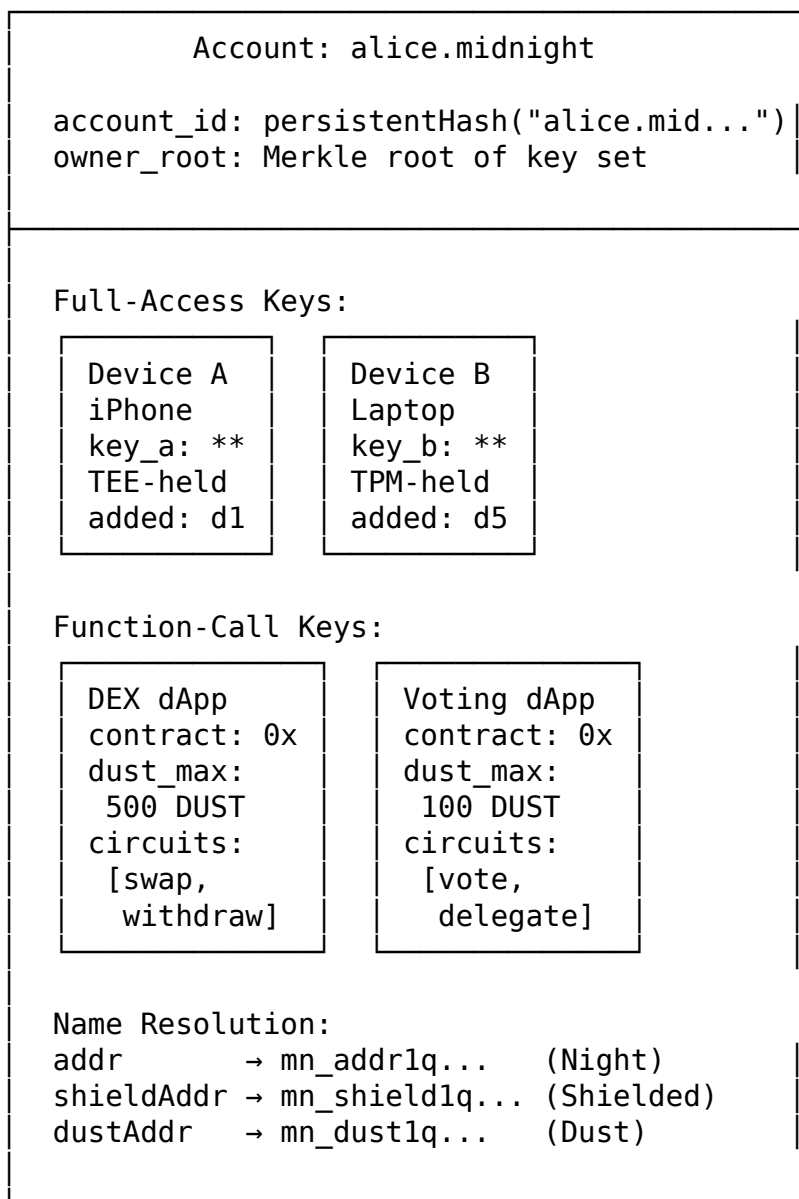
The key itself is a BLS12-381 scalar derived via CIP-1852 from the device's seed. Authorization is ZK witness-based: the key is provided as a Compact witness, and the circuit proves `persistentHash([domain_sep, witness_key]) == account_owner_hash` without revealing the key.

Function-Call Keys When a user connects to a dApp, the wallet generates a scoped function-call key:

- **Contract-scoped:** The key can only invoke circuits on a specific deployed contract address.
- **DUST-limited:** The key has a maximum DUST expenditure allowance, preventing a malicious dApp from draining fee resources.
- **Non-transferable:** The key cannot initiate NIGHT transfers or modify account keys.
- **Revocable:** Any full-access key can revoke a function-call key at any time.

Function-call keys give “sign-once, use many” UX – after initial wallet approval, the dApp can submit transactions without per-operation user confirmation, up to its allowance limit.

7.3 Device Addition and Removal



Adding a device: The new device generates its own seed and derives a key. The existing device signs an `add_key` transaction that adds the new key to the account’s key set. The Merkle root of authorized keys is updated on-chain.

Removing a device: Any remaining full-access key signs a `remove_key` transaction. The revoked key is immediately invalidated – any subsequent transaction signed with it fails ZK verification because the key is no longer in the Merkle tree.

Key rotation: A device rotates its own key by atomically adding a new key and removing the old one in a single transaction. Routine security hygiene, no user interaction needed beyond the initiating device.

7.4 Comparison with NEAR

Feature	NEAR Protocol	Midnight Adaptation
Account naming	alice.near (protocol-level)	alice.midnight (Compact registry)
Key types	Full-access, function-call	Same model, adapted for Compact circuits
Key storage	Plain Ed25519	BLS12-381 in TEE, ZK witness authorization
Key authorization	Signature verification	ZK proof: $\text{hash}(\text{key}) == \text{owner}$
Function-call scope	Contract + method list	Contract + circuit list + DUST allowance
Key addition	On-chain tx signed by full-access key	Same, but uses Compact circuit
Privacy	Keys and operations fully public	Keys are witness values; only hashes on-chain

The Midnight advantage: **the key itself is never revealed on-chain**. NEAR stores public keys in plaintext contract state. Midnight stores only `persistentHash([domain_sep, key])` - the key stays a witness value in the TEE.

8. Identity and Credential Layer

The identity layer pairs Midnight's attestation tree mechanism with zkMe as the off-chain credential issuer. Users can prove attributes (age, citizenship, accreditation status) without revealing any PII.

8.1 Midnight's Native Attestation Trees

Midnight's credential model uses Merkle trees stored on-chain with leaves computed from user secrets. The core construction:

```
leaf = persistentHash([domain_separator, user_secret_key])
```

Where: - `domain_separator` is a 32-byte padded string that identifies the credential type (e.g., "age18:", "eures:", "acred:") - `user_secret_key` is the user's private key, held as a witness value in the TEE

The Merkle root of all leaves is published to a Compact contract's ledger state. Individual leaves and proofs are held off-chain by the credential holder.

8.2 Multi-Attribute Attestation Trees

Each credential type maintains its own independent Merkle tree with its own domain separator:

Credential	Domain Separator	Merkle Root Ledger Field	Issuer
Age >= 18	pad(32, "age18:")	age_root: Bytes<32>	zkMe (via OCR + liveness) zkMe (via document verification) zkMe (via financial institution attestation) zkMe (via full KYC flow)
EU Residency	pad(32, "eures:")	eu_res_root: Bytes<32>	
Accredited Investor	pad(32, "acred:")	accred_root: Bytes<32>	
KYC Completed	pad(32, "zkkyc:")	kyc_root: Bytes<32>	

A single Compact circuit can prove membership in multiple trees simultaneously:

```
export circuit verify_eligible(
  // witnesses provided by user's TEE
): [] {
  const sk = secret_key();

  // Prove age >= 18
  const age_leaf = persistentHash<Vector<2, Bytes<32>>>([pad(32, "age18:"), sk]);
  const age_computed_root = hash_merkle(age_leaf, age_sibling());
  assert(age_computed_root == age_root, "age check failed");

  // Prove EU residency
  const eu_leaf = persistentHash<Vector<2, Bytes<32>>>([pad(32, "eures:"), sk]);
  const eu_computed_root = hash_merkle(eu_leaf, eu_sibling());
  assert(eu_computed_root == eu_res_root, "residency check failed");

  // Emit nullifier to prevent reuse (different domain)
  const nf = disclose(persistentHash<Vector<2, Bytes<32>>>([pad(32, "nullf:"), sk]));
  // ... store nullifier
}
```

8.3 zkMe as Off-Chain Issuer

zkMe is the trusted off-chain verification layer between real-world identity and on-chain attestation trees:

Verification Flow:

1. **Identity Capture:** User submits government-issued ID (passport, driver's license) to zkMe's SDK embedded in the Midnight app.

2. **OCR + Liveness:** zkMe extracts document data, runs anti-spoofing liveness detection, and matches the selfie to the document photo.
3. **FHE Biometrics:** Biometric data is processed under Fully Homomorphic Encryption. zkMe's servers never see raw biometric data.
4. **Groth16 Proof Generation:** zkMe generates a Groth16 zk-SNARK proof that the identity claims (age ≥ 18 , citizenship, etc.) hold, without revealing the underlying PII.
5. **Attestation Tree Construction:** zkMe computes `persistentHash([domain_sep, user_secret_key])` for each verified attribute and builds the Merkle tree.
6. **Root Publication:** zkMe publishes the updated Merkle roots to the corresponding Compact contracts on Midnight.
7. **Proof Distribution:** The user receives their Merkle proofs (sibling hashes) and stores them locally (encrypted in TEE).
8. **Soulbound Token Minting:** Optionally, zkMe mints a non-transferable Soulbound Token (SBT) on Midnight as a persistent on-chain anchor indicating verification status (without revealing any PII).

8.4 Nullifier-Based Reuse Prevention

Each verifying service requires a nullifier to prevent the same credential from being used twice (e.g., double-claiming a one-per-person airdrop):

```
nullifier = persistentHash([pad(32, "nullf:"), secret_key])
```

The nullifier is `disclose()`-d in the Compact circuit, making it public on-chain. Properties:

- **Deterministic:** Same secret key always produces the same nullifier for a given domain.
- **Unlinkable:** The nullifier uses a different domain separator ("nullf:") than the attestation leaf ("age18:"), so observers cannot link a nullifier to a specific tree leaf.
- **Stored on-chain:** The contract keeps a set of used nullifiers and rejects any transaction that reuses one.

8.5 Privacy Guarantees

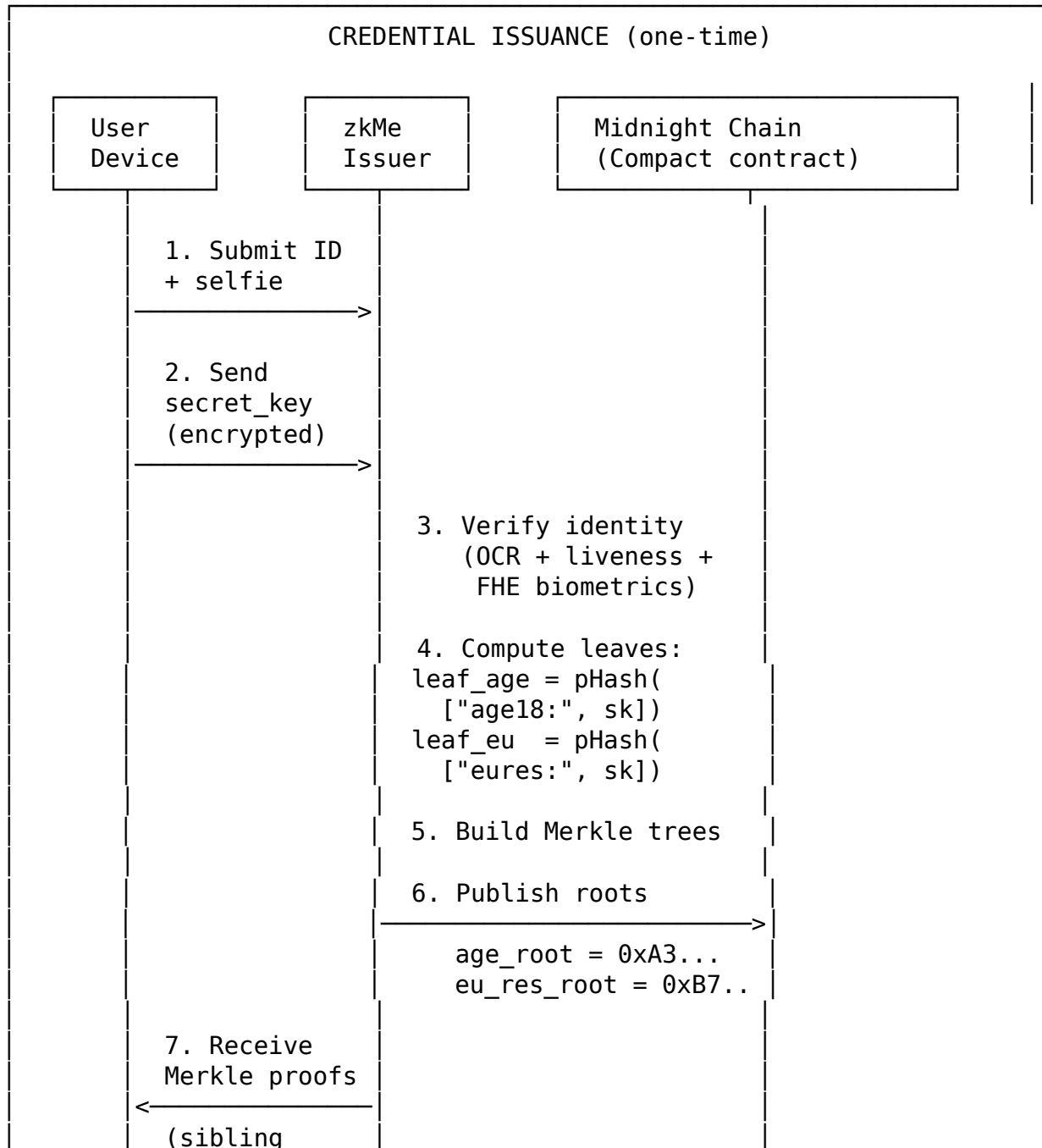
Midnight's Compact compiler enforces privacy at compile time via disclosure analysis:

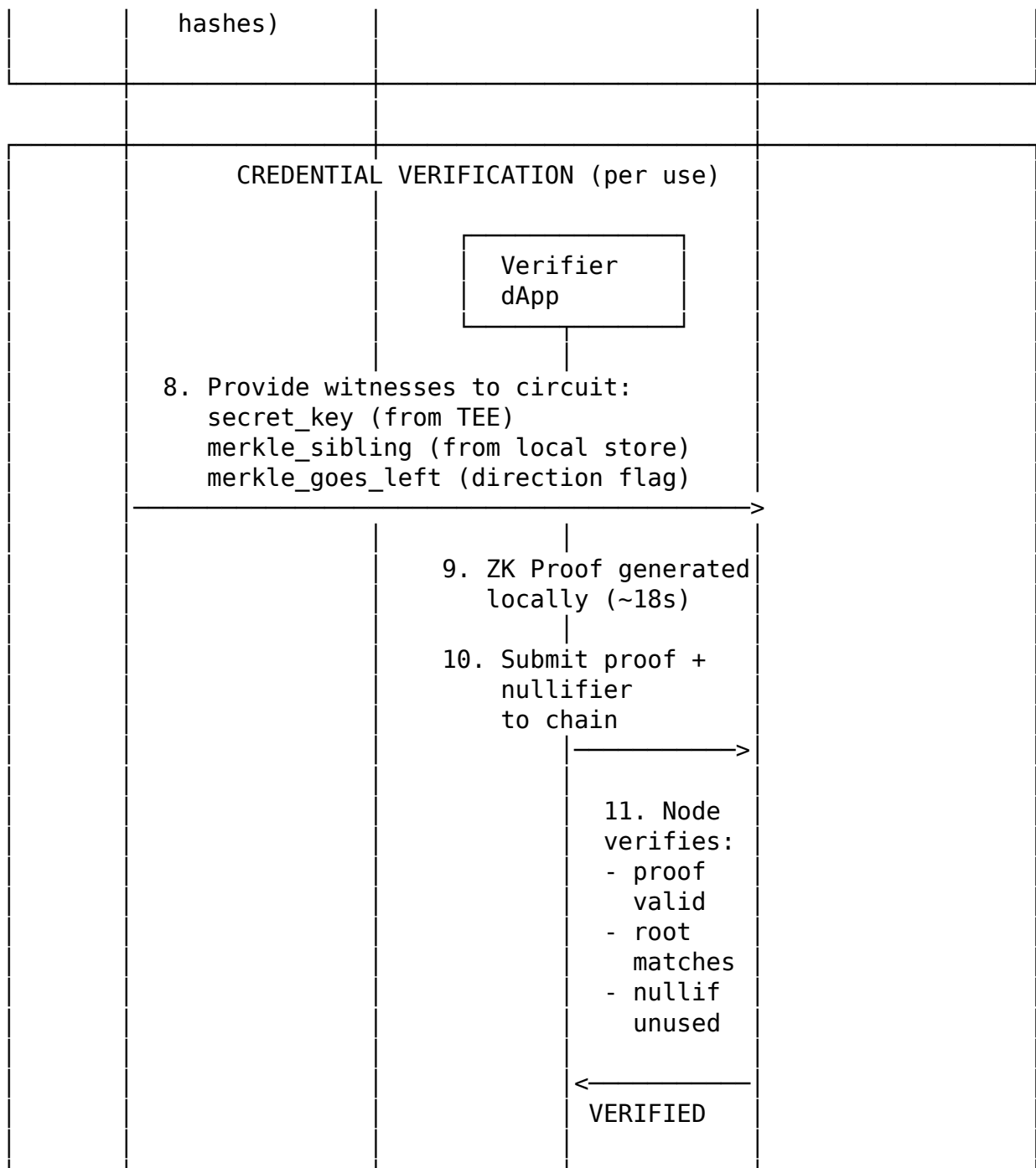
1. **Witness values cannot leak:** Any attempt to `disclose()` a witness value directly (rather than a hash of it) is caught at compile time.
2. **Branch privacy:** Conditional branches on witness values cannot produce different public-side effects. The compiler rejects patterns like `if (witness_age  $\geq$  18) { ledger_approved = true; }` because the ledger write leaks the branch decision.
3. **Unconditional patterns required:** The attestation tree pattern uses `assert()` (which aborts the entire circuit on failure) rather than conditional writes. The only observable outcome is success or failure of the whole proof.

What an observer sees for a credential verification: - A nullifier (unlinkable random-looking 32 bytes) - A boolean result (the proof either verifies or it does not) - The Merkle root being checked against (already public on the contract)

What an observer does NOT see: - The user's secret key - Which leaf in the Merkle tree belongs to the user - The user's actual age, citizenship, or any PII - Any linkage between the nullifier and the Merkle leaf

8.6 Attestation Tree Verification Flow





8.7 Credential Lifecycle Management

SBTs and attestation tree entries are not permanent. Credentials expire, get revoked, and require re-verification. The lifecycle is managed through on-chain state and circuit-level enforcement.

Expiration Every attestation entry carries two timestamp fields stored in the Compact ledger alongside the Merkle root:

Field	Type	Description
issuedAt	uint64	Unix timestamp when the credential was minted
expiresAt	uint64	Unix timestamp after which the credential is invalid

The verification circuit enforces expiration directly:

```
circuit credential_verify:
  public: merkle_root, current_block_time, nullifier
  private: secret_key, merkle_proof, expires_at

  assert blockTimeLt(expires_at) // fails if block time >= expires_at
  assert merkle_verify(root, leaf, proof)
  assert nullifier == pHash([secret_key, domain_sep, nonce])
```

When the block timestamp exceeds expiresAt, proof generation fails. The user cannot produce a valid proof for an expired credential. No on-chain revocation transaction is needed for time-based expiry – the circuit rejects it at proof time.

Revocation Revocation is root-based. The issuer maintains the canonical Merkle tree off-chain. To revoke a user:

1. Issuer removes the user's leaf from the tree.
2. Issuer computes a new Merkle root excluding that leaf.
3. Issuer publishes the new root to the Compact contract via `updateRoot()`.
4. The contract increments `currentRootEpoch`.

Any proof generated against the old root fails verification because the circuit checks:

```
circuit root_epoch_check:
  public: on_chain_root, on_chain_epoch
  private: proof_root, proof_epoch

  assert proof_root == on_chain_root
  assert proof_epoch == on_chain_epoch
```

The revoked user still holds their old Merkle proof and secret key, but these are useless – the root they prove against no longer matches the contract state.

Root Versioning The Compact contract stores a `currentRootEpoch` counter that increments on every root update. This prevents replay attacks where a user submits a proof generated against a previous (pre-revocation) root:

```
ledger {
  age_root: Bytes<32>;
```

```

    age_root_epoch: Counter;
    eu_res_root: Bytes<32>;
    eu_res_root_epoch: Counter;
}

```

Verification nodes reject proofs where `proof_epoch != currentRootEpoch`. The epoch value is a public input to the circuit, so it cannot be forged.

Re-Verification When a credential expires or is revoked, the user must re-verify:

1. User contacts the issuer (e.g., zkMe) and repeats the identity verification process.
2. Issuer generates a new attestation leaf using the user's **new** secret key material (derived from the same seed but with an incremented credential index).
3. Issuer inserts the new leaf into the current Merkle tree and publishes an updated root.
4. A new SBT is minted with a fresh `issuedAt` and `expiresAt`.
5. The old SBT is marked inactive (or simply ignored - expired SBTs fail proof generation).

Using new secret key material for each re-verification cycle prevents cross-epoch linkability. The derivation path includes the credential generation number:

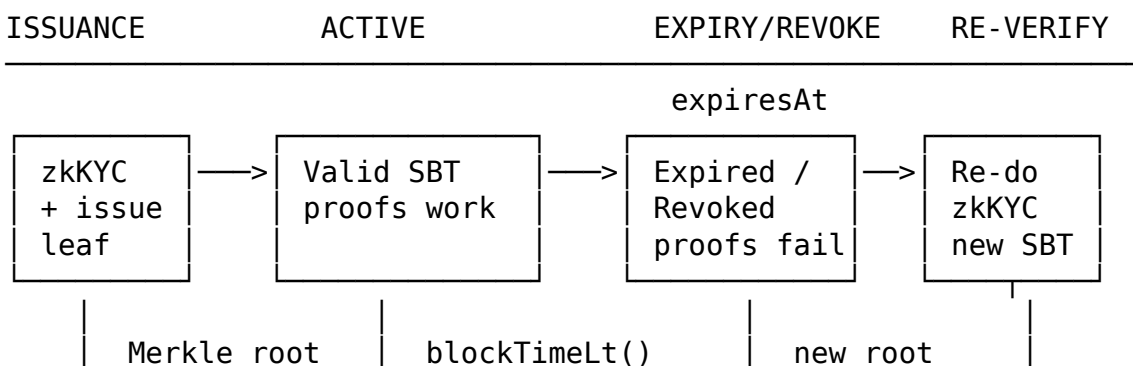
`secret_key_gen_N = BIP32(seed, m/44'/2400'/0'/4/credential_type/N)`

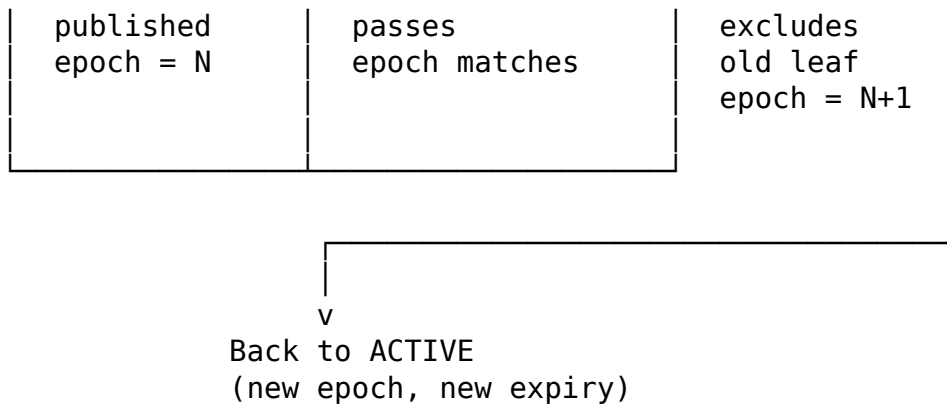
Monitoring and Asynchronous Revocation The issuer runs a background monitoring process:

- Periodically checks external data sources (sanctions lists, regulatory databases, PEP lists).
- When a flagged user is detected, the issuer removes their leaf and publishes a new root.
- The user's existing proofs stop working the next time they attempt to generate one (the on-chain root has changed).
- No notification to the user is required - the proof simply fails.

This is asynchronous and non-interactive. The issuer does not need the user's cooperation to revoke.

Credential Lifecycle Flow





9. Key Management

A single BIP39 seed, generated and stored in the TEE, deterministically derives all keys for all three wallet layers, credential proofs, and authentication operations. Everything in the security model depends on this.

9.1 Derivation Architecture

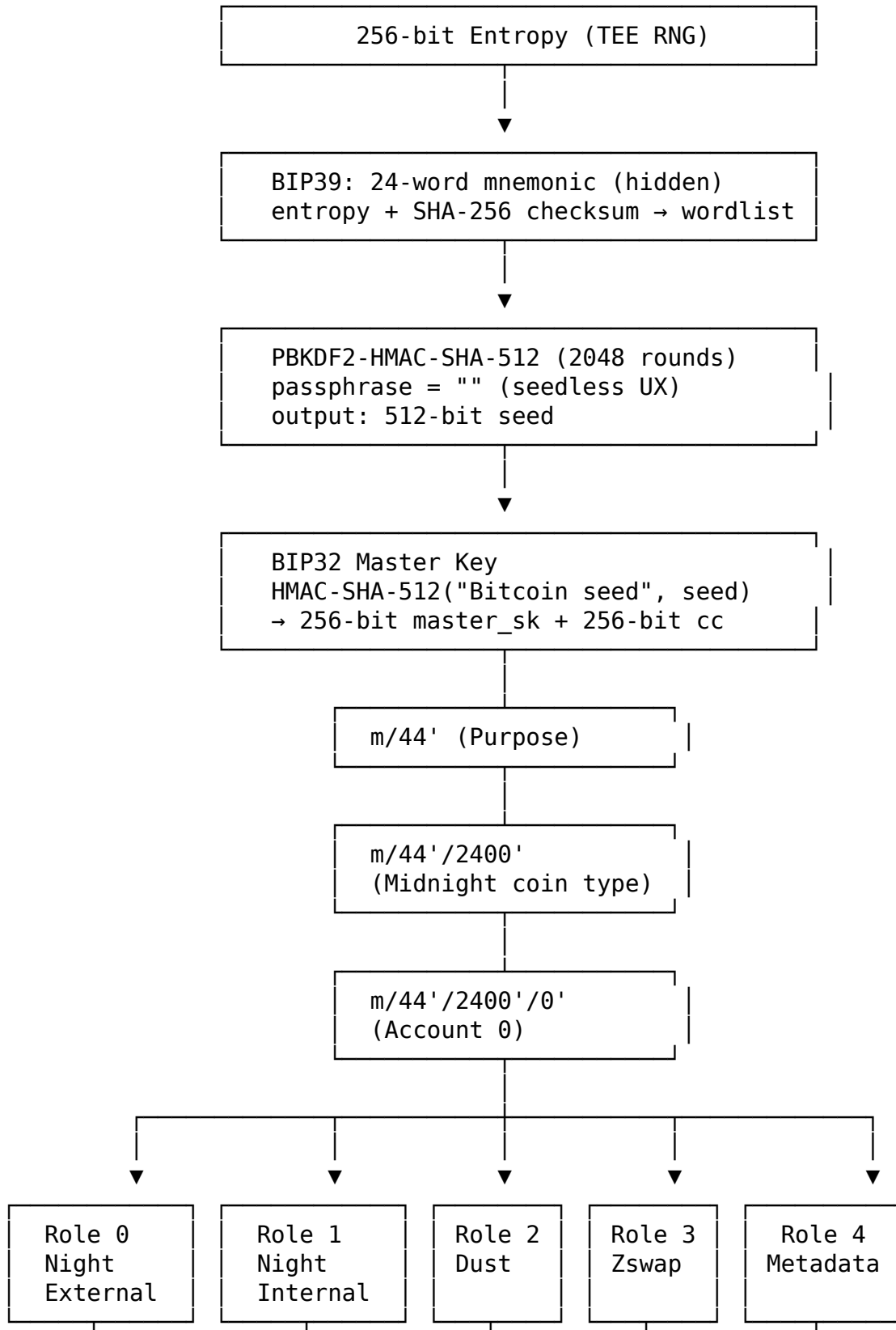
Derivation follows a strict hierarchy:

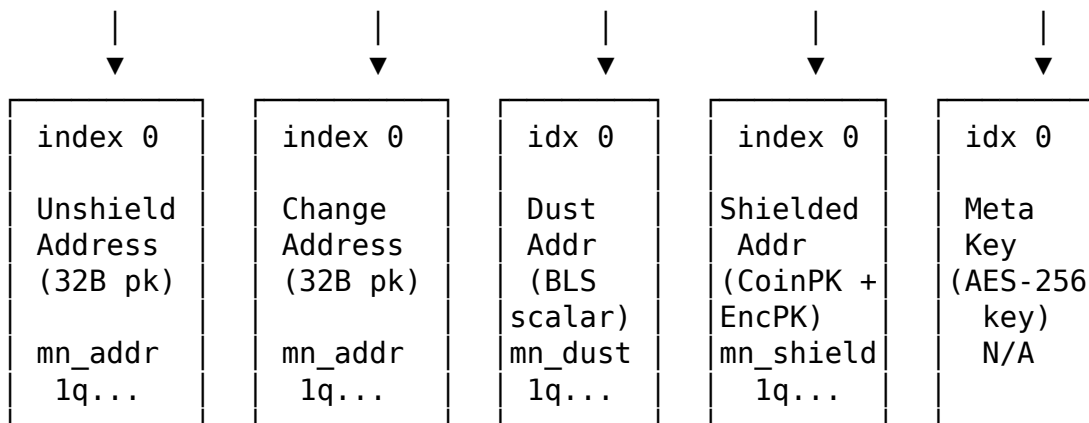
1. **BIP39**: 256-bit entropy → 24-word mnemonic → PBKDF2 → 512-bit seed
2. **BIP32**: 512-bit seed → HMAC-SHA-512 → master key (256-bit private key + 256-bit chain code)
3. **CIP-1852**: Master key → hardened derivation through purpose/coin-type/account → role-based child keys

Midnight uses coin type 2400 (registered BIP44 allocation). Five roles partition keys by function:

Role	Index	Key Type	Used For
NightExternal	0	BLS12-381 scalar	Receiving Night tokens (UnshieldedAddress)
NightInternal	1	BLS12-381 scalar	Change addresses for Night transactions
Dust	2	BLS12-381 scalar	Dust generation registration (DustAddress)
Zswap	3	Jubjub scalar	Shielded operations (ZswapCoinPublicKey + EncryptionPublicKey)
Metadata	4	AES key material	Encrypting contract private state

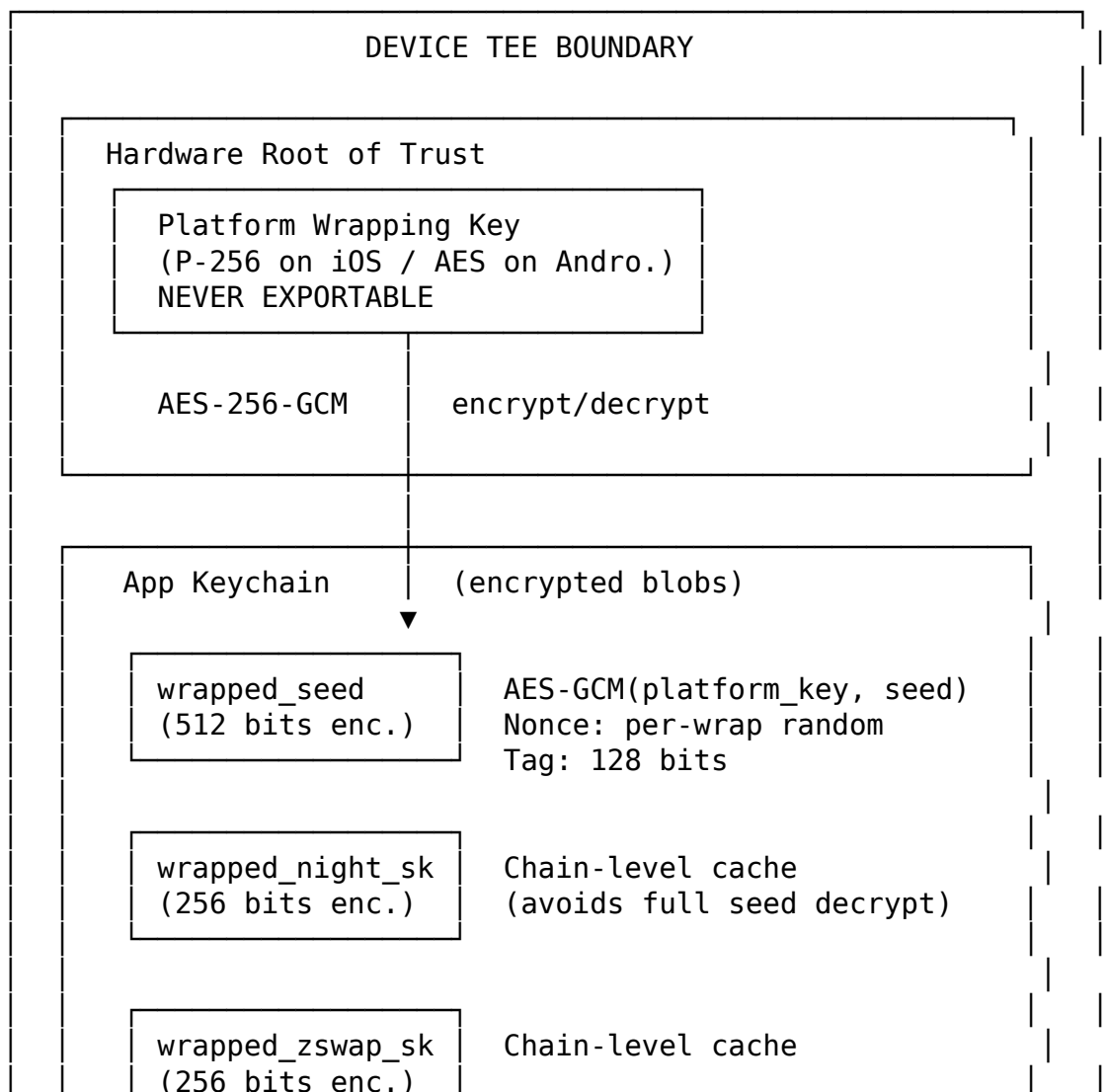
9.2 CIP-1852 Derivation Tree

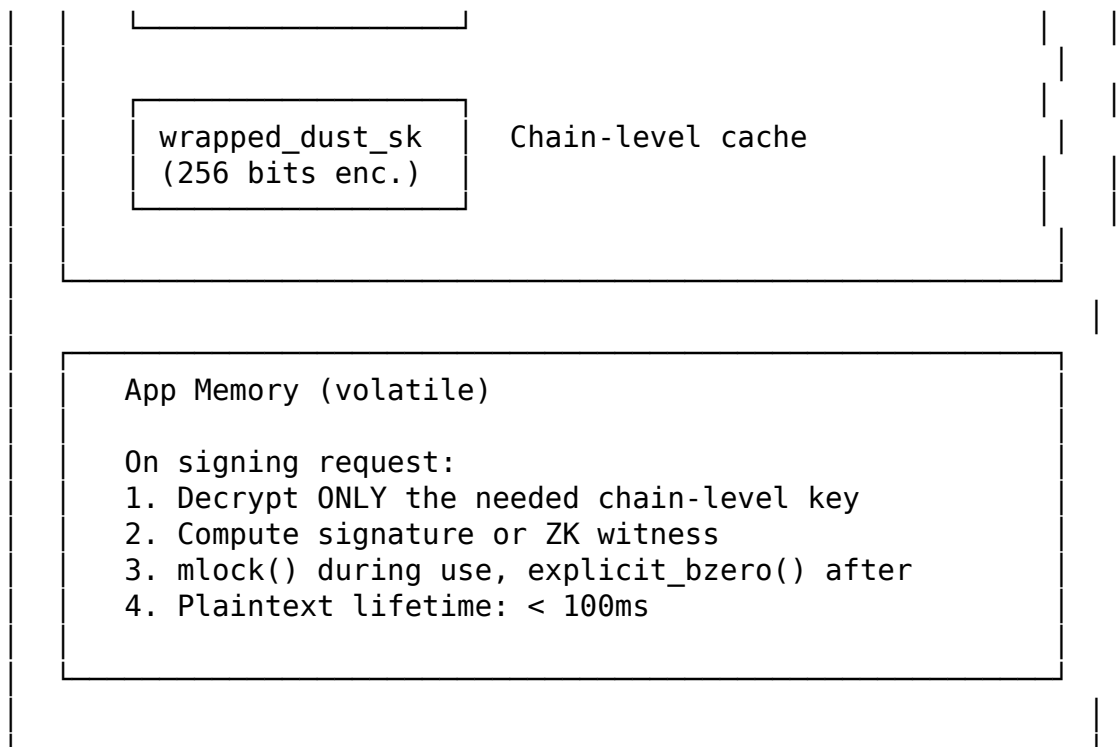




9.3 TEE Wrapping Architecture

The TEE's hardware-isolated key wraps all cryptographic material at rest.





9.4 Signing Operations

The two wallet layers use different signing mechanisms:

Unshielded (Night) transactions: BIP-340 Schnorr signatures. The NightExternal key signs the transaction directly; the node verifies on-chain.

Shielded (Zswap) transactions: ZK witness authorization. The Zswap key is provided as a witness input to the Compact circuit:

```

witness get_key(): Bytes<32>;

export circuit authorize_transfer(): [] {
  const sk = get_key();
  // Circuit proves: persistentHash(["owner:", sk]) == ledger_owner
  // without revealing sk
  const owner_hash = persistentHash<Vector<2, Bytes<32>>>([pad(32, "owner:"), sk]);
  assert(owner_hash == owner, "not authorized");
  // ... transfer logic
}
  
```

The key feeds into the ZK circuit to generate a proof. The proof goes to the chain. The key never appears in the transaction or on-chain state.

9.5 Chain-Level Key Caching

Chain-level caching is the primary mitigation for the CRITICAL seed exposure risk:

1. **At onboarding:** The seed is decrypted once, all five role keys are derived, each is independently wrapped with the TEE's AES-256-GCM key, and the seed plaintext is zeroized.

2. **Day-to-day:** Only the role key for the current operation gets decrypted. A Night transfer decrypts wrapped_night_sk; a shielded operation decrypts wrapped_zswap_sk. The seed blob is never touched.
3. **At recovery:** The seed is reconstructed from DeRec shares, all role keys are re-derived and re-wrapped, and the seed is zeroized again.

This drops the seed’s plaintext exposure from “every transaction” to “initial setup and recovery only.”

9.6 Multi-Account and Identity Switching

A single seed supports multiple independent accounts on the same device. This is not a new protocol — it falls directly out of CIP-1852’s account index parameter that is already part of the derivation tree.

Derivation Path The account index lives at depth 3 of the BIP-44/CIP-1852 path:

```
m / 44' / 2400' / {account}' / role / index
                ^^^^^^^
                0 = default
                1 = second identity
                2 = third identity
                ...
```

Account 0 is created at onboarding. The user never interacts with derivation paths. The wallet UI exposes an “Add Account” action that increments the account index internally, derives the new subtree, and wraps the resulting keys.

What Each Account Gets Every account is a fully independent identity:

Account 0 ("alice.midnight")	Account 1 ("businessalice.midnight")
├─ Name: alice.midnight	├─ Name: businessalice.midnight
├─ Wallet Layer 1: Night keys	├─ Wallet Layer 1: Night keys
├─ Wallet Layer 2: Zswap keys	├─ Wallet Layer 2: Zswap keys
├─ Wallet Layer 3: Function-call keys	├─ Wallet Layer 3: Function-call keys
├─ Attestation leaves (MeID, age...)	├─ Attestation leaves (separate set)
└─ DeRec helper set (can differ)	└─ DeRec helper set (can differ)

Each account derives its own secret keys from the seed, so each account produces different attestation tree leaves, different nullifiers, and different wallet addresses. There is zero on-chain overlap between accounts derived from the same seed.

Account Switching The wallet UI provides a simple toggle:

1. User taps the active account badge (e.g., “alice.midnight”).
2. A picker shows all created accounts.
3. Selecting an account switches the active context: which name resolves, which keys sign transactions, which credentials are available for disclosure.

4. The underlying TEE-wrapped key blobs are already present — switching accounts does not require seed decryption. The wallet loads the wrapped keys for the selected account index.

Cross-Account Privacy Accounts are unlinkable by design. The privacy guarantee comes from the derivation structure:

- Different account index produces different secret keys at every role.
- Different secret keys produce different `persistentHash("owner:", sk)` values, meaning different UTXO owners.
- Different secret keys produce different attestation tree leaves, meaning different Merkle proofs.
- Different attestation leaves produce different nullifiers when spending or disclosing.

An observer who sees transactions from Account 0 and Account 1 cannot determine they belong to the same person without compromising the seed itself. This is not a bolt-on feature — it is a structural consequence of how CIP-1852 derivation and Compact's hash-based ownership work together.

Recovery Implications DeRec recovers the BIP39 seed. The seed is the root of all accounts. Therefore:

- **Recovery restores all accounts.** Reconstruct the seed, re-derive account 0, 1, 2, ..., re-wrap each account's keys in the new device's TEE.
- **Selective recovery is not possible.** You cannot recover Account 1 without also recovering Account 0. The seed is atomic.
- **Account discovery after recovery:** The wallet scans account indices (0, 1, 2, ...) and checks for on-chain activity at each derived address. Accounts with history are restored; the scan stops after N consecutive empty accounts (default N=20, same convention as BIP-44 gap limit).

10. Recovery Design

Key loss is the hard problem in self-sovereign systems. The design uses DeRec for seed-level recovery and the multi-key account model for device-level recovery.

10.1 DeRec Seed Recovery

DeRec uses Shamir's Secret Sharing with a (3,5) threshold to distribute the BIP39 seed across five trusted helpers. Any three shares reconstruct the seed.

Share Distribution

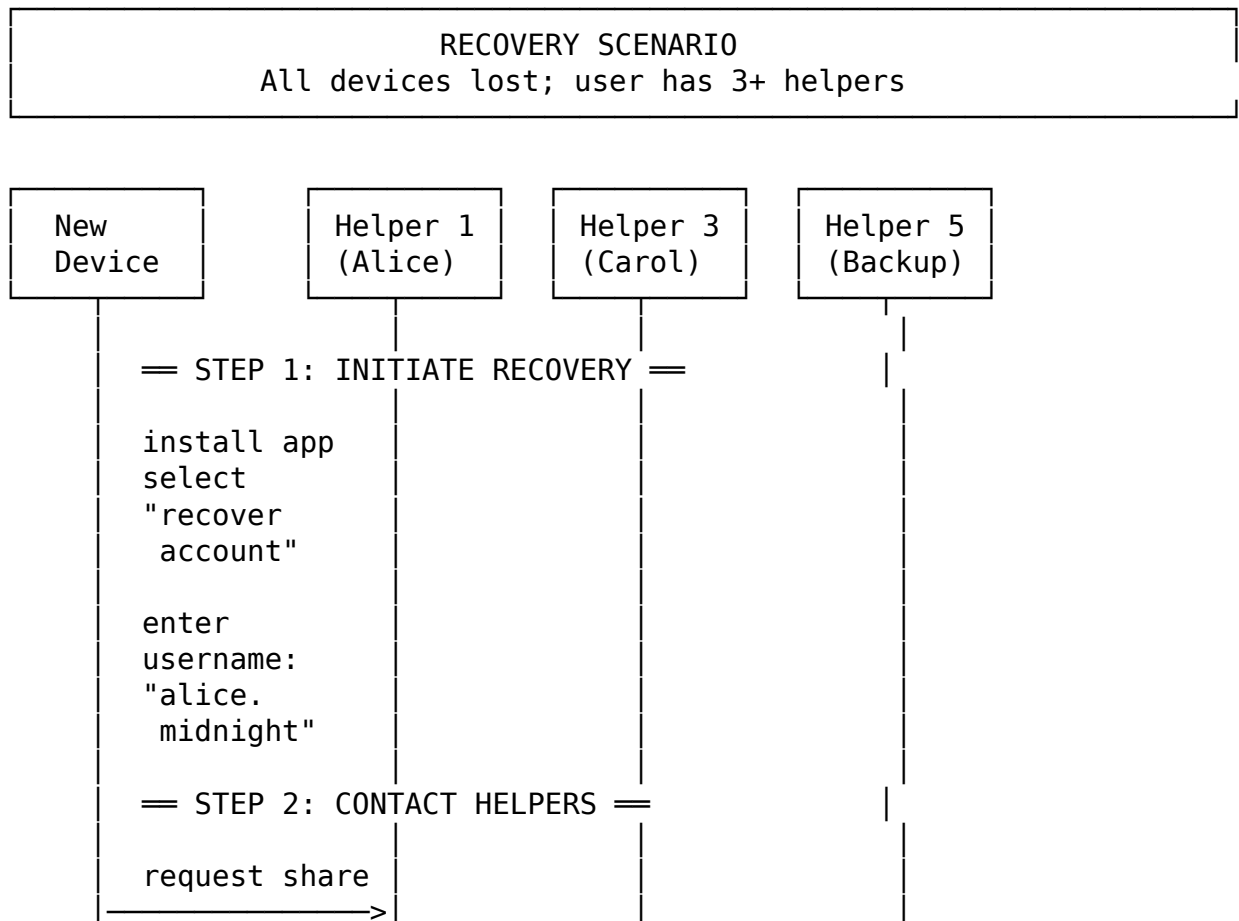
1. **Split:** The raw 512-bit BIP39 seed is split into 5 shares via Shamir's Secret Sharing over $GF(2^8)$. Each share is 512 bits plus a share index byte.

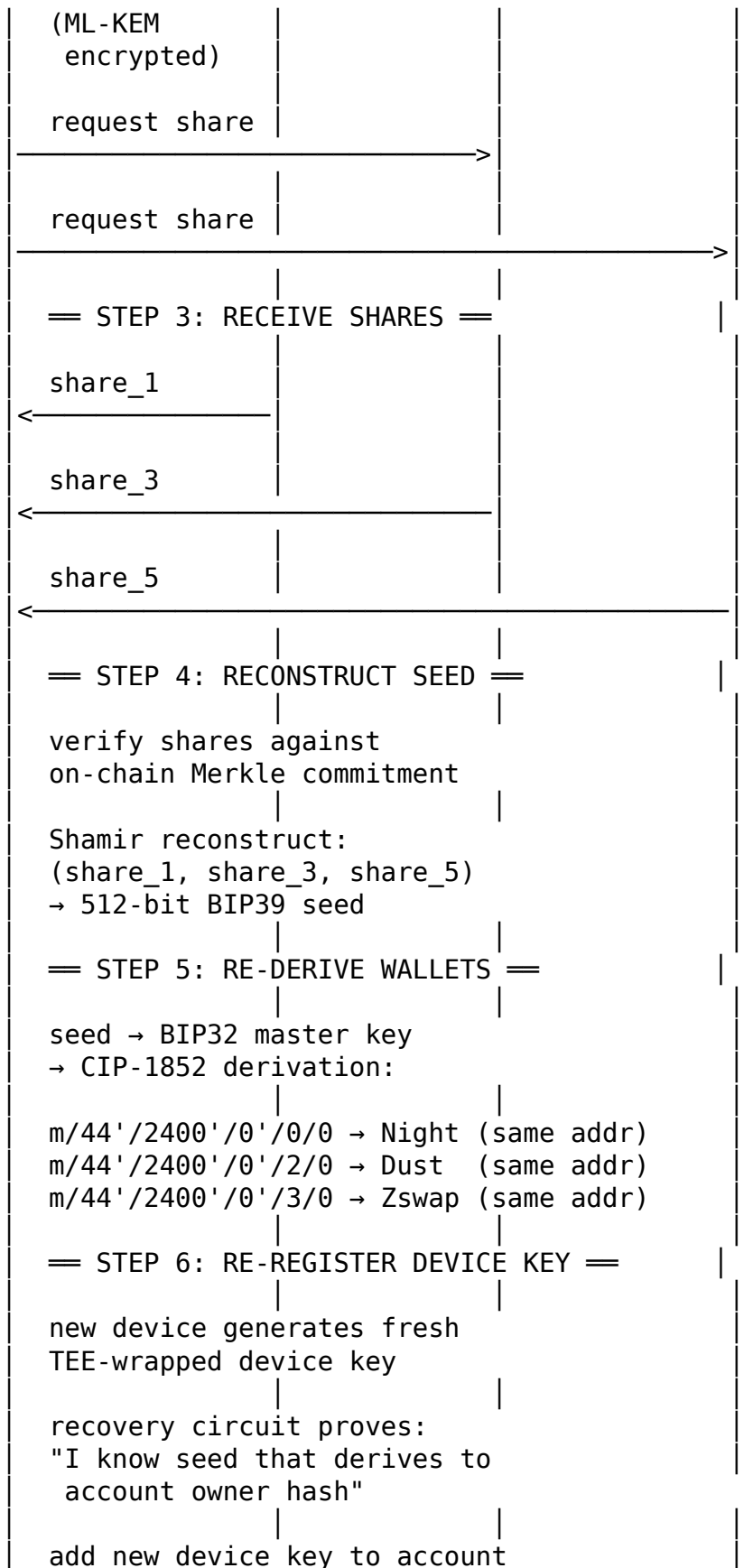
2. **Merkle commitment:** A Merkle tree over the share hashes - $\text{root} = \text{merkle-Root}([H(\text{share}_1), H(\text{share}_2), \dots, H(\text{share}_5)])$ - is published on-chain, binding the share set to the user's account.
3. **Post-quantum transport:** Each share is encapsulated with the helper's ML-KEM-768 public key. ML-KEM (FIPS 203) gives IND-CCA2 security against quantum adversaries at a 192-bit classical security level.
4. **Helper storage:** Helpers store their encrypted share in their own TEE or secure storage. The share is opaque; helpers cannot read it.

Ongoing Verification

- **Daily challenge-response:** The app sends a random challenge to each helper. The helper proves share possession by returning $\text{HMAC-SHA-256}(\text{share}, \text{challenge})$. The app verifies against its local copy of the share hashes.
- **Liveness monitoring:** If a helper fails to respond within 48 hours, the user is prompted to designate a replacement and reshare.
- **Epoch-based resharing:** Every 90 days, shares are re-randomized (same secret, fresh polynomial). This limits the window for share compromise. Daily verifications are suspended during resharing to prevent race conditions.

10.2 Recovery Flow





```

remove all old device keys
= STEP 7: CREDENTIALS SURVIVE =
same secret_key derived from
same seed → same attestation
tree leaves → Merkle proofs
still valid

SBTs on-chain: unchanged
Nullifiers on-chain: unchanged
(no double-claim risk)

= STEP 8: RESHARE TO NEW HELPERS =
optionally replace helpers
and reshare with fresh polynomial

```

10.3 Why Credentials Survive Recovery

This is worth spelling out. Attestation tree leaves derive from the user's secret key, and the secret key derives from the BIP39 seed. Recovering the seed recovers all credential capabilities:

```

seed → BIP32 → CIP-1852 → secret_key (same as before)
secret_key + domain_sep → persistentHash → same Merkle leaf
same Merkle leaf + same tree → same Merkle proof works

```

On-chain state is unaffected by recovery: - **Merkle roots**: Published by the issuer, independent of user's device. - **SBTs**: Minted to the account (not the device key), persist on-chain. - **Nullifiers**: Already on-chain; the recovered user cannot double-claim because the same nullifier regenerates from the same key.

The only state requiring manual recovery is **contract private state** - the AES-256-GCM encrypted local storage that Compact contracts use for per-user data. Users should keep encrypted backups, or re-sync from dApp providers.

10.4 Multi-Key Account Recovery (Device-Level)

When only one device is lost (but others remain active):

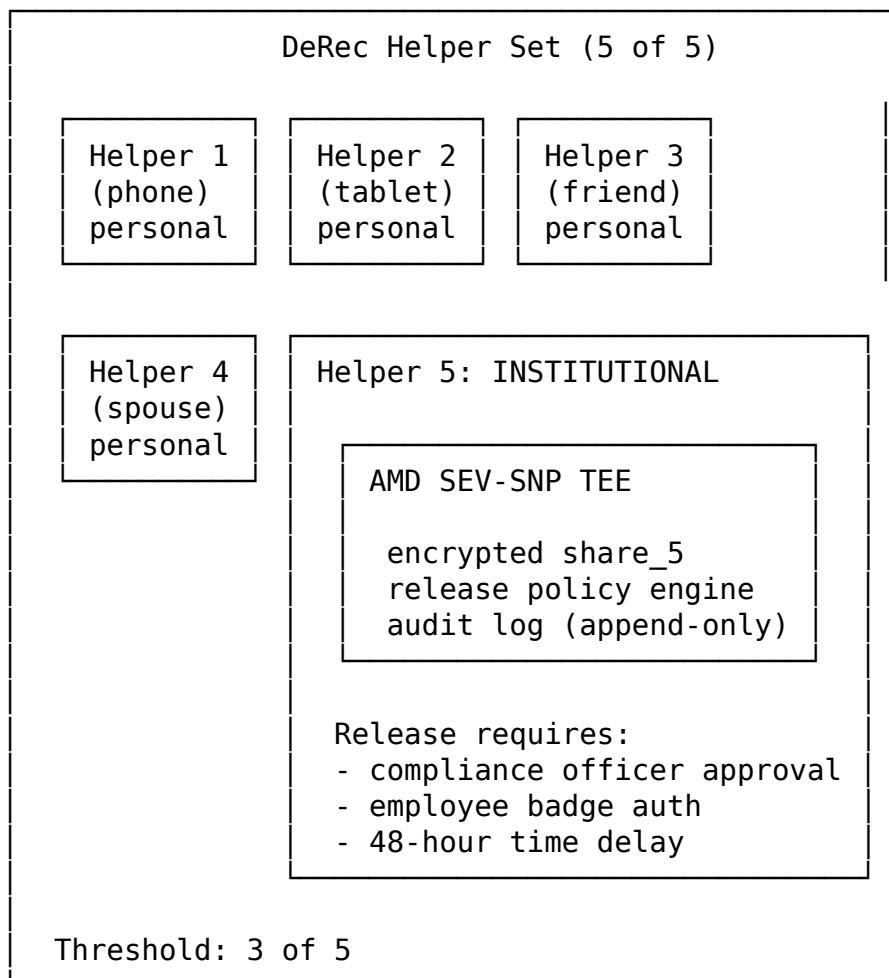
1. **Remaining device** signs a `remove_key` transaction, revoking the lost device's full-access key.
2. **New device** generates its own seed in its own TEE.
3. **Remaining device** signs an `add_key` transaction, granting the new device's key full-access status.
4. No DeRec involved. No seed transfer. The account remains unchanged.

This is the common case. Full DeRec recovery is a last resort when all devices are lost simultaneously.

10.5 Institutional Recovery

Enterprise Ethan cannot tell auditors that seed recovery depends on his college roommate answering a push notification. Regulated entities need a recovery path that satisfies compliance requirements without abandoning self-custody. The solution: one of the five DeRec helpers is an institutional recovery service.

Architecture The institutional helper is a server operated by the enterprise (or a contracted recovery provider) that participates in the DeRec protocol as a standard helper. It holds one encrypted share of the user's seed, identical in format to any other helper's share.



TEE Isolation The institutional helper runs inside a Trusted Execution Environment (AMD SEV-SNP or equivalent). The TEE guarantees:

- The share is encrypted at rest and only decrypted inside the enclave.

- The institution's IT administrators cannot extract the share from memory.
- Remote attestation proves to the DeRec protocol that the helper is running the correct, unmodified release policy code.

The institution holds one share. It never sees the seed. Even if the institutional helper is fully compromised, the attacker gets one share of five – useless without two more.

Release Policy The institutional helper does not release its share on a simple API call. A configurable policy engine gates release:

Factor	Description
Compliance officer approval	Named officer signs a release authorization
Employee authentication	Badge tap + PIN or biometric from the account holder
Time delay	48-hour waiting period after approval (configurable)
Reason code	Mandatory field: device loss, employee offboarding, legal hold
Rate limiting	Maximum 1 release attempt per 30-day window

All factors must pass. The policy is enforced inside the TEE – the institution cannot bypass it without replacing the enclave code (which invalidates the remote attestation).

Threshold Arithmetic The threshold model remains 3-of-5:

- **User recovers without institution:** 3 of the 4 personal helpers respond. The institutional share is not needed. This is the normal case.
- **User recovers with institution:** Institutional share + 2 personal helpers = 3 of 5. This covers cases where the user has lost contact with most personal helpers.
- **Institution recovers without user:** Impossible. The institution holds 1 share. It needs 2 more. Personal helpers only release shares to authenticated recovery requests from the user's new device (per DeRec protocol). The institution alone cannot reconstruct the seed.

This is the critical property: the institution can assist recovery but cannot initiate it unilaterally.

Audit Trail The institutional helper logs every event to an append-only audit store:

```
{
  "event":      "share_release_requested",
  "timestamp":  "2026-03-15T14:30:00Z",
  "requester":  "ethan@enterprise.com",
  "approver":   "compliance-officer@enterprise.com",
  "reason":     "device_loss",
  "status":     "pending_time_delay",
  "delay_expires": "2026-03-17T14:30:00Z"
```

}

This log satisfies regulatory reporting requirements (SOC 2, internal audit). The institution can demonstrate:

- When recovery was requested and by whom.
- Who approved it.
- Whether the time delay was respected.
- Whether the share was ultimately released or the request was cancelled.

The audit log does not contain the share itself or any seed material.

Integration with Enterprise Identity For enterprise deployments, the institutional helper can integrate with existing identity infrastructure:

- **SSO:** Employee authenticates via the organization's identity provider before initiating recovery.
- **HR lifecycle:** When an employee is offboarded, the institutional helper can be instructed to stop responding to that user's recovery requests (effectively reducing the helper set to 4, with threshold still 3).
- **Multi-sig approval:** Large organizations can require N-of-M compliance officers to approve a release, adding an internal threshold on top of DeRec's threshold.

None of this changes the DeRec protocol. The institutional helper is a standard helper that happens to have a policy engine in front of its share-release function.

10.6 Emergency Procedures

Normal operations fail gracefully – retries, queues, degraded modes. Emergency procedures cover the scenarios where graceful degradation is not enough: coordinated attacks, permanent vendor shutdowns, contract bugs, and full network outages. Each procedure assumes the worst case and specifies what happens to user funds and identity.

Scenario 1: DeRec Helper Network Compromise A coordinated attacker compromises 3 or more helpers simultaneously, reaching the recovery threshold. This is the single most dangerous scenario in the system because it allows seed reconstruction without the user's involvement.

Prevention (before the attack).

The helper selection UI enforces geographic diversity. When a user adds helpers, the system checks that no 3 helpers share the same country or the same cloud provider (for institutional helpers). If a user tries to add a third helper in the same region, the UI displays a warning and requires explicit override:

! Geographic concentration risk

3 of your 5 helpers are in the same region.
If that region is compromised, your recovery

threshold could be reached by an attacker.

Recommendation: choose a helper in a different country or continent.

[Choose Different Helper] [Accept Risk]

Detection (during the attack).

Anomaly detection runs on the DeRec helper protocol layer. If 2 or more helpers access their shares within a 1-hour window without a corresponding recovery request initiated by the user's device, the system triggers an alert:

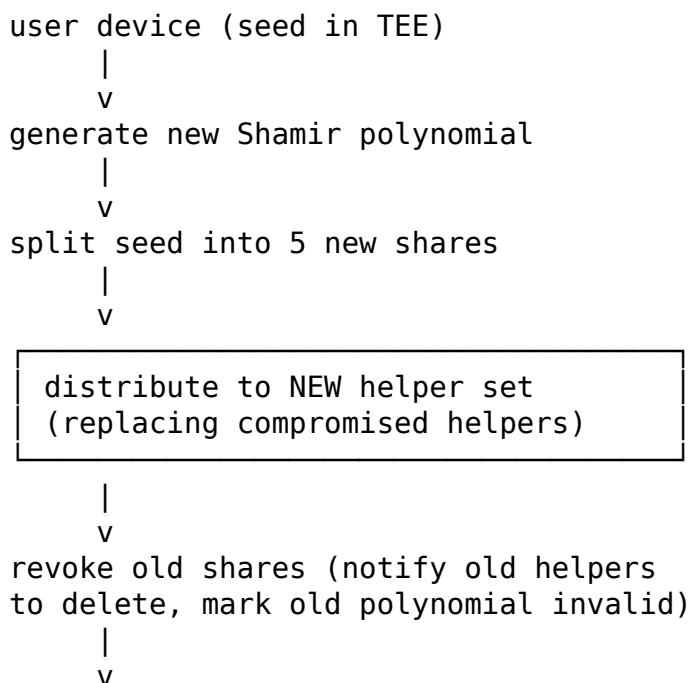
1. Push notification to all of the user's registered devices.
2. SMS/email to the user's recovery contact (if configured).
3. Automatic 24-hour freeze on share release from all remaining helpers.

The freeze buys time. The attacker has obtained some shares but cannot reach threshold if the remaining helpers are locked.

Response (after detection).

Emergency resharing. The user initiates resharing from any device that still holds the seed (or from a device that has already recovered it via the uncompromised helpers). Resharing generates a new Shamir polynomial, produces new shares, distributes them to a new set of helpers, and invalidates all old shares. The attacker's captured shares become useless – they correspond to a polynomial that no longer reconstructs the seed.

Emergency Resharing Flow:



old shares now reconstruct nothing

If the attacker has already reconstructed the seed before the user reshares, the damage is done – the attacker has the seed and can derive all keys. The only mitigation at that point is to transfer all assets to a new wallet generated from a new seed. This is why prevention (geographic diversity) and detection (anomaly alerts) matter more than response.

Scenario 2: zkMe Permanent Shutdown zkMe is an external vendor. If they shut down permanently, no new identity credentials can be issued through their service. The question is whether existing credentials survive.

They do. Here is why:

- The user’s attestation tree leaves are derived from the user’s `secret_key`, not from zkMe’s infrastructure. The leaf value is `persistentHash(secret_key, attribute_type, attribute_value)`. zkMe’s role was to verify the attribute and sign a certificate that the Compact circuit checked during leaf insertion. Once the leaf is in the tree, zkMe is no longer in the loop.
- Merkle roots are stored on-chain. They persist regardless of zkMe’s operational status.
- SBTs (MeID, age verification) are on-chain tokens. They do not phone home to zkMe.
- Nullifiers derived from the user’s secret key remain valid for double-claim prevention.

What breaks. Users cannot obtain new credentials or refresh expired ones. The MeID SBT has an expiration date – once it expires, the user cannot renew it through zkMe.

Recovery path. The DAO designates a replacement credential issuer. The replacement issuer re-verifies users (biometric check, document scan) and issues new signed certificates. The Compact circuit that checks certificate signatures is updated (via DAO governance) to accept the new issuer’s public key. Existing attestation tree leaves do not need to change – only the certificate verification logic is updated. Users who need to add new leaves or refresh expired SBTs go through the replacement issuer.

Timeline: replacement issuer onboarding takes weeks to months. During the gap, existing credentials work but new issuance is paused.

Scenario 3: Proof Server Extended Outage The proof server generates ZK proofs for transactions. If it goes down, no new proofs can be created, and no shielded transactions can be submitted.

User experience. Transactions queue locally on the device. The wallet UI shows:

Proof service maintenance

Your transaction has been saved and will complete automatically when service resumes.

```
Queued transactions: 2
Estimated wait: checking...
```

```
Your funds are safe. Shielded UTXOs remain
yours -- they just can't move until proofs
can be generated again.
```

Fund safety. No funds are at risk. Shielded UTXOs are committed on-chain and owned by the user's keys. The proof server does not hold keys, does not hold funds, and does not have custody of anything. It is a compute service. Its absence means transactions stall, not that funds are lost.

Recovery. When the proof server comes back online, queued transactions are submitted in order. The wallet drains its local queue automatically. If proof server infrastructure is permanently lost, the proof generation code is open-source – anyone can run a replacement instance. The only constraint is hardware: proof generation requires sufficient compute (see Section 5.8 for proof pipeline details).

Scenario 4: Name Registry Contract Bug The naming registry is a Compact contract on-chain. If a bug is discovered (incorrect ownership logic, broken resolution, state corruption), names could become unresolvable or transferable to the wrong party.

Immediate impact. Names stop resolving correctly. Users sending to `alice.midnight` might get an error or (worst case) resolve to the wrong address. Addresses still work directly – only name-to-address resolution is affected.

Response.

1. The DAO deploys a patched version of the registry contract.
2. The state migration circuit reads ownership records from the old contract and writes them to the new contract. This is an on-chain operation that preserves all name-to-owner mappings.
3. The resolver contract is updated to point to the new registry.
4. Wallet software updates the registry contract address in the next release (or via a well-known on-chain pointer that wallets already check at startup).

User action. None required. Name resolution breaks temporarily (hours to days depending on DAO response time), then resumes automatically after the migration. Users who need to send funds during the outage use raw addresses instead of names.

Scenario 5: Full Network Outage All Midnight nodes go offline. No blocks are produced. No transactions are processed.

Impact. Every on-chain operation stops: token transfers, name registration, credential issuance, proof verification. The blockchain is frozen.

Fund and key safety. The user's seed is in the TEE. The user's keys are derived from the seed. Neither depends on the network being online. DeRec helper shares are stored on helpers' devices, also independent of the network. Nothing is lost.

Recovery. When nodes come back online and consensus resumes, the wallet syncs the chain state from where it left off. Locally queued transactions are submitted. No user action is required beyond waiting.

Emergency Procedure Summary

Scenario	Impact	User Action Required	Expected Resolution Time
DeRec helper network compromise	Seed at risk if 3+ helpers compromised simultaneously	Emergency reshare from surviving device; transfer assets if seed already extracted	Minutes (reshare) to hours (asset transfer)
zkMe permanent shutdown	No new credentials; existing credentials valid until expiration	None immediately; re-verify with replacement issuer when available	Weeks to months for replacement issuer
Proof server extended outage	Transactions queue locally; no fund loss	None; transactions auto-submit on recovery	Hours to days
Name registry contract bug	Name resolution breaks; addresses still work	Use raw addresses during outage	Hours to days (DAO migration)
Full network outage	All on-chain operations halt; keys and funds safe	None; wallet auto-syncs on recovery	Depends on outage cause

The common thread: user keys and funds are never at risk from infrastructure failures. The worst case for any infrastructure outage is temporary loss of functionality, not loss of assets. The one exception is a DeRec helper compromise where the attacker

reconstructs the seed before the user can reshare – that is a true key compromise, and the only response is moving assets to a new wallet.

11. Security Model

11.1 Threat Model

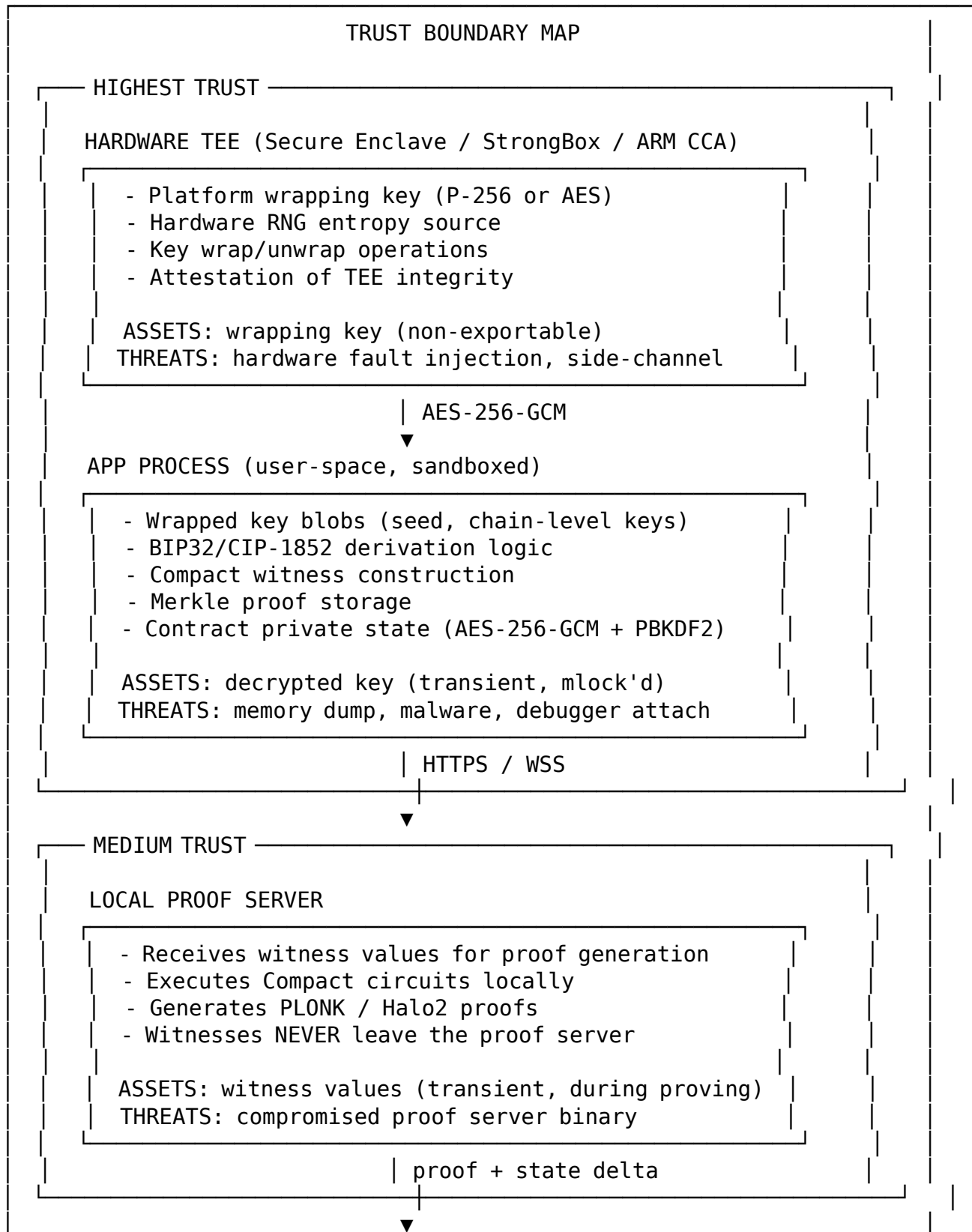
Threats are considered at every layer: hardware, protocol, and social engineering. Findings use the standard CRITICAL/HIGH/MEDIUM/LOW/INFO severity scale.

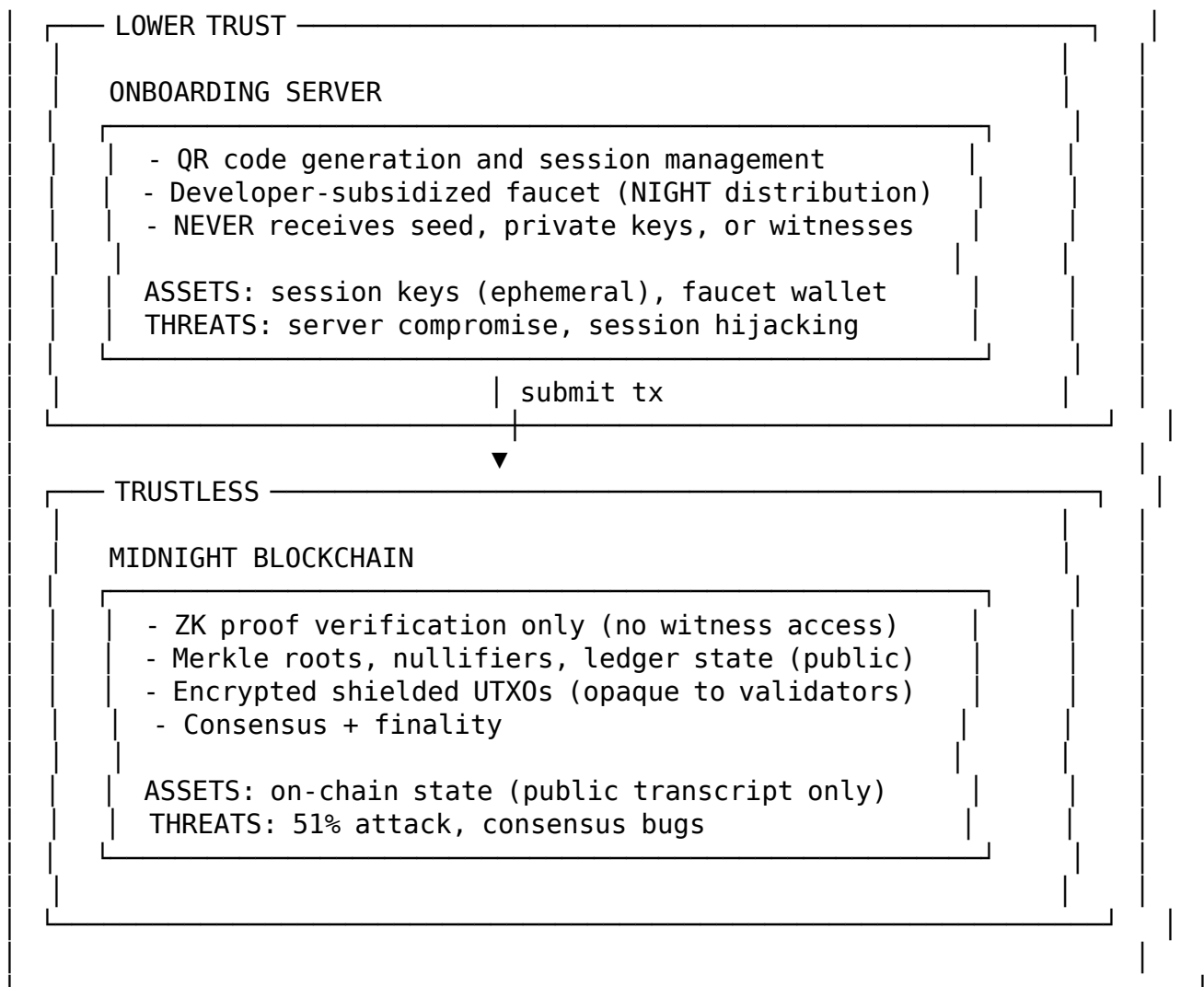
11.2 Findings Summary

ID	Severity	Threat	Mitigation	Residual Risk
S-01	CRITICAL	Seed exposure in app memory during derivation	Chain-level key caching; seed decrypted only at setup/recovery; mlock + explicit_bzero	Brief window (~100ms) where seed is in volatile memory
S-02	HIGH	iOS P-256 Secure Enclave cannot natively hold BLS12-381 keys	Use P-256 as AES-256-GCM wrapping key only; BLS12-381 seed wrapped symmetrically	Wrapping key is hardware-protected but seed itself is software-encrypted
S-03	HIGH	QR protocol: replay, relay, MitM attacks	Server-authenticated ECDH; 60s timestamp; single-use session_id; visual confirmation code	Physical shoulder-surfing of QR remains possible (60s window)
S-04	HIGH	Single seed controls all three wallet layers	DeRec (3,5) threshold; BIP-85 compartmentalization option; chain-level caching limits exposure	Seed compromise = total account compromise until DeRec recovery executes

ID	Severity	Threat	Mitigation	Residual Risk
S-05	MEDIUM	Proof generation latency (~18-21s) enables timing attacks	Not a direct security risk; progress indicator for UX; parallel proof generation where possible	Users may abandon slow transactions, leading to incomplete state
S-06	MEDIUM	Quantum computers threaten BLS12-381 signatures	DeRec uses ML-KEM (PQ-safe); monitor NIST PQ signature standards; migration path via key rotation	BLS12-381 signing keys vulnerable to future quantum attacks
S-07	MEDIUM	Compromised DeRec helper collusion (2 of 5)	(3,5) threshold requires 3 colluding helpers; epoch-based resharing limits window; daily verification detects missing helpers	Social engineering of 3 helpers remains theoretical risk
S-08	MEDIUM	Malicious dApp with function-call key	DUST allowance limits; circuit-scoped access; user can revoke at any time	dApp can drain allocated DUST; cannot access NIGHT or keys
S-09	LOW	Name squatting / phishing names	Commit-reveal prevents frontrunning; homoglyph detection in app UI; dispute resolution via DAO	Similar-looking names remain a social engineering vector
S-10	INFO	Midnight compiler-enforced privacy	Compact disclosure analysis catches witness leaks at compile time; no mitigation needed — this is a strength	Compiler bugs could theoretically bypass disclosure analysis

11.3 Trust Boundary Diagram





11.4 Critical Attack Vectors and Mitigations

S-01: Seed Exposure in App Memory (CRITICAL) **Attack:** Malware or memory dump captures the BIP39 seed during derivation or key use.

Mitigation stack: 1. **Chain-level caching:** Seed is decrypted only during initial setup and recovery. Day-to-day operations use independently-wrapped chain-level keys. 2. **Minimal decryption window:** When the seed must be decrypted, it exists in plaintext for <100ms (derivation time). 3. **Memory locking:** `mlock()` / `VirtualLock()` prevents the seed from being swapped to disk. 4. **Zeroization:** `explicit_bzero()` / `SecureZeroMemory()` overwrites the memory region immediately after use. 5. **Process isolation:** Mobile OS sandboxing prevents other apps from reading process memory.

Residual risk: A device with root/jailbreak access and a debugger attached can capture the transient plaintext. This is accepted as a physical-access compromise scenario.

S-02: iOS P-256 / BLS12-381 Incompatibility (HIGH) Attack: The iOS Secure Enclave only supports P-256 (NIST) curves. Midnight uses BLS12-381 and Jubjub natively. Storing BLS12-381 keys directly in the Secure Enclave would fail.

Mitigation: Use the Secure Enclave only for symmetric key wrapping. It generates a P-256 key pair, but we only use it to derive a symmetric AES-256-GCM key via ECDH-with-self or direct key generation. The BLS12-381 seed material is encrypted with this AES key and stored in the iOS Keychain. The Secure Enclave never needs to understand BLS12-381 – it just wraps and unwraps a blob.

S-03: QR Protocol Attacks (HIGH) Attack vectors: - **Replay:** Attacker captures a QR code and replays it later. - **Relay:** Attacker relays QR content to a remote device in real-time. - **MitM:** Attacker intercepts the ECDH exchange.

Mitigations: - **Timestamp:** 60-second validity window eliminates delayed replay. - **Single-use session:** session_id is invalidated server-side after first use. - **Server authentication:** QR payload is signed with pinned server key. - **Visual confirmation:** 4-digit code displayed on both device and onboarding screen for out-of-band MitM detection. - **ECDH forward secrecy:** Ephemeral keys mean past sessions cannot be decrypted even if the server's long-term key is later compromised.

12. Cross-Platform Considerations

The onboarding system must work identically on iOS, Android, and laptops. The core invariant: **the same BIP39 seed produces the same three wallet addresses on any platform.** Platform differences are confined to the TEE wrapping layer.

12.1 Platform TEE Comparison

Feature	iOS Secure Enclave	Android StrongBox	Laptop (ARM CCA / TPM)
Hardware isolation	Dedicated coprocessor	Separate secure element	ARM CCA: hardware realm; TPM: discrete chip
Supported curves	P-256 only	P-256, Ed25519 (some), AES	Varies widely; TPM 2.0: P-256, AES
BLS12-381 support	No	No	No
AES-256-GCM	Via SecKey wrapping	Direct StrongBox support	TPM: via HMAC sealing; CCA: direct
Key generation speed	<100ms	~9s (StrongBox HAL)	TPM: ~500ms; CCA: <100ms
Biometric binding	Face ID / Touch ID gates key access	BiometricPrompt gates key access	Windows Hello / fingerprint reader

Feature	iOS Secure Enclave	Android StrongBox	Laptop (ARM CCA / TPM)
Attestation	DeviceCheck, App Attest	Key Attestation, Play Integrity	TPM remote attestation
Key export	Never	Never (kSecAttrExtractable = false)	Sealed to PCR state

12.2 Unified Wrapping Strategy

All platforms use the same approach: **TEE provides AES-256-GCM symmetric wrapping; BLS12-381 material is encrypted, not natively held.**

iOS: SecKey(P-256) → ECDH-with-self → AES-256-GCM key → wrap(seed)

Android: StrongBox AES-256-GCM key → wrap(seed)

Laptop: TPM HMAC key + sealed policy → AES-256-GCM key → wrap(seed)

ARM CCA: AES-256-GCM key in realm → wrap(seed)

12.3 Platform-Specific Notes

iOS: The Secure Enclave generates a P-256 key pair. To get a symmetric wrapping key, the app performs ECDH between the Secure Enclave's key and an ephemeral P-256 key, producing a shared secret used as AES-256-GCM key material. The wrapped seed blob goes into the iOS Keychain with `kSecAttrAccessible = kSecAttrAccessible-WhenUnlockedThisDeviceOnly`, optionally gated by biometric authentication.

Android: StrongBox supports AES-256-GCM key generation directly. The key is created with `setIsStrongBoxBacked(true)` and `setUserAuthenticationRequired(true)`. Key generation is slow (~9 seconds) but only happens once at onboarding. The wrapped seed goes into the Android Keystore. Devices without StrongBox fall back to the TEE-backed Keymaster.

Laptop: On ARM platforms with CCA (Confidential Compute Architecture), a hardware-isolated realm gives direct AES-256-GCM capability. On Intel/AMD, TPM 2.0 provides HMAC-based sealing – the seed is encrypted with an AES key derived from `TPM2_HMAC(sealKey, seed_nonce)`, and the sealed object is bound to the platform's PCR state (it cannot be decrypted if the boot chain is tampered with).

12.4 Cross-Platform Recovery

TEE-wrapped blobs are not portable: an iOS-wrapped seed cannot be unwrapped on Android. Cross-platform migration always goes through DeRec:

1. Old device's seed is already shared via DeRec (3,5) threshold.
2. New device (any platform) initiates recovery.
3. Three helpers provide their shares.
4. Seed is reconstructed in new device's TEE.
5. New device wraps seed with its own platform's TEE key.
6. New device key is added to the account; old device key is revoked.

The user's username.midnight identity, wallet addresses, and credentials stay the same. Only the TEE wrapping layer differs.

12.5 Privacy-Preserving Analytics

Onboarding funnel metrics are essential for product decisions — but Midnight cannot collect them the way a normal app would. No wallet addresses, no device fingerprints, no timestamps tied to users. The analytics system has to produce aggregate counts without ever seeing individual behavior.

What to Measure The onboarding funnel has clear stages, and each stage has a drop-off risk:

Stage 1: QR scan / pairing	--> Did the session establish?
Stage 2: Seed generation	--> Did key creation complete?
Stage 3: Wallet setup	--> Are all three layers derived?
Stage 4: Name registration	--> Did commit-reveal succeed?
Stage 5: Name confirmed	--> Is the name on-chain?
Stage 6: Identity (zkKYC)	--> Did user opt in? Did verification pass?
Stage 7: DeRec helper pairing	--> Are 5 helpers paired?
Stage 8: First transaction	--> Did user send/receive successfully?

The key metrics: funnel completion rate by stage, drop-off rates at each transition, proof generation latency (p50 and p95), helper pairing success rate, and name registration retry rate (how often commit-reveal needs a second attempt due to name conflicts).

How to Measure Without PII The mechanism is on-chain aggregate counters implemented as a Compact circuit. Each counter is a single global state value that can only be incremented, never decremented, never queried per-user.

```
export circuit increment_stage_counter(stage: Field): [] {
  // Read current count for this stage
  const current = get_counter(stage);
  // Increment by 1
  set_counter(stage, current + 1);
  // The circuit proves the increment is valid
  // without revealing WHO incremented it
}
```

The counter circuit accepts a stage number (1-8) and increments the global count. The ZK proof guarantees that the caller completed the relevant stage (e.g., Stage 5 requires a valid name registration proof as input) without revealing any identifying information about the caller. The on-chain state after N users complete Stage 5 is simply the number N — no addresses, no timestamps, no device IDs.

To prevent double-counting, each counter increment consumes a nullifier derived from the user's secret key and the stage number: `nullifier = persistentHash("analytics:", sk, stage)`. A user who completes Stage 5 twice (e.g., on two devices) only increments the counter once.

Reading the Counters Anyone can read the aggregate counters. They are public on-chain state. A dashboard query looks like:

```
Stage 5 completions (all time):    142,387
Stage 6 opt-ins (all time):        98,201
Stage 8 successes (all time):      87,445
```

Drop-off: Stage 5 -> 6: 31.0%

Drop-off: Stage 6 -> 8: 11.0%

Daily resolution is possible by deploying epoch-bucketed counters (one counter per stage per day). The epoch is derived from block height ranges, not wall-clock time, so no timestamp oracle is needed.

Opt-In Local Telemetry For more granular diagnostics — proof generation latency, network retry counts, UI interaction timing — the app can collect telemetry locally on the device. This data never leaves the device by default. It is stored in the app's local database, not transmitted to any server.

If the user explicitly opts in, the app can export an anonymized aggregate summary: “My average proof time was 16.2s, I retried name registration 2 times, helper pairing took 3 attempts.” This export contains no keys, no addresses, no account identifiers. The user can review the export before sharing it.

KPIs Tied to the Customer Journey The three numbers that matter most for onboarding health map directly to the on-chain counters:

KPI	Counter	What it tells you
Name registration rate	Stage 5 completions / Stage 1 starts	Are users making it through the core flow?
zkKYC opt-in rate	Stage 6 opt-ins / Stage 5 completions	Is the identity layer compelling or scary?
First-tx success rate	Stage 8 successes / Stage 5 completions	Can users actually use the wallet they just set up?

These KPIs are computable by anyone reading the public counters. No analytics vendor, no tracking SDK, no data processor. The chain is the analytics backend.

13. Appendix A — Security Review Summary

All security findings from the cryptographic review (10 findings) and protocol analysis (23 findings) in one table.

13.1 Cryptographic Findings

ID	Severity	Category	Finding	Mitigation	Status
C-01	CRITICAL	Key Management	BIP39 seed plaintext in app memory during derivation	Chain-level key caching; mlock(); explicit_bzero(); <100ms exposure window	Mitigated
C-02	HIGH	Curve Mismatch	iOS Secure Enclave P-256 cannot hold BLS12-381 keys	AES-256-GCM symmetric wrapping; Secure Enclave as key-wrapping service only	Mitigated
C-03	HIGH	Key Derivation	Single seed controls all wallet layers — compromise = total loss	DeRec (3,5) threshold recovery; BIP-85 app-level compartmentalization option	Mitigated
C-04	HIGH	Hash Function	persistentHash (Poseidon) security assumptions differ from SHA-256	Poseidon is designed for arithmetic circuits; security proofs exist for BLS12-381 field; monitor cryptanalysis	Accepted
C-05	MEDIUM	Quantum Resistance	BLS12-381 signing vulnerable to Shor's algorithm	DeRec transport uses ML-KEM (post-quantum); monitor NIST PQ signature standards for migration path	Monitoring
C-06	MEDIUM	Entropy	BIP39 mnemonic quality depends entirely on platform RNG	iOS: SecRandomCopyBytes (hardware RNG); Android: StrongBox RNG; health checks at generation time	Mitigated

ID	Severity	Category	Finding	Mitigation	Status
C-07	MEDIUM	Derivation	PBKDF2 with empty passphrase reduces to single SHA-512 iteration	Seed is 256-bit entropy (not password-derived); PBKDF2 iteration count is irrelevant for high-entropy input	Accepted
C-08	LOW	Nullifiers	Domain separation strings are short and could collide	Standardized 6-byte prefixes with registry; collision probability negligible for planned domain count	Accepted
C-09	LOW	Merkle Trees	Depth-1 trees (2 leaves) in examples limit scalability	Production trees should use depth 20+ (1M+ leaves); tree depth is a deployment parameter, not a protocol limitation	Accepted
C-10	INFO	Compiler	Compact disclosure analysis enforces privacy at compile time	No mitigation needed — this is a security strength; contributes to defense-in-depth	N/A

13.2 Protocol Findings

ID	Severity	Category	Finding	Mitigation	Status
P-01	HIGH	QR Protocol	Relay attack: attacker forwards QR content to remote device in real-time	60s timestamp window; visual confirmation code (4-digit out-of-band); single-use session_id	Mitigated

ID	Severity	Category	Finding	Mitigation	Status
P-02	HIGH	QR Protocol	Shoulder-surfing: physical capture of QR code image	Short validity window (60s); server-side session invalidation after first use; rate limiting	Partially Mitigated
P-03	HIGH	QR Protocol	Compromised onboarding server generates malicious QR codes	Server key pinned in app binary; certificate transparency monitoring; multiple server redundancy	Mitigated
P-04	HIGH	Recovery	DeRec helper collusion (3 of 5 collude to steal seed)	Social diversity requirement for helper selection; resharing limits window; monitoring for simultaneous access	Partially Mitigated
P-05	HIGH	Recovery	All helpers simultaneously unavailable	Encourage geographic and social diversity; backup export (encrypted) as secondary recovery path	Partially Mitigated
P-06	MEDIUM	Account Model	Malicious dApp drains DUST via function-call key	DUST allowance caps; automatic revocation on limit breach; user notification at 80% usage	Mitigated
P-07	MEDIUM	Account Model	Race condition in concurrent key addition/removal	Transaction ordering via nonce; atomic add-and-remove for rotation; conflict detection in circuit	Mitigated

ID	Severity	Category	Finding	Mitigation	Status
P-08	MEDIUM	Naming	Frontrunning name registration despite commit-reveal	Commit hidden behind persistentHash; 60s wait period; penalty for abandoned commits	Mitigated
P-09	MEDIUM	Naming	Homoglyph attack: aI-ice.midnight vs aI-ice.midnight	Unicode normalization (NFC); confusable detection in app UI; human review for disputed names	Partially Mitigated
P-10	MEDIUM	Identity	zkMe issuer compromise: false attestations	Multiple issuer support in attestation tree design; issuer rotation; on-chain issuer reputation	Partially Mitigated
P-11	MEDIUM	Identity	Merkle proof replay after credential revocation	Credential revocation updates Merkle root; old proofs fail against new root; revocation event indexable	Mitigated
P-12	MEDIUM	Latency	18-21s proof generation degrades onboarding UX	Parallel proof generation where possible; progress indicators; server-side pre-computation for subsidized txs	Partially Mitigated
P-13	MEDIUM	Privacy	Timing correlation between proof submission and verification	Batched proof submission; randomized delays; mixer-style submission patterns	Partially Mitigated

ID	Severity	Category	Finding	Mitigation	Status
P-14	LOW	QR Protocol	QR code too large for camera to scan reliably	Binary encoding; fallback to deep link / NFC tap	Mitigated
P-15	LOW	Account Model	Orphaned function-call keys (dApp no longer exists)	Expiration TTL on function-call keys; periodic cleanup circuit	Mitigated
P-16	LOW	Recovery	DeRec share corruption during storage	Merkle commitment enables corruption detection; re-request from user if verification fails	Mitigated
P-17	LOW	Recovery	Epoch resharing interrupted by network partition	Resume protocol; shares from previous epoch remain valid until new epoch completes	Mitigated
P-18	LOW	Identity	SBT metadata leaks verification status	SBT stores only verified: bool and nullifier; no PII in token metadata; encrypted optional fields	Mitigated
P-19	LOW	Cross-Platform	Android StrongBox key generation latency (~9s)	One-time cost at onboarding; progress indicator; background generation	Accepted
P-20	LOW	Cross-Platform	TPM sealed objects tied to boot state — OS update breaks seal	Version-tolerant sealing policy; fallback to DeRec recovery on seal failure	Mitigated

ID	Severity	Category	Finding	Mitigation	Status
P-21	INFO	Privacy	Midnight's disclosure analysis prevents accidental witness leaks	Compile-time enforcement is a major security advantage over runtime-only protections	N/A
P-22	INFO	Architecture	Witness values never leave the local proof server	Architectural guarantee: proof pipeline is local-only; no witness exfiltration path in SDK	N/A
P-23	INFO	Fees	DUST regeneration from NIGHT enables fee-free UX after initial airdrop	Developer subsidy model is sustainable; DUST is non-transferable (no market manipulation)	N/A

14. Appendix B — Research Sources

Eight research streams fed into this design. Each contributed specific architectural decisions, protocol choices, or security analysis.

14.1 Research Stream Index

Stream	Focus Area	Key Contributions to Design
CAKE (Chain Abstraction Key Encapsulation)	Cross-chain intent routing, MPC chain signatures	Chain abstraction layer design; intent-based transaction model; MPC key management patterns
OpenWallet Foundation	Wallet architecture, credential formats, interoperability standards	Credential format decisions; wallet SDK patterns; DID method selection
DeRec (Decentralized Recovery)	Social recovery protocol, Shamir SS, helper verification	Recovery architecture (Section 10); (3,5) threshold design; daily verification protocol; epoch resharing

Stream	Focus Area	Key Contributions to Design
NEAR Protocol	Multi-key account model, named accounts, function-call keys	Account model (Section 7); device key management; function-call key scoping; account-stable identity
zkMe	zkKYC, Groth16 proofs, FHE biometrics, Soulbound Tokens	Identity layer (Section 8); off-chain issuer design; SBT anchoring; FHE biometric processing pipeline
ENS (Ethereum Name Service)	Human-readable naming, commit-reveal registration, CCIP-Read	Naming system (Section 4); multi-address resolution; wildcard subdomains; name wrapper permissions
TEE (Trusted Execution Environments)	Secure Enclave, StrongBox, ARM CCA, TPM	Key management (Section 9); platform-specific wrapping; TEE trust boundary; cross-platform strategy
Midnight Platform Docs	Compact, ZKIR, wallet SDK, attestation trees, proof pipeline	All Midnight-native design decisions: attestation tree construction, three-layer wallet, witness authorization, Bech32m addressing, BLS12-381/Jubjub curves

14.2 Primary References

CAKE / Chain Abstraction

- CAKE Framework: Chain Abstraction Key Encapsulation specification (2024)
- MPC Chain Signatures: Threshold ECDSA/EdDSA for cross-chain signing
- Intent-based transaction routing architecture

OpenWallet Foundation

- OpenWallet Foundation Architecture SIG: Wallet engine specification
- Verifiable Credentials Data Model 2.0 (W3C)
- DID Core Specification 1.0 (W3C)

DeRec (Decentralized Recovery)

- DeRec Protocol Specification v0.9
- Shamir’s Secret Sharing: “How to Share a Secret” (Shamir, 1979)
- ML-KEM (FIPS 203): Module Lattice-Based Key Encapsulation Mechanism
- Proactive Secret Sharing: epoch-based resharing protocols

NEAR Protocol

- NEAR Account Model documentation
- NEP-366: Meta Transactions (gasless transactions)

- NEP-364: Function-Call Access Keys
- NEAR Multi-key account architecture whitepaper

zkMe

- zkMe zkKYC Whitepaper: Privacy-preserving identity verification
- Groth16: On the Size of Pairing-based Non-interactive Arguments (Groth, 2016)
- Soulbound Tokens: EIP-5192 (minimal Soulbound NFTs)
- FHE biometric verification pipeline

ENS (Ethereum Name Service)

- EIP-137: Ethereum Domain Name Service (namehash specification)
- EIP-181: ENS Reverse Resolution
- ENSIP-10: Wildcard Resolution (CCIP-Read)
- ENS Name Wrapper: permission fuse system

TEE (Trusted Execution Environments)

- Apple Secure Enclave documentation (iOS Security Guide)
- Android StrongBox Keymaster HAL specification
- ARM Confidential Compute Architecture (CCA) specification
- TPM 2.0 Library Specification (Trusted Computing Group)
- Tempo/Passkeys investigation: WebAuthn + passkey integration patterns

Midnight Platform

- Midnight Developer Guide (assembled, verified against devnet)
- Compact Language Reference (formal grammar + compilation evidence)
- Wallet SDK Reference (13-package API surface, 95 passing tier-1 tests)
- ZKIR Reference (circuit constraints, BLS12-381 field operations)
- SDK Reference (transaction pipeline, providers)
- Midnight Architecture: hybrid UTXO + account model
- BLS12-381 / Jubjub curve specifications (as used in Midnight's Zswap layer)
- BIP-32 (HD wallets), BIP-39 (mnemonics), BIP-44 (multi-account), CIP-1852 (Cardano-style derivation)
- BIP-340 (Schnorr signatures for unshielded UTXO spending)
- Bech32m address encoding (BIP-350, adapted for mn_ prefix)

14.3 Standards References

Standard	Used For
BIP-32	Hierarchical deterministic key derivation
BIP-39	Mnemonic generation (24 words, 256-bit entropy)

Standard	Used For
BIP-44	Multi-account derivation paths (coin type 2400)
BIP-85	Deterministic entropy derivation (compartmentalization option)
BIP-340	Schnorr signatures for unshielded UTXO spending
BIP-350	Bech32m address encoding
CIP-1852	Cardano-style HD wallet derivation (adapted for Midnight's 5 roles)
FIPS 197	AES-256 (used in GCM mode for TEE wrapping)
FIPS 203	ML-KEM (post-quantum key encapsulation for DeRec transport)
NIST SP 800-56A	ECDH key agreement (QR protocol Phase 1)
NIST SP 800-108	HKDF key derivation (session key from ECDH shared secret)
RFC 8032	EdDSA (reference for digital signature patterns)
SLIP-44	Registered coin types (Midnight = 2400)
EIP-137	Namehash algorithm (adapted for persistentHash)
EIP-5192	Soulbound Token interface (adapted for Midnight SBTs)

15. Appendix C - Customer Journey Map: Newcomer Nadia

Persona: Nadia, 28, Nairobi. Uses M-Pesa daily for payments and savings groups. Comfortable with mobile apps but has zero blockchain experience. Heard about Midnight from a friend who described it as “M-Pesa but you control your own money and nobody can see your balance.” Has a mid-range Android phone with StrongBox support.

Goal: Go from zero state to sending a shielded payment to her friend, in one session.

15.1 Journey Stages

#	Stage	Actions	Touchpoints	Emotion (1-10)	Pain Points	Opportunities
1	Awareness	Friend shows Nadia a shielded payment on their phone; Nadia asks "how do I get this?"	Word of mouth; friend's device screen	7 - Curious, intrigued	No clear next step from conversation alone; friend may not remember exact app name	Shareable referral deeplink so friend can text a direct download link; referral incentive (small NIGHT airdrop for both parties)
2	Download	Searches app store, downloads Midnight wallet app (~45 MB), opens it	Google Play Store; app landing screen	6 - Neutral, slight impatience	App name ambiguity (other "midnight" apps exist); download time on slower connections; storage permission prompts	Distinctive app icon and verified publisher badge; minimal permissions at install (camera only requested at QR step); friend's deeplink bypasses search entirely
3	QR Scan	App displays "Scan to begin" screen; Nadia scans QR code shown by onboarding kiosk or friend's referral screen	Device camera; QR code (kiosk, web page, or friend's phone)	6 - Familiar (M-Pesa uses QR)	Camera permission prompt may confuse if unexpected; poor lighting or cracked screen protector can fail scan; no fallback visible	QR pattern is familiar from M-Pesa till/Lipa Na M-Pesa; fallback: tap-to-paste deeplink shown below QR; NFC tap option for newer phones

#	Stage	Actions	Touchpoints	Emotion (1-10)	Pain Points	Opportunities
4	Wallet Creation	ECDH channel established; BIP39 seed generated in StrongBox TEE; three wallet addresses derived; NIGHT airdrop + DUST generation initiated	Progress animation ("Setting up your wallet..."); ~3-5s on device	7 - Impressed by speed	StrongBox key generation can take up to 9s on some Android devices, causing anxiety if no feedback	Animated progress with stage labels ("Securing your keys... Creating your wallets... Almost ready"); pre-warm StrongBox during QR scan to overlap latency

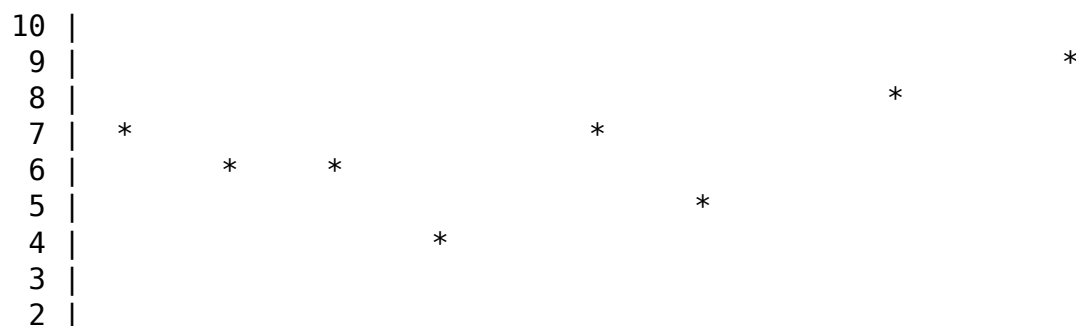
#	Stage	Actions	Touchpoints	Emotion (1-10)	Pain Points	Opportunities
5	Name Registration	Prompted to choose a name; types "na-dia.midnight"; commit transaction submitted; ~60s wait for reveal	Name input field; commit-reveal progress bar; countdown or spinner	4 - Frustrated, anxious (lowest point)	This is the #1 abandonment risk. 60 seconds feels like an eternity on mobile. User does not understand why it takes so long. "Did it crash?" anxiety. If user backgrounds the app, commit may be wasted.	Explain the wait honestly: "Securing your name on the network - about 1 minute"; show a progress bar with real block confirmations; gamify: display fun facts about Midnight privacy features during wait; pre-validate name availability before commit to avoid wasted waits; allow user to continue setup (identity, recovery) in parallel while commit resolves

#	Stage	Actions	Touchpoints	Emotion (1-10)	Pain Points	Opportunities
6	Identity Verification	Prompted: "Verify your identity (optional - enables more features)"; most users tap "Skip for now"	Modal with "Verify" and "Skip" buttons; if chosen: camera for OCR + liveness; ~15s FHE + Groth16 proof	5 - Confused about why, wary of surveillance	"Optional" framing causes 70%+ skip rate , but many services later require it, forcing a second interruption. Users from Kenya associate KYC with government surveillance. The word "verify" triggers distrust.	Reframe: "Unlock full features" instead of "Verify identity"; show concrete benefits ("Send larger amounts, access DeFi"); defer gracefully with a reminder after first transaction; if chosen, the ZK proof framing ("Your data stays on your phone - we only check a yes/no") can convert wary users. Consider making it a progressive unlock: basic features now, advanced after verification

#	Stage	Actions	Touchpoints	Emotion (1-10)	Pain Points	Opportunities
7	Recovery Setup	prompted to select 5 recovery helpers from contacts; Nadia picks 3 family members and 2 chama (savings group) friends; shares are distributed	Contact picker; helper invitation messages (SMS/WhatsApp); confirmation of 5/5 accepted	8 – Empowered, feels safe	Contacts may not have the app yet (cold-start problem); users may pick 5 people from same household (defeats geographic diversity); “helper” concept needs explanation	This stage resonates strongly with M-Pesa group savings (chama) patterns – Nadia already trusts a circle of 5+ people with money. Frame as “your financial safety circle”; warn if all helpers share a phone number prefix (same area); allow helpers without the app to hold an encrypted share via SMS link; daily silent health checks give ongoing confidence

#	Stage	Actions	Touchpoints	Emotion (1-10)	Pain Points	Opportunities
8	First Trans-action	Friend shares their address as “amina.midnight” Nadia types it in, enters amount, taps send; shielded payment completes in ~8s	Send screen; name resolution; transaction confirmation with animation	9 - Peak emotion: delight, accomplishment	Amount entry UX (which token? KES equivalent?); if friend’s name is misspelled, unclear error; first-time fee confusion (DUST is invisible but still consumed)	This is the emotional peak - the “aha” moment. Name-based sending feels like M-Pesa (“send to Amina” not “send to 0x7f3a...”); show KES equivalent alongside NIGHT amount; celebrate with a confirmation animation; prompt to share the experience (“Tell a friend”); DUST fees are invisible (subsidized) so the “free” feeling reinforces adoption

15.2 Emotion Curve



Priority	Recommendation	Addresses Stage
P1	Show KES equivalent alongside token amounts	8
P2	Pre-warm StrongBox during QR scan to hide key generation latency	4
P2	Warn when all recovery helpers share geographic proximity	7
P2	Celebration animation + share prompt after first transaction	8
P3	Fun facts / privacy education interstitial during 60s wait	5
P3	Allow recovery helpers without the app (encrypted SMS share)	7